

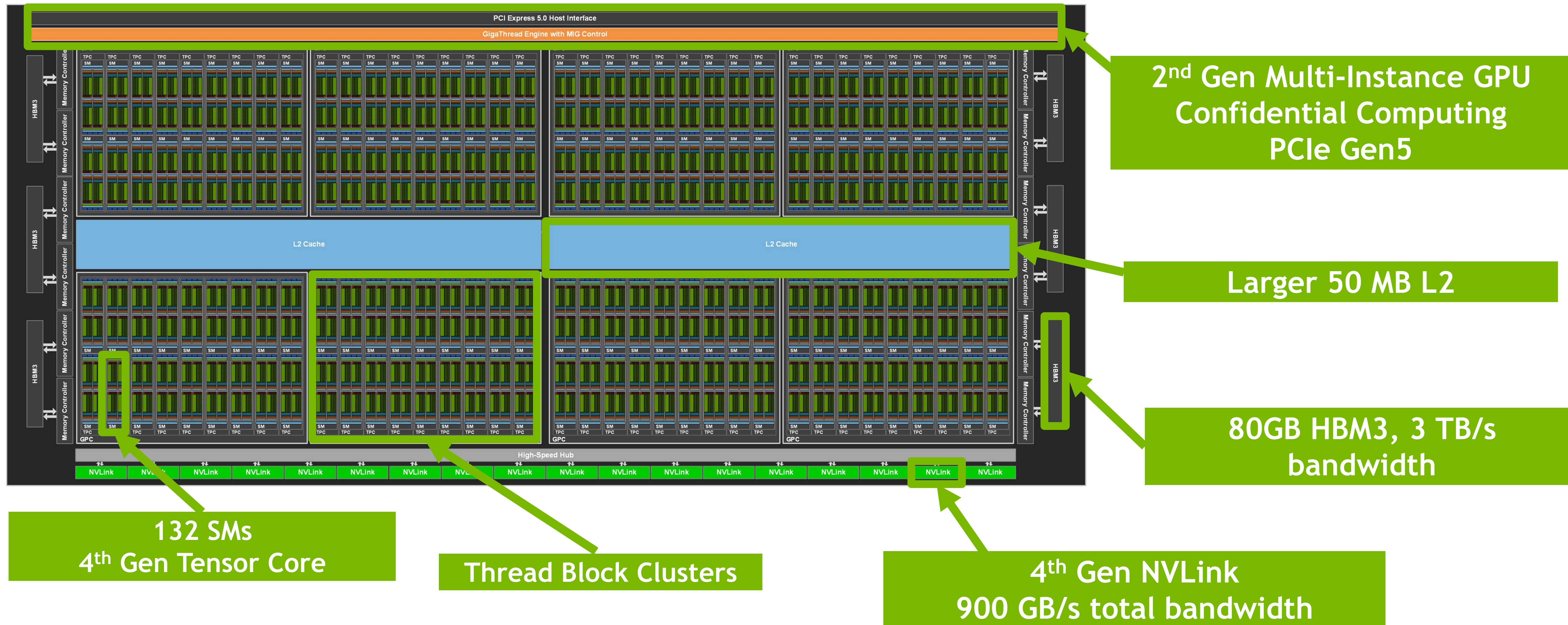


CUDA Programming Model for Hopper Architecture

Vishal Mehta, DevTech Compute | GTC Fall, September 2022

Hopper Architecture

H100 GPU Key features



See also:

[Enabling Hopper-Specific Optimizations in CUDA Applications \(A41147\)](#)

[CUDA: New Features and Beyond \(A41100\)](#)

Hopper Architecture

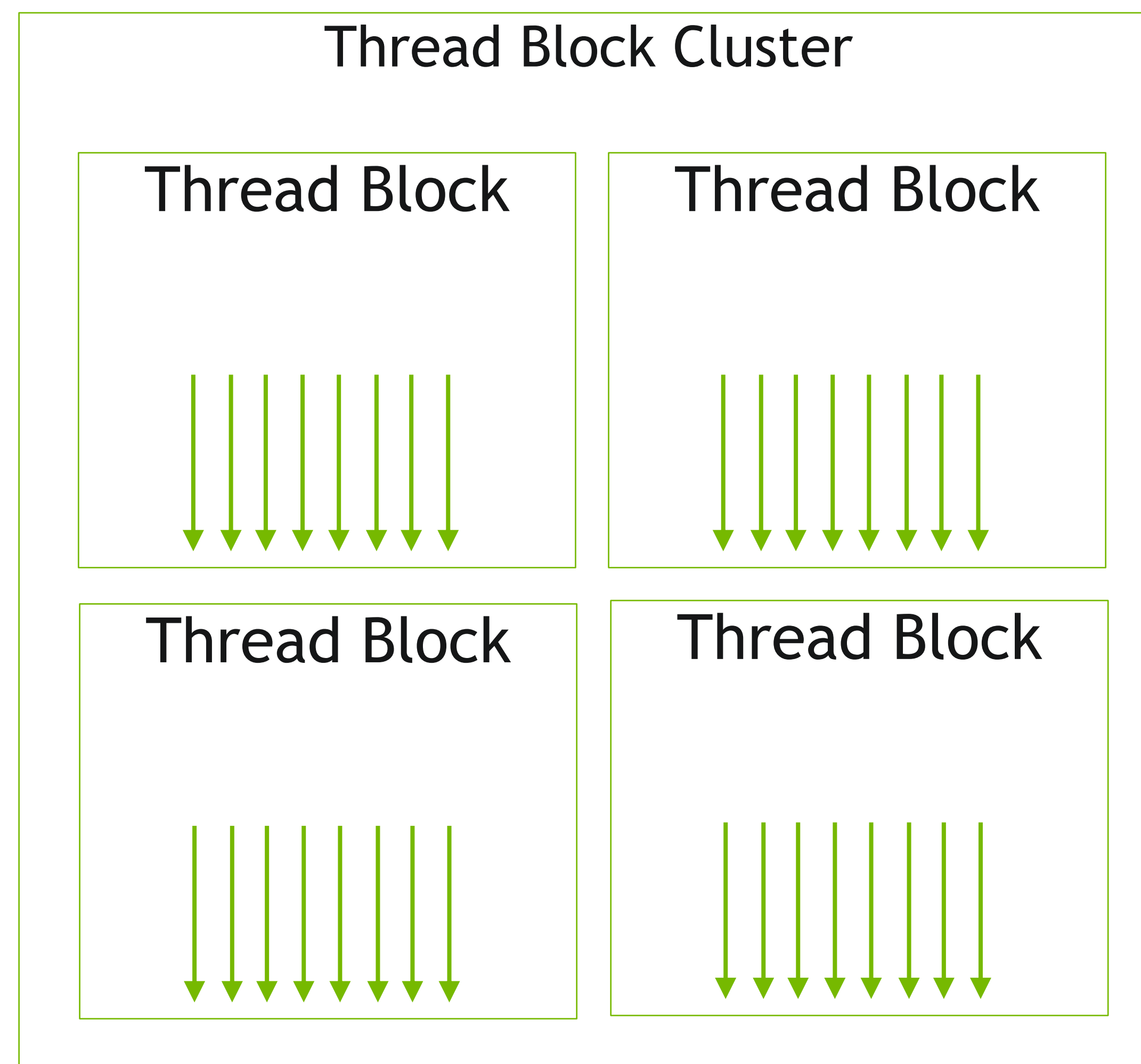
H100 Streaming Multiprocessor Key Features



- 256 KB combined L1 cache/Shared memory per SM. 33% over A100
- New Thread Block Clusters and Distributed Shared Memory
- New Tensor Memory Accelerator and Asynchronous Transaction Barriers
- DPX instructions to accelerate Dynamic Programming

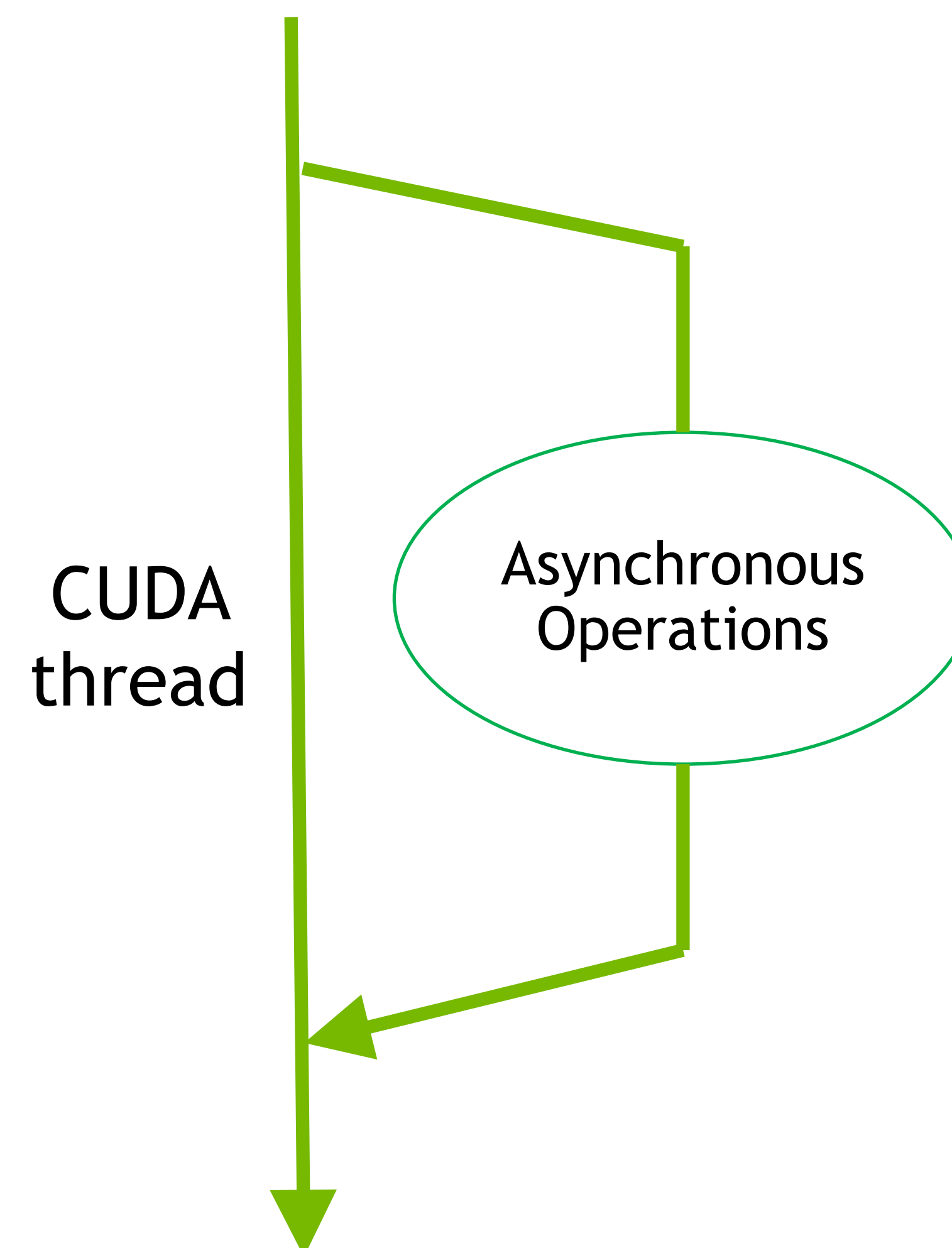
CUDA Programming Model for Hopper

How to write more scalable algorithms ?



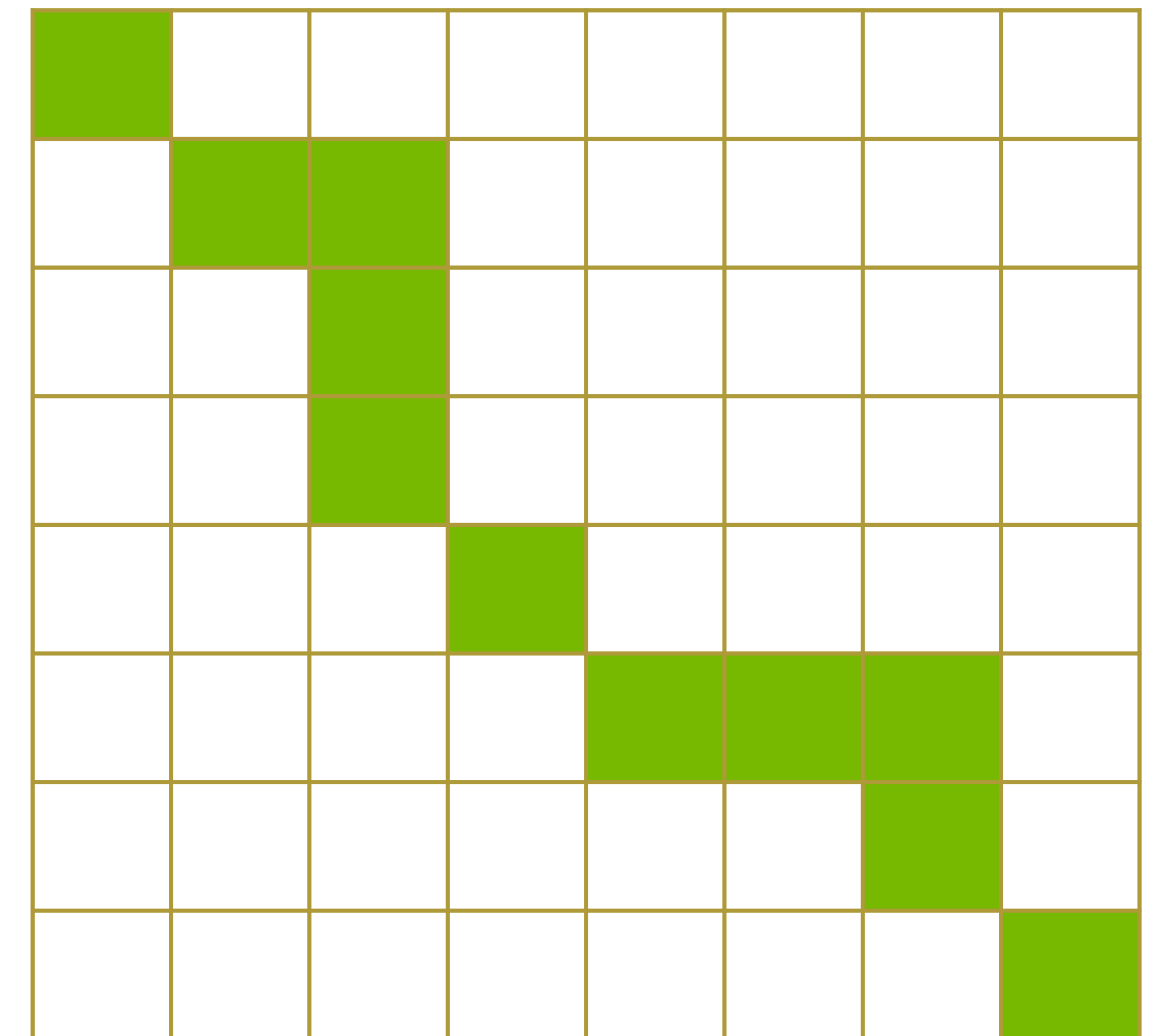
Hierarchy

How to keep all GPU units busy ?



Asynchronous SIMT Programming

How to accelerate computations in dynamic programming ?



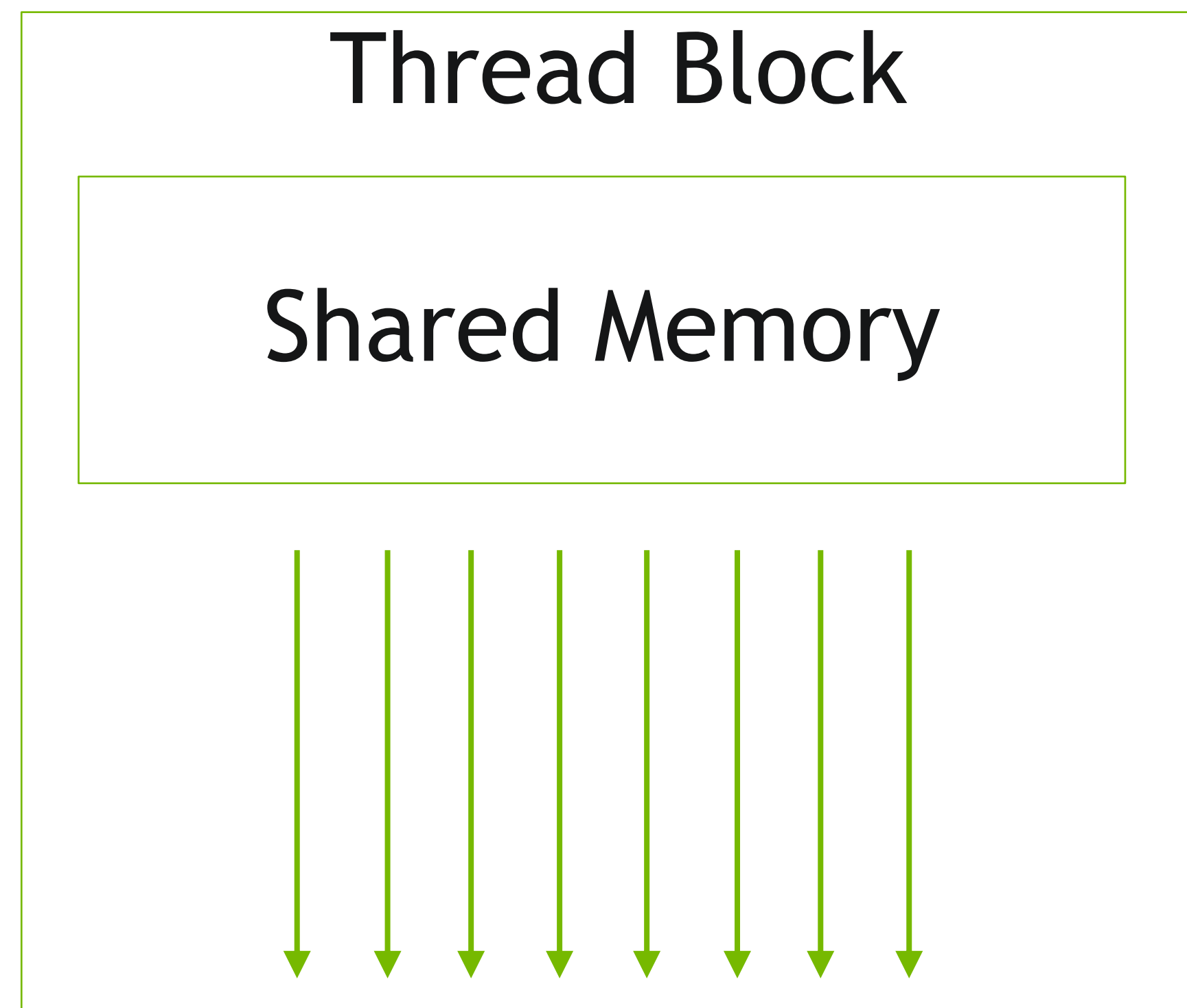
Dynamic Programming

The background of the slide is a black field filled with numerous thin, curved, and slightly blurred lines in shades of green and yellow. These lines create a sense of motion and depth, resembling a stylized representation of data flow or a microscopic view of a material. The lines are more concentrated in the lower right quadrant, where they form a more complex, almost crystalline structure, and become sparser towards the top left.

Scaling CUDA Programming Model using Thread Block Clusters

Thread and Memory Hierarchy

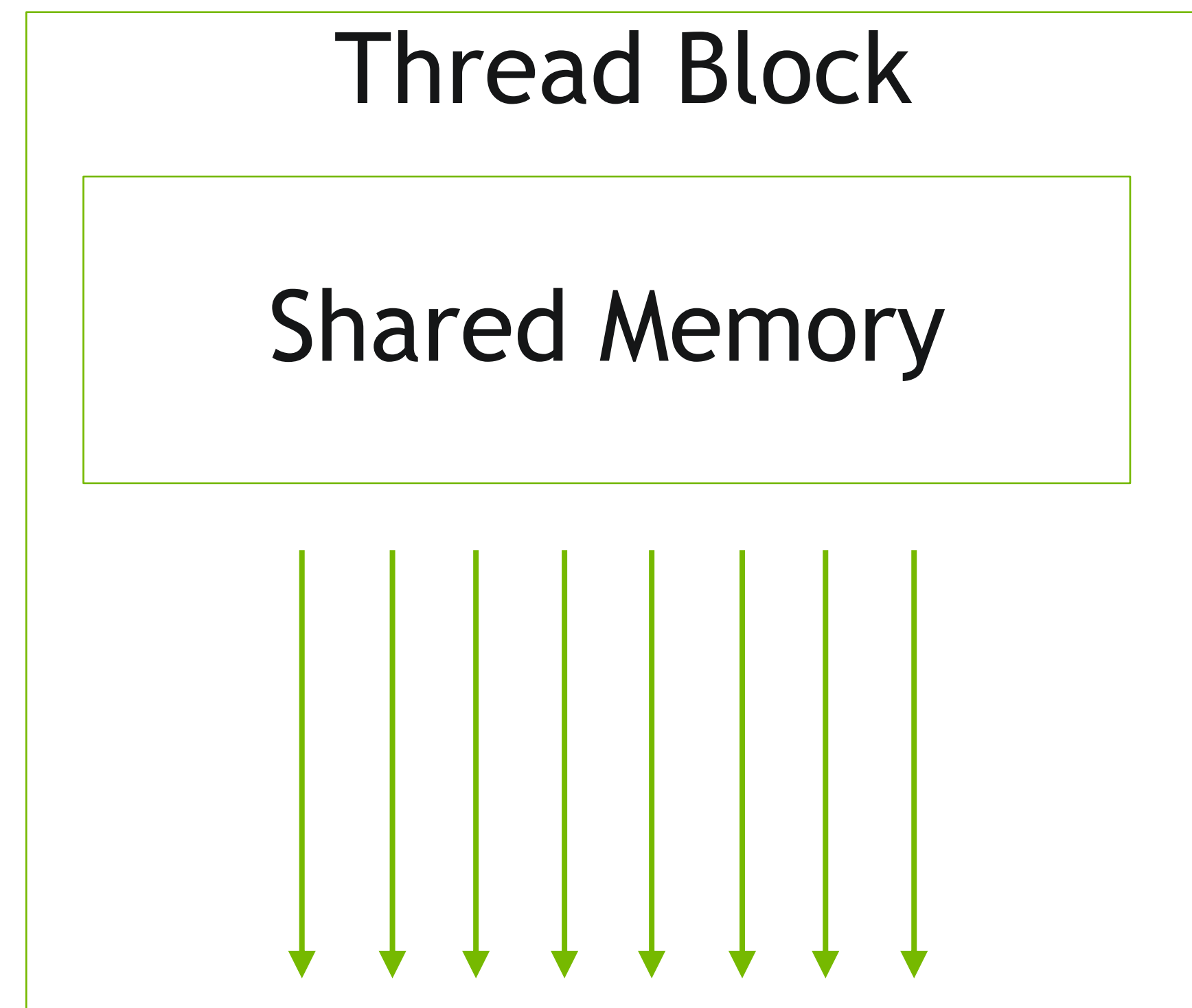
CUDA Thread & Memory Hierarchy pre-Hopper



- All threads in a thread block can collaborate using **shared memory**.
- All threads in a thread block **are guaranteed to be co-scheduled** on a Streaming Multiprocessor (SM).

Thread and Memory Hierarchy

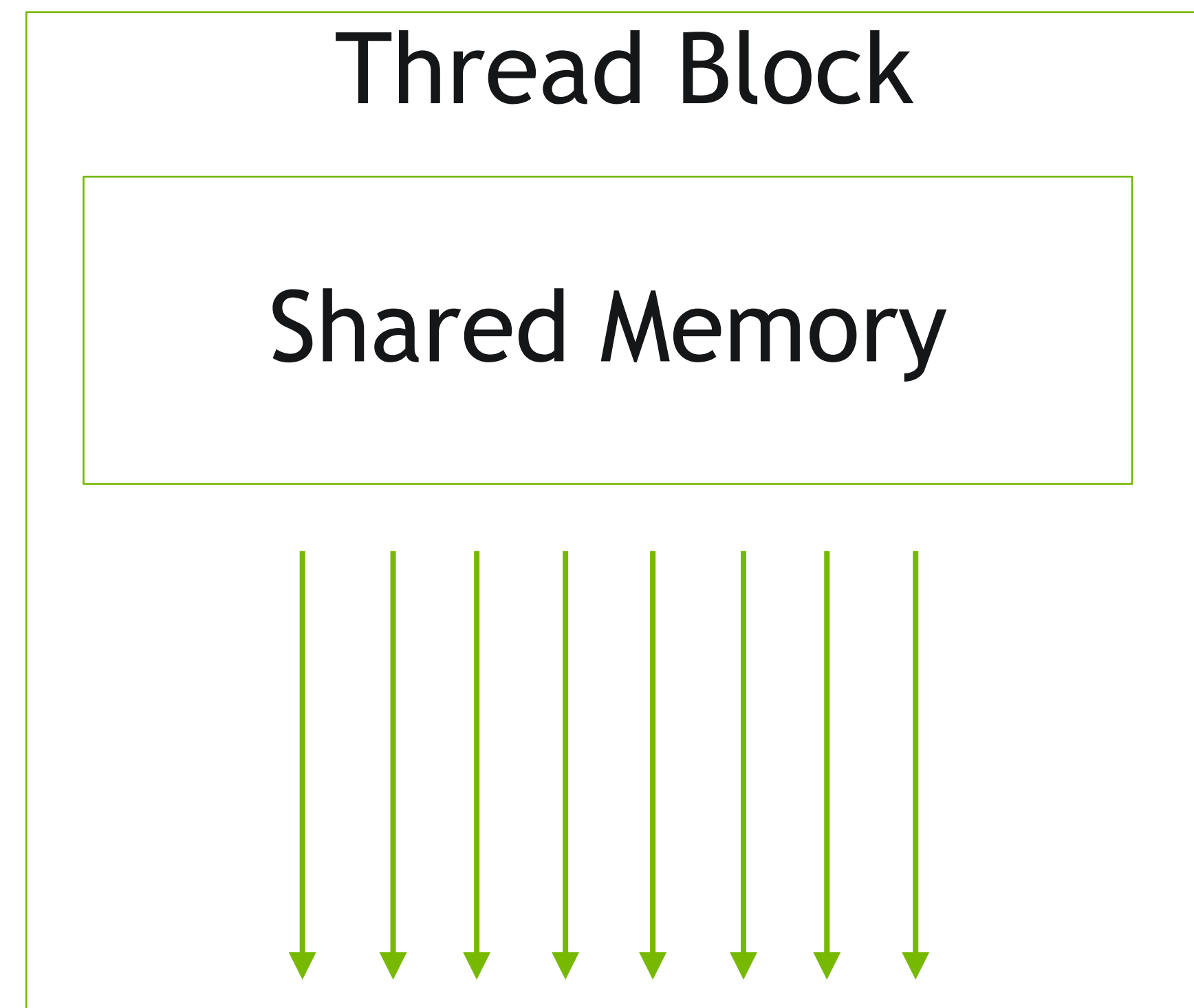
CUDA Thread & Memory Hierarchy pre-Hopper



- All threads in a thread block can collaborate using **shared memory**.
- All threads in a thread block **are guaranteed to be co-scheduled** on a Streaming Multiprocessor (SM).
- Threads in a thread block can synchronize / communicate data using
 - `cooperative_groups::this_thread_block.sync();`
OR
`__syncthreads();`
 - `cuda::barrier<thread_scope_block>::arrive()` and `::wait()`

Thread and Memory Hierarchy

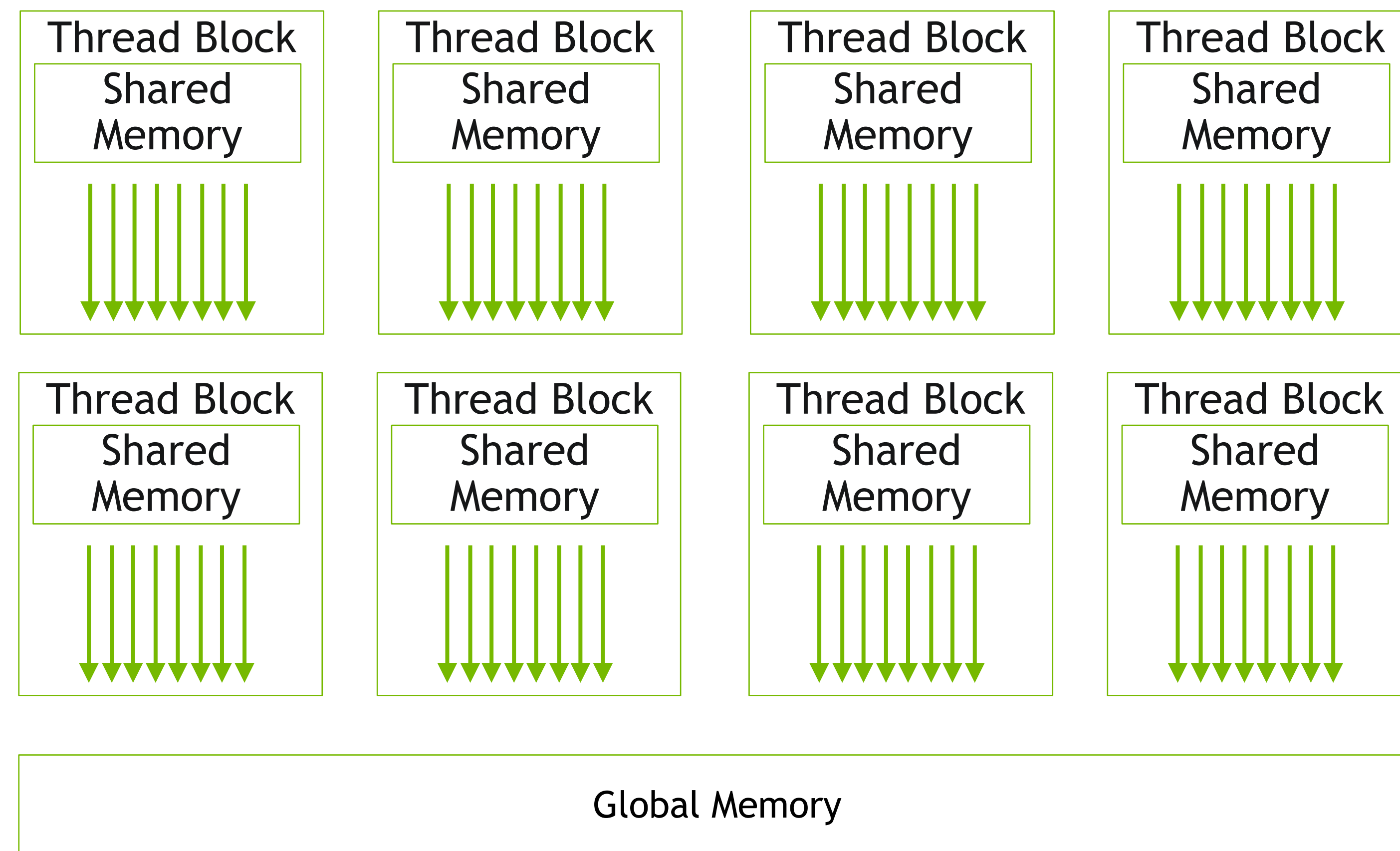
CUDA Thread & Memory Hierarchy pre-Hopper



- All threads in a thread block can collaborate using **shared memory**.
- All threads in a thread block **are guaranteed to be co-scheduled** on a Streaming Multiprocessor (SM).
- Threads in a thread block can synchronize / communicate data using
 - `cooperative_groups::this_thread_block.sync();`
OR
`__syncthreads();`
 - `cuda::barrier<thread_scope_block>::arrive()` and `::wait()`
- Threads in thread block can also perform collectives like `cooperative_groups::reduce()`

Thread and Memory Hierarchy

CUDA Thread & Memory Hierarchy pre-Hopper



- All thread blocks form the CUDA grid or CUDA kernel
- All thread blocks share **global memory** to collaborate
- Independent thread blocks can be scheduled out of order to improve occupancy, and hence GPU utilization

Scaling the CUDA Programming Model

Kepler GK110 GPU, 2012



15 SMs

SM

Hopper H100 GPU, 2022



132 SMs

SM

GPC

Scaling the CUDA Programming Model

Kepler GK110 GPU, 2012



15 SMs

SM

Hopper H100 GPU, 2022



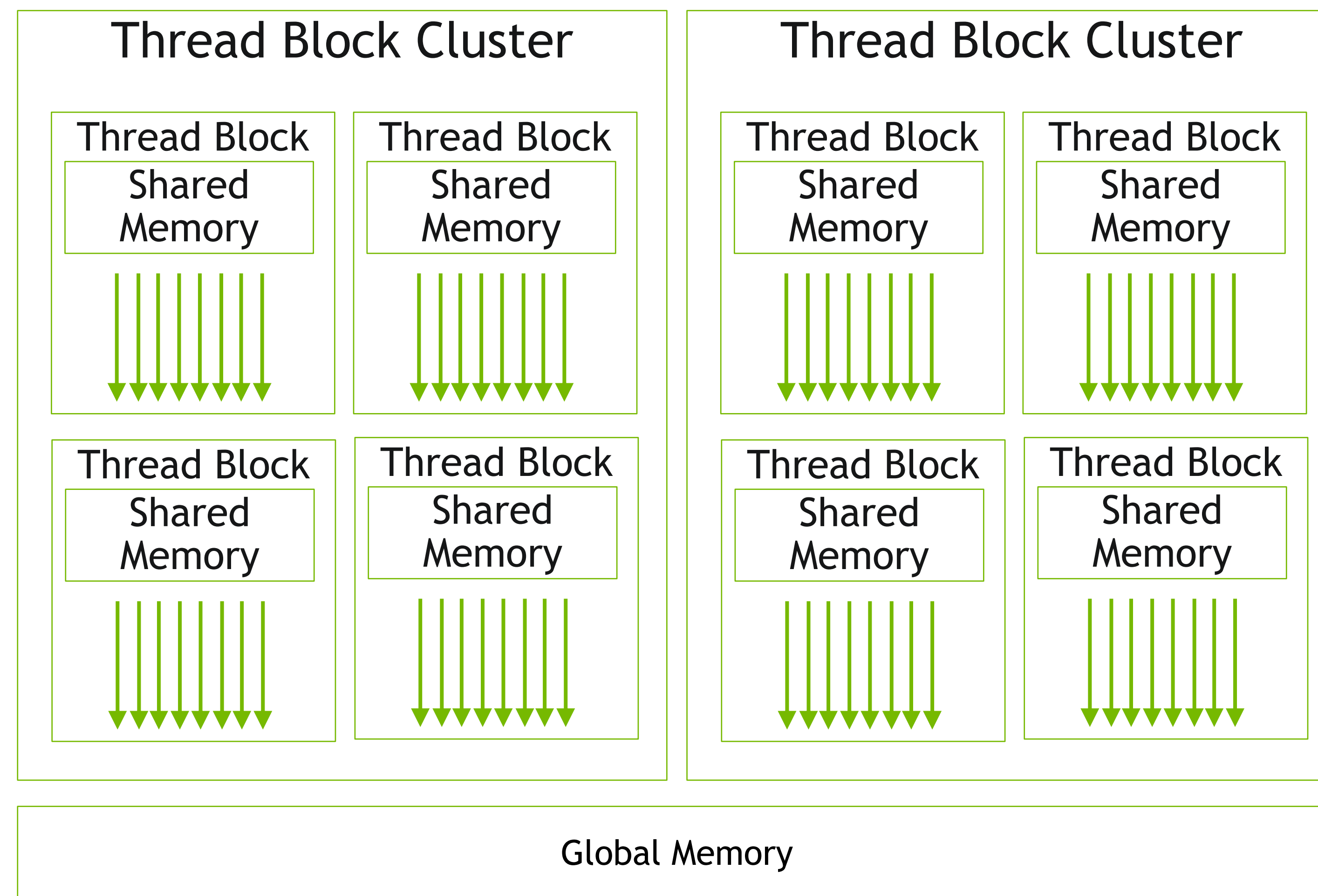
132 SMs

SM

GPC

Thread and Memory Hierarchy

Introducing Thread Block Clusters in Hopper



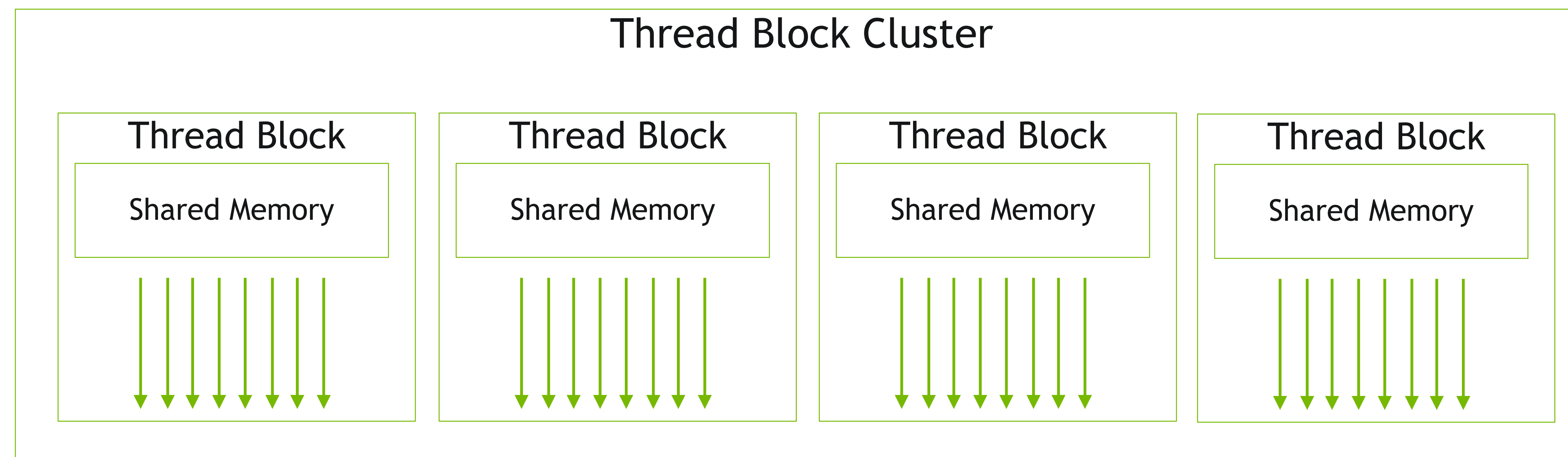
- Thread Block Cluster introduces a new **optional** level of **hierarchy** in the CUDA programming model.
- Thread blocks in a cluster are guaranteed to be co-scheduled on SMs in a GPU Processing Clusters (GPC)
- All shared memory within a cluster form the **Distributed shared memory**

Thread and Memory Hierarchy

Thread Block Cluster are available from CUDA Cooperative Groups

```
namespace cg = cooperative_groups;  
auto block = cg::this_thread_block();  
cg::cluster_group cluster = cg::this_cluster();
```

<...>



Thread and Memory Hierarchy

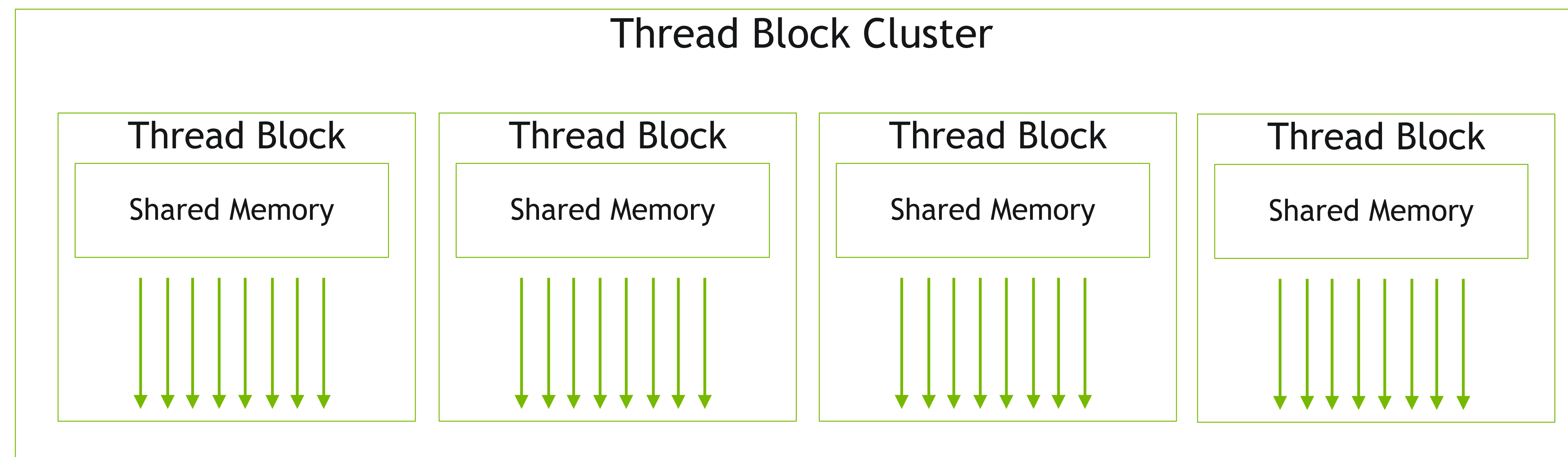
Accelerated Synchronization for all threads in cluster

```
namespace cg = cooperative_groups;  
cg::cluster_group cluster = cg::this_cluster();
```

<...>

```
cluster.sync();
```

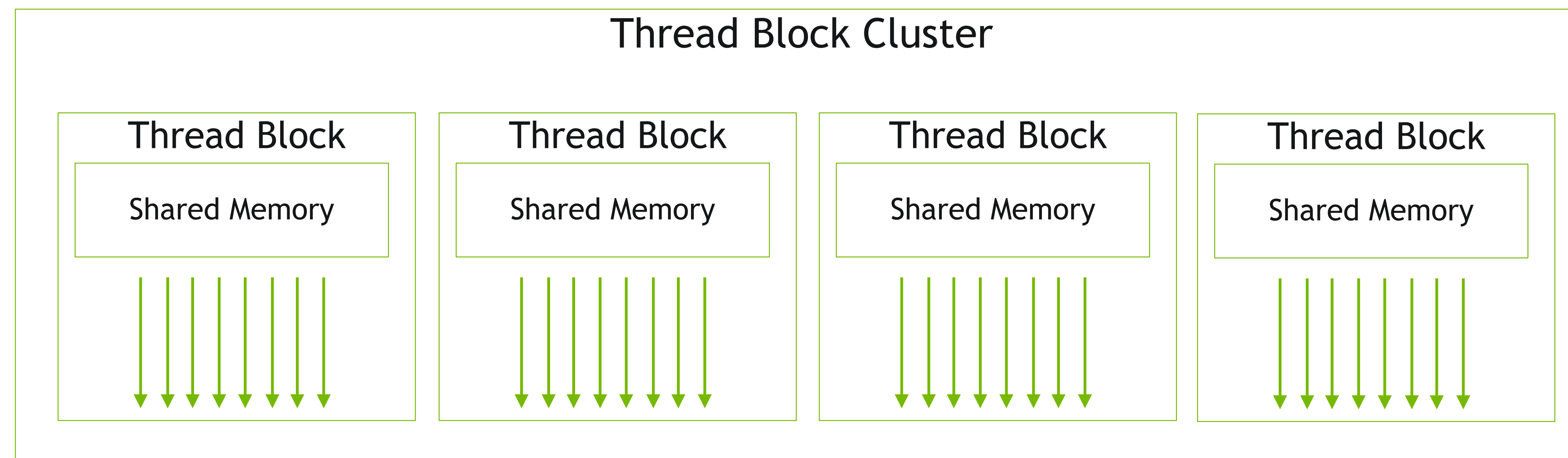
- **Cluster synchronization** is accelerated in hardware
- H100 can support up to **16 thread blocks** or **16384 threads** per cluster



Thread and Memory Hierarchy

Distributed Shared Memory Operations

- All blocks within a thread block cluster can collaborate using **Distributed shared memory**
- Thread blocks can **read, write and perform atomics** on each other's shared memory

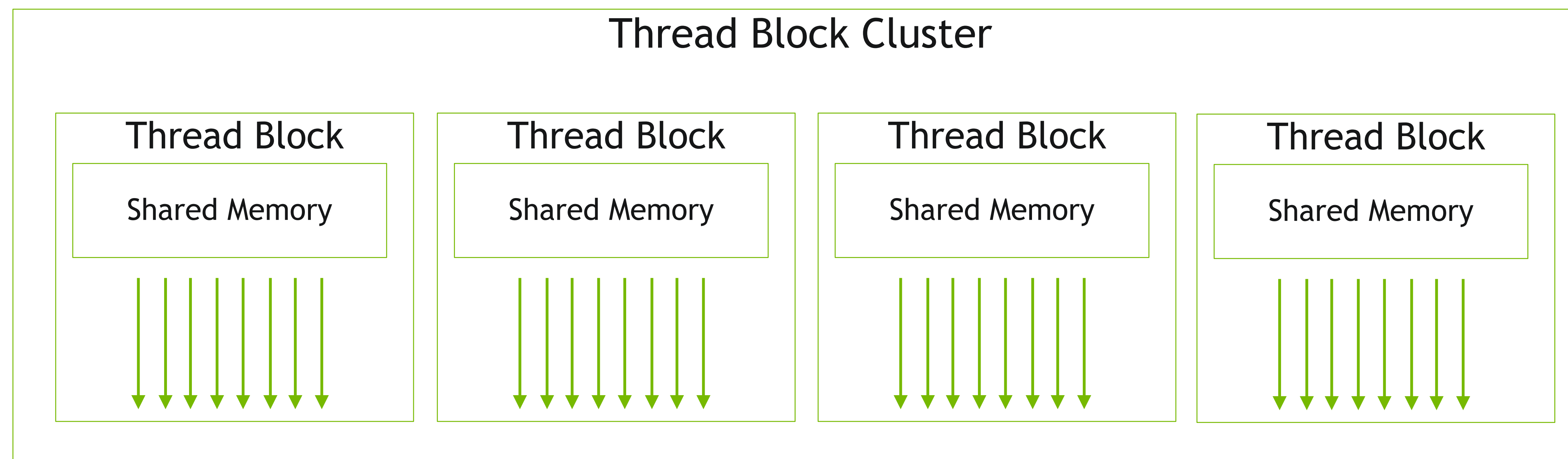


Thread and Memory Hierarchy

Distributed Shared Memory Operations

```
// All blocks in the cluster have the variable smem
__shared__ int smem;
namespace cg = cooperative_groups;
cg::cluster_group cluster = cg::this_cluster();
unsigned int BlockRank = cluster.block_rank();
int cluster_size = cluster.dim_blocks().x;
```

- All blocks within a thread block cluster can collaborate using **Distributed shared memory**
- Thread blocks can **read, write and perform atomics** on each other's shared memory



Thread and Memory Hierarchy

Distributed Shared Memory Operations

```
// All blocks in the cluster have the variable smem
```

```
__shared__ int smem;
```

```
namespace cg = cooperative_groups;
```

```
cg::cluster_group cluster = cg::this_cluster();
```

```
unsigned int BlockRank = cluster.block_rank();
```

```
int cluster_size = cluster.dim_blocks().x;
```

```
//Get a pointer to peer smem variable based on
```

```
//pointer from current block
```

```
int *remote_smem = cluster.map_shared_rank(&smem, (BlockRank + 1) %  
cluster_size);
```

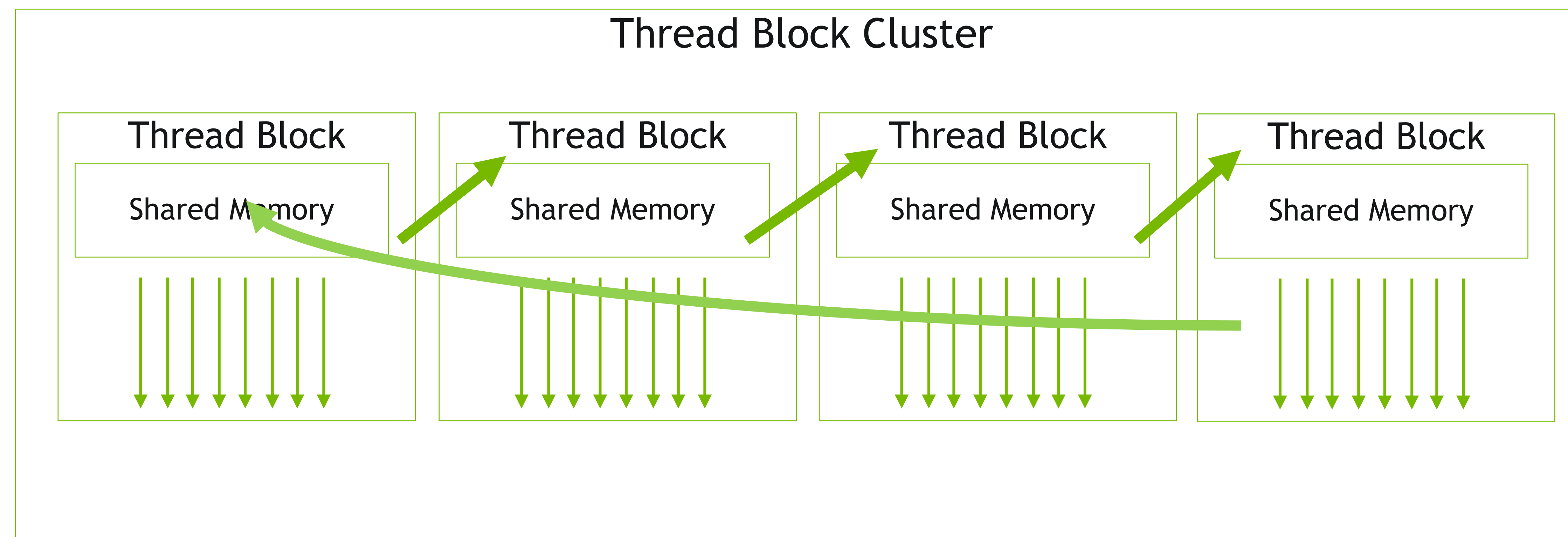
```
if (threadIdx.x == 0)
```

```
    *remote_smem = 10; // Store to remote memory
```

```
cluster.sync(); // Sync to ensure store is done
```

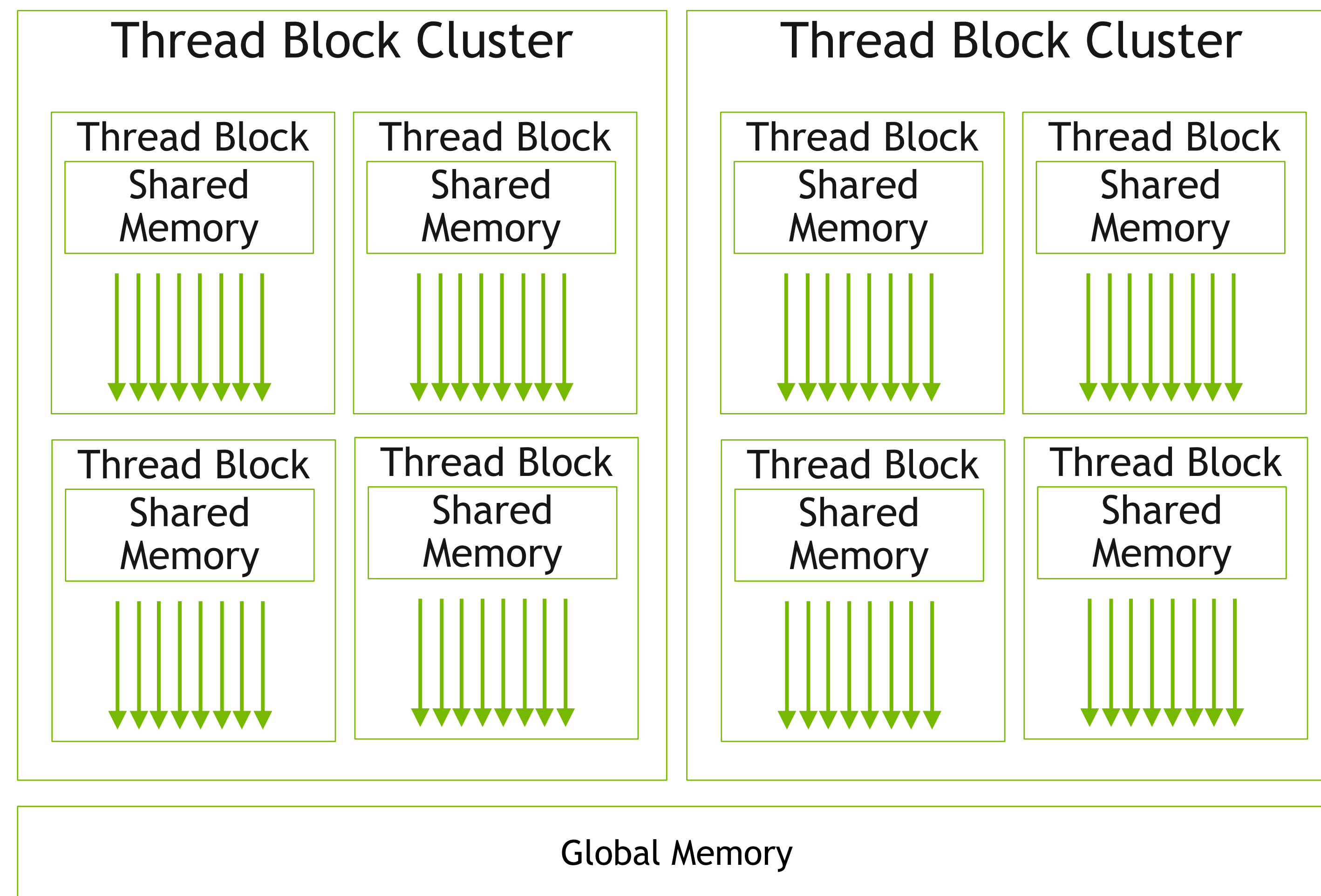
- All blocks within a thread block cluster can collaborate using **Distributed shared memory**

- Thread blocks can **read, write and perform atomics** on each other's shared memory



Thread and Memory Hierarchy

Launching CUDA Kernels with Clusters



- Annotate kernels with compile time cluster size
- Kernel launch done in classical way <<< , >>>

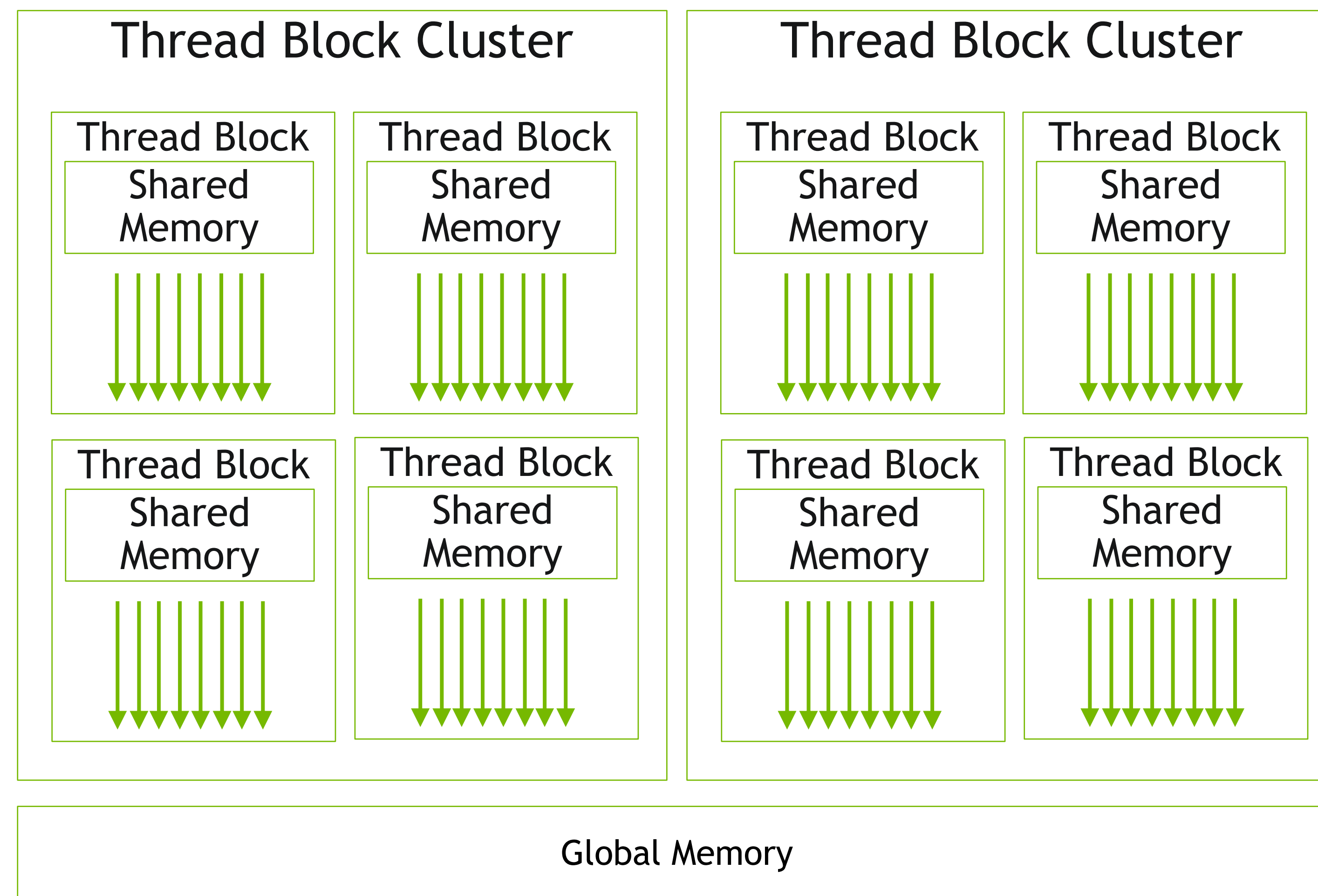
```
//Compile time: Kernel where each kernel is  
// 2 Thread Blocks in X-dimension and 2 in Y-  
dimension.
```

```
// Requires number of thread blocks to be multiple of  
4
```

```
__global__ void __cluster_dims__(2, 2, 1)  
clusterKernel()  
{ ... }
```


Thread and Memory Hierarchy

Launching CUDA Kernels with Clusters



- Using CUDA Extensible Kernel Launch API

```
// Launch via extensible launch API
{
    cudaLaunchConfig_t config = {0};
    cudaLaunchAttribute attribute[1];
    attribute[0].id = cudaLaunchAttributeClusterDimension;
    attribute[0].val.clusterDim.x = 2; // 2 blocks in X
    attribute[0].val.clusterDim.y = 2; // 2 blocks in Y
    attribute[0].val.clusterDim.z = 1;
    config.attrs = attribute;

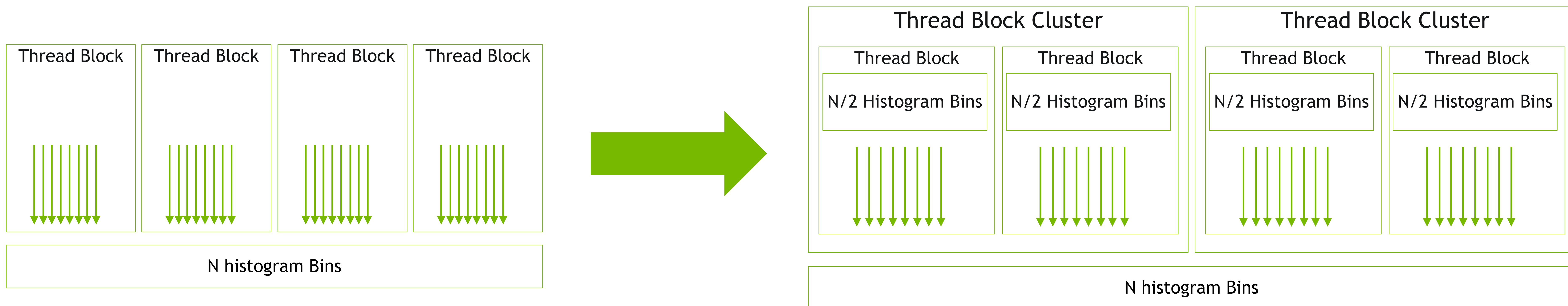
    config.numAttrs = 1;
    const int clusterSize = 2 * 2;
    config.gridDim = numClusters * clusterSize;
    config.blockDim = numThreads;

    cudaLaunchKernelEx(&config, (void*)clusterKernel,
    param1, param2, ...);
}
```


Histogram example

Histogram Computing in Distributed Shared Memory

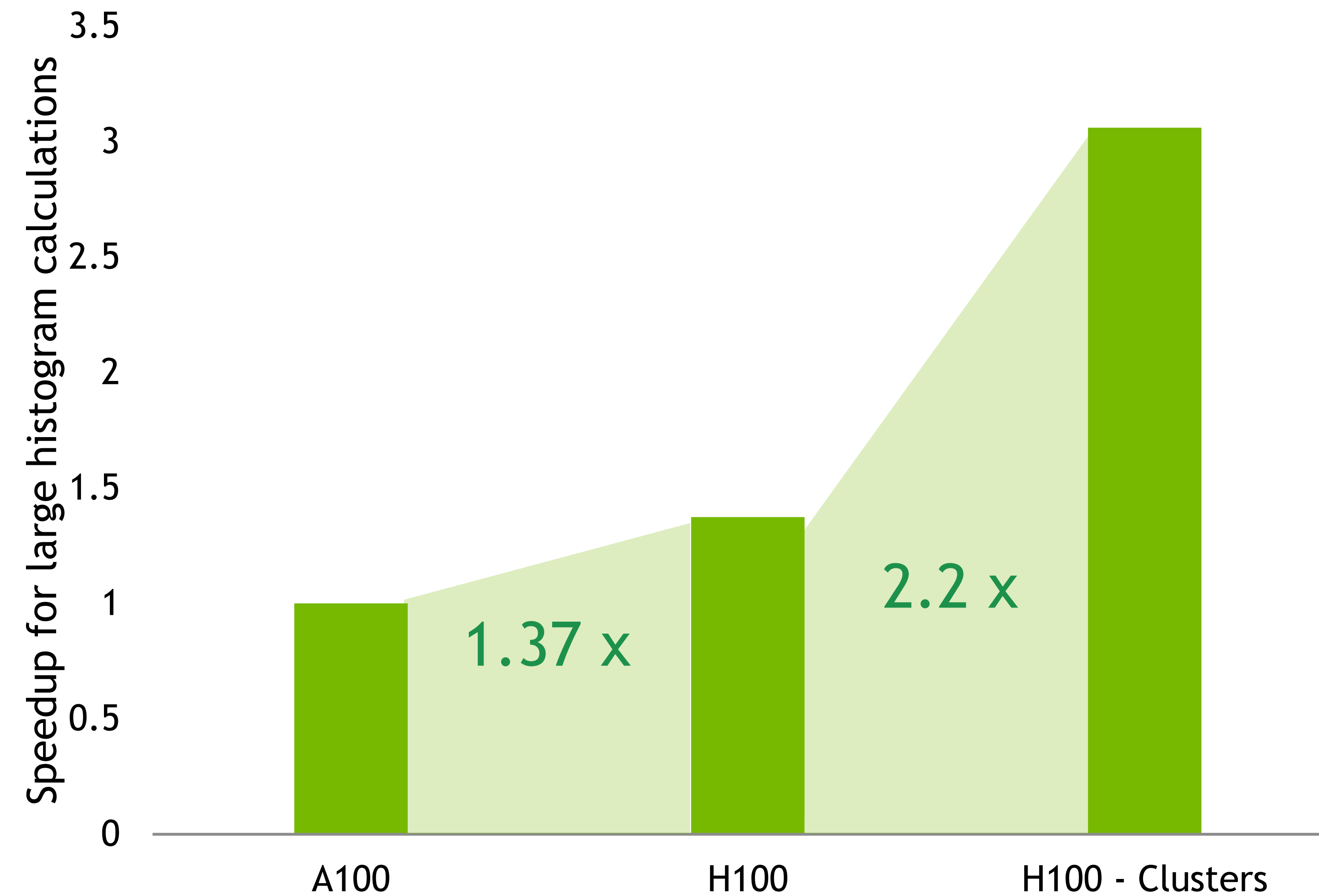
- Without clusters, histogram is directly computed in global memory
- With cluster, thread blocks can pool shared memory to compute local histogram



Histogram example

Histogram Performance A100 vs H100 up to 3.0 x

- Without clusters, histogram is directly computed in global memory
- With cluster, thread blocks can pool shared memory to compute local histogram



75K Histogram bins (300KB) fits in distributed shared memory of 2-block cluster → 37.5K (150KB) per thread block

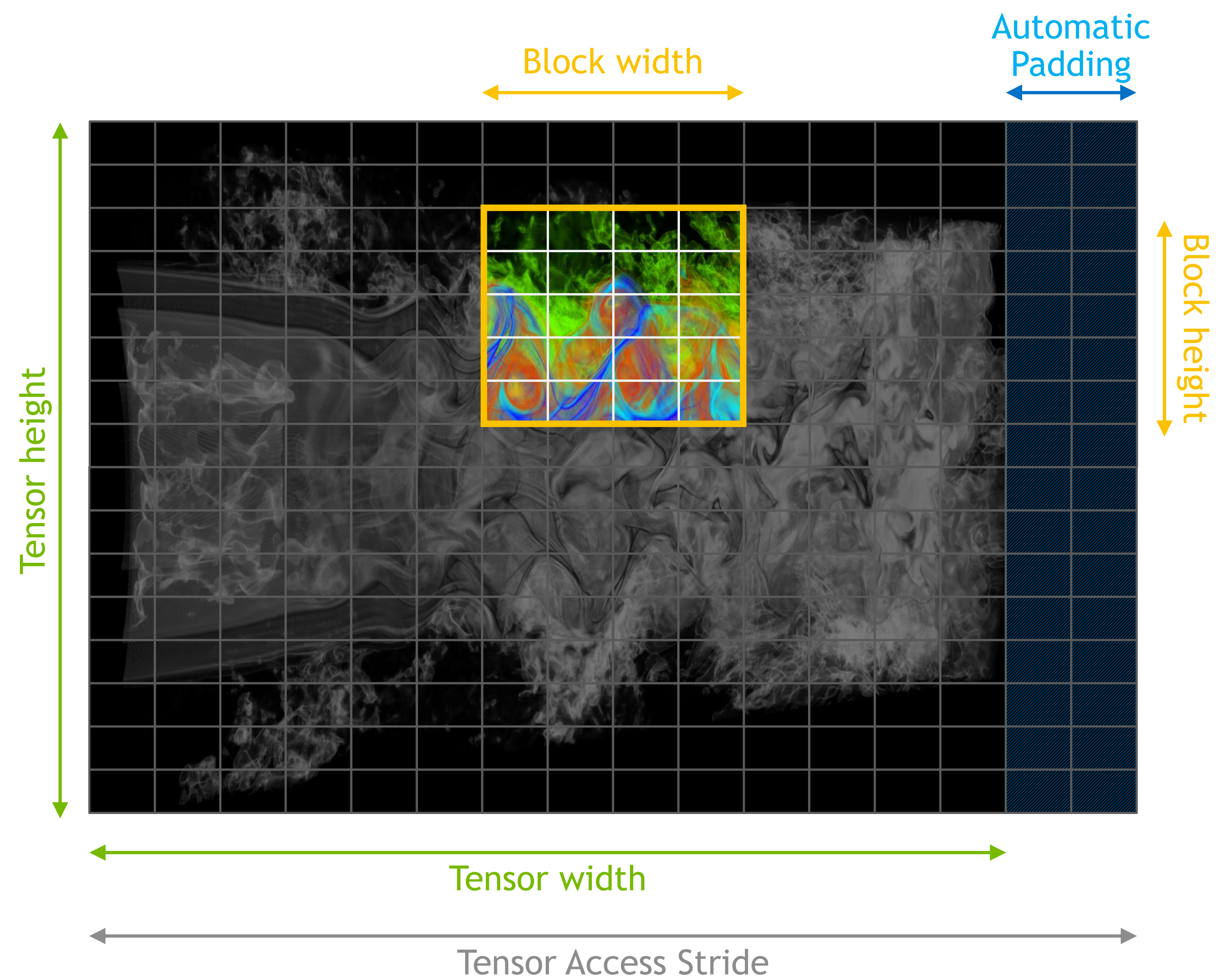
Asynchronous SIMT Programming Model



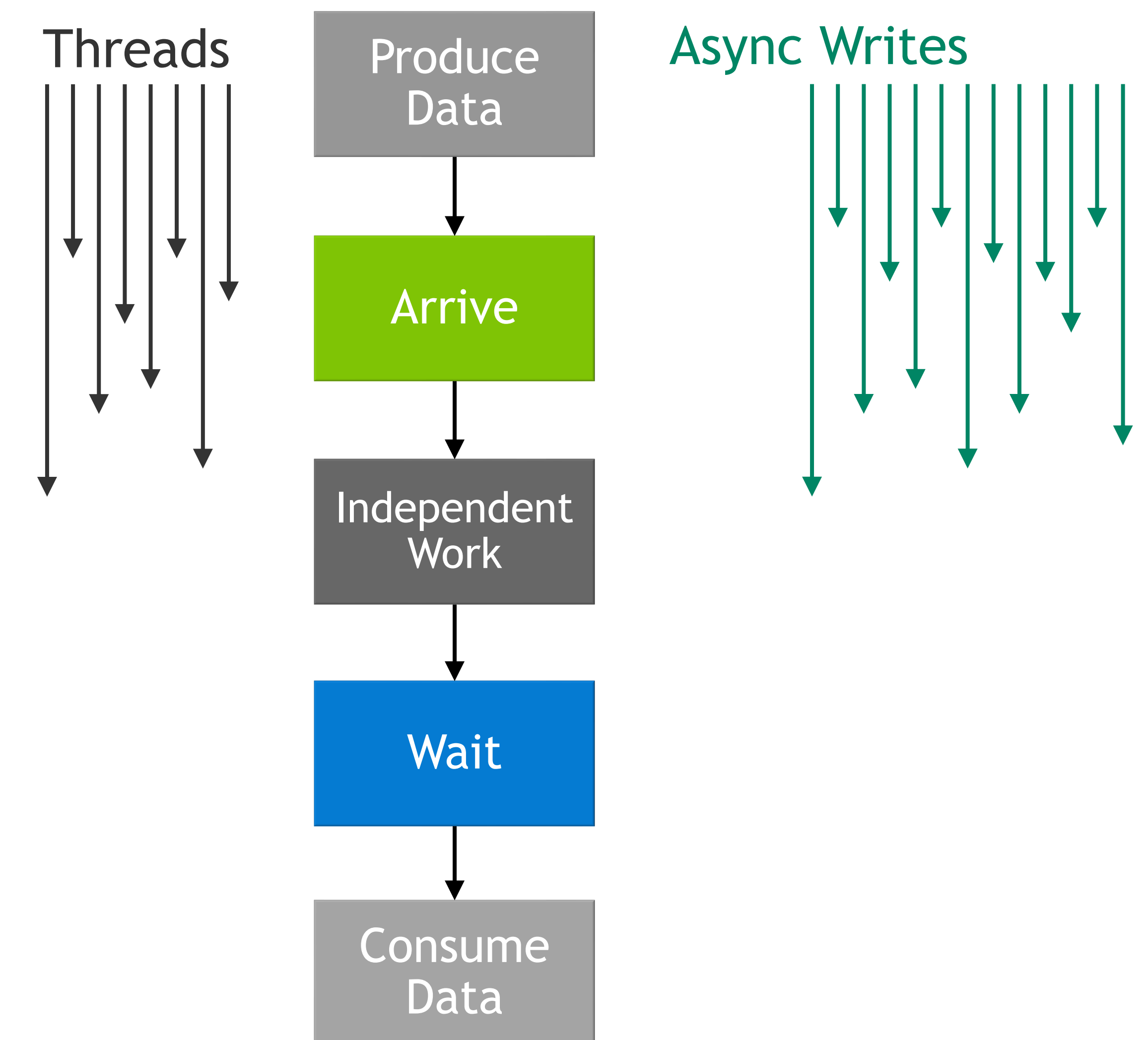
Asynchronous SIMT Programming

Building Blocks

Tensor Memory Accelerator



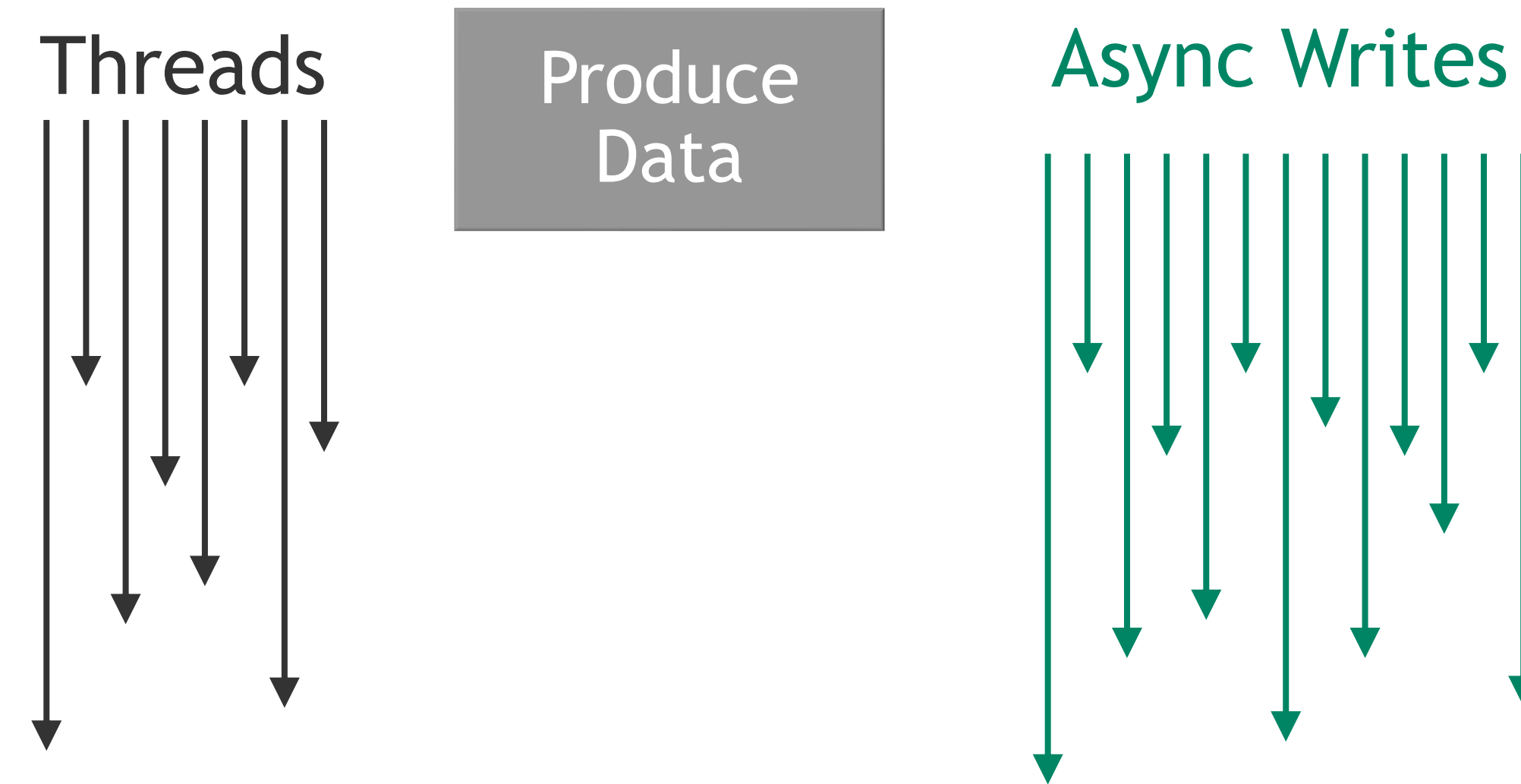
Async Transaction Barrier (New on H100)



ASYNCHRONOUS SIMT PROGRAMMING

Asynchronous Transaction Barrier

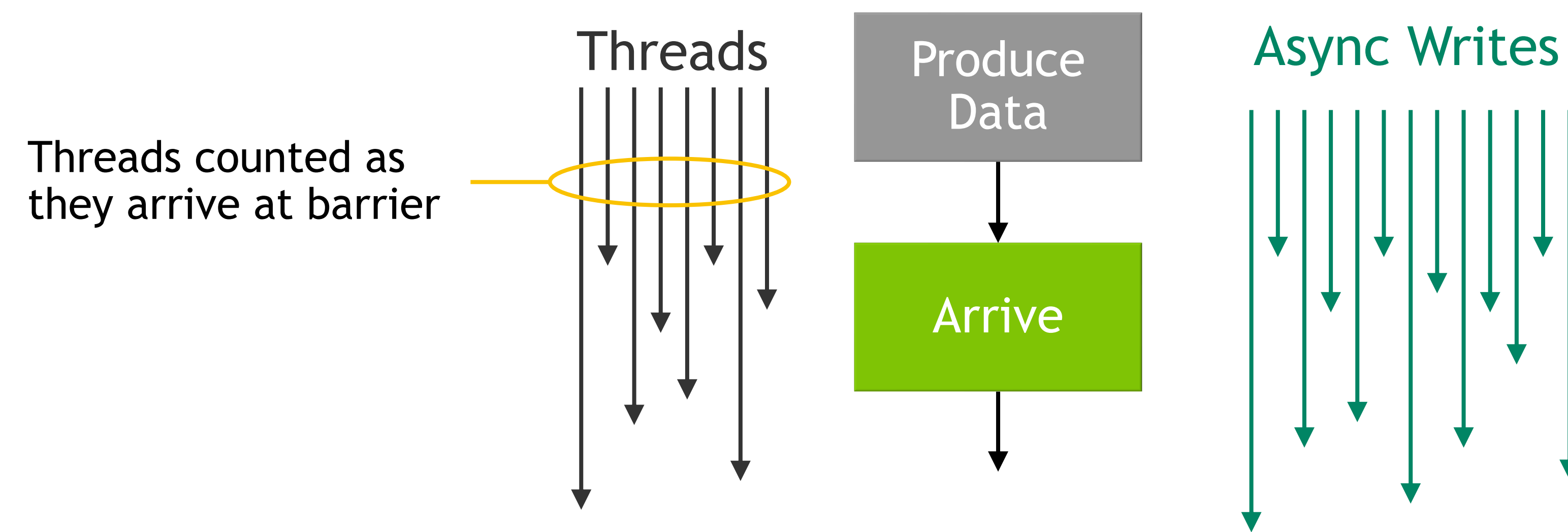
Async Transaction Barrier (New on H100)



ASYNCHRONOUS SIMT PROGRAMMING

Asynchronous Transaction Barrier

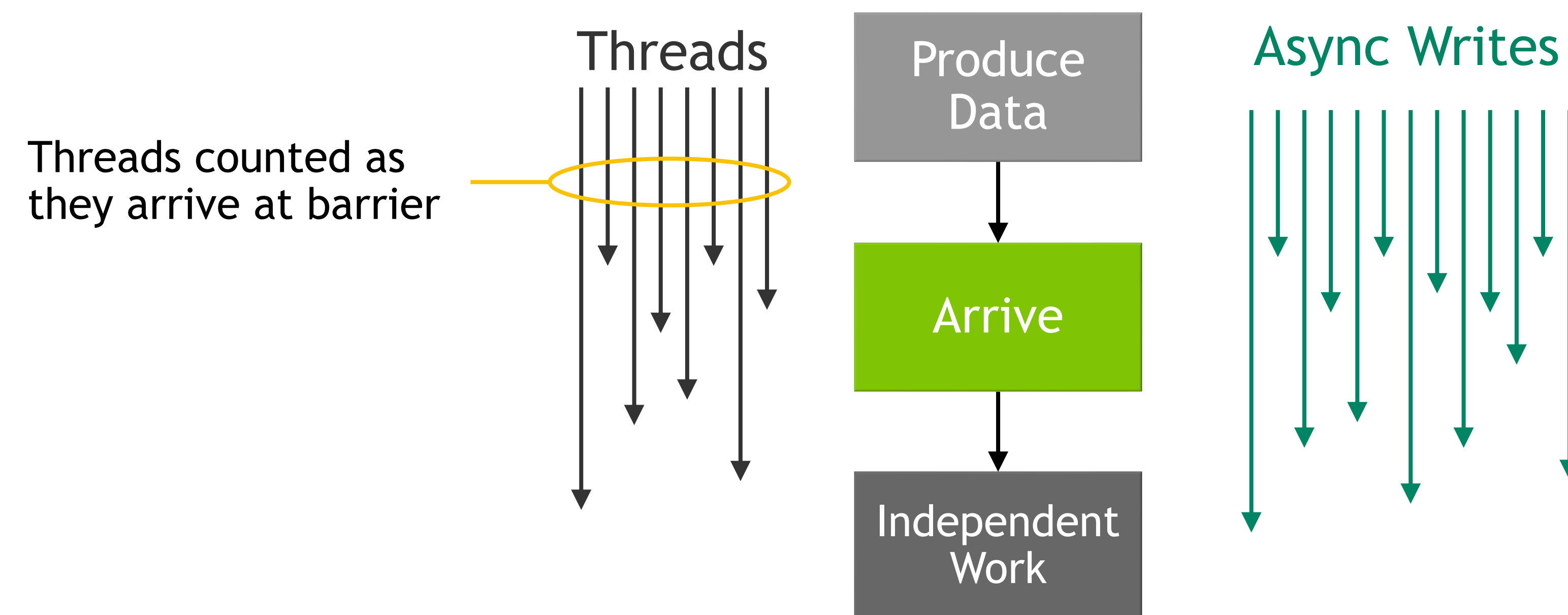
Async Transaction Barrier (New on H100)



ASYNCHRONOUS SIMT PROGRAMMING

Asynchronous Transaction Barrier

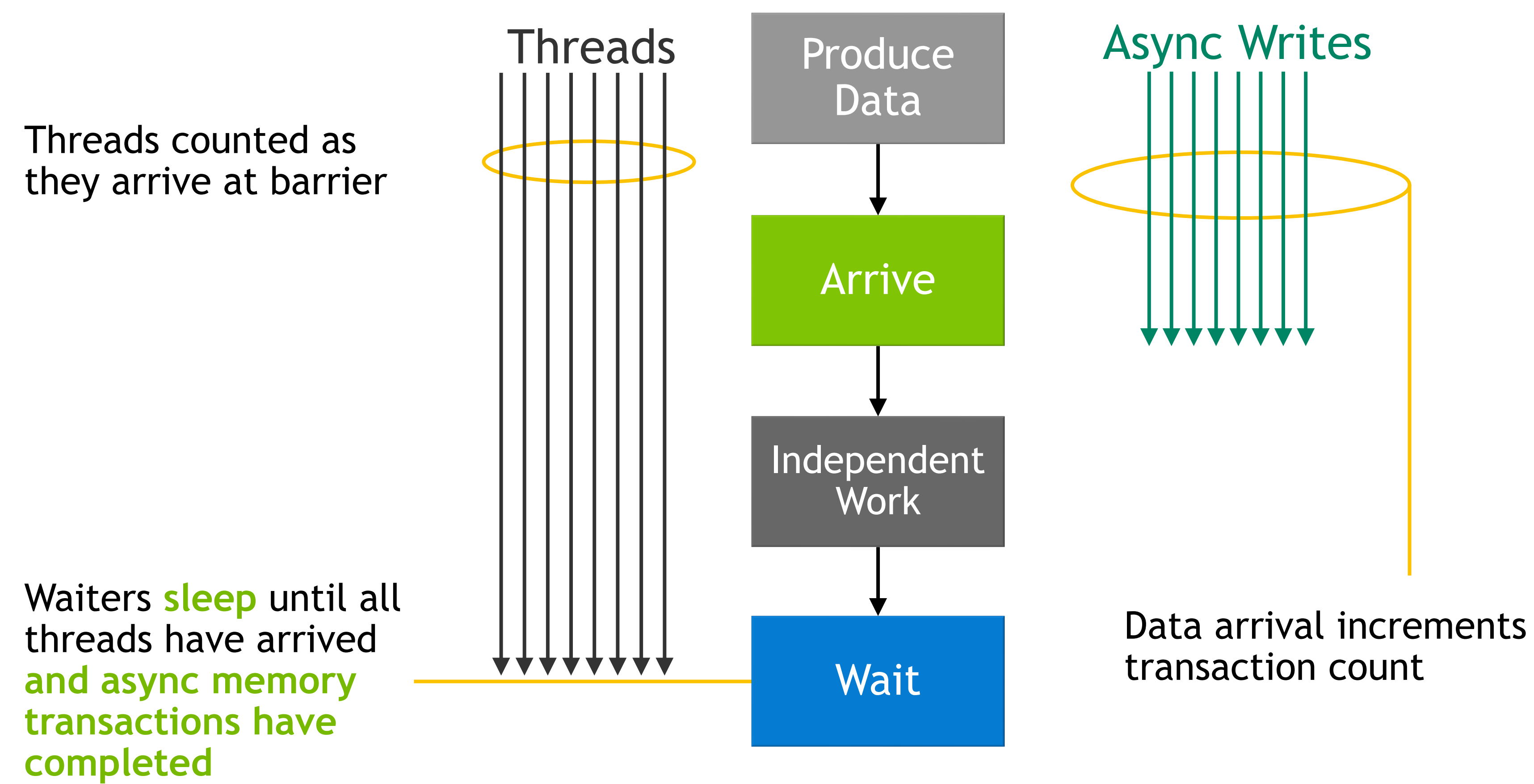
Async Transaction Barrier (New on H100)



ASYNCHRONOUS SIMT PROGRAMMING

Asynchronous Transaction Barrier

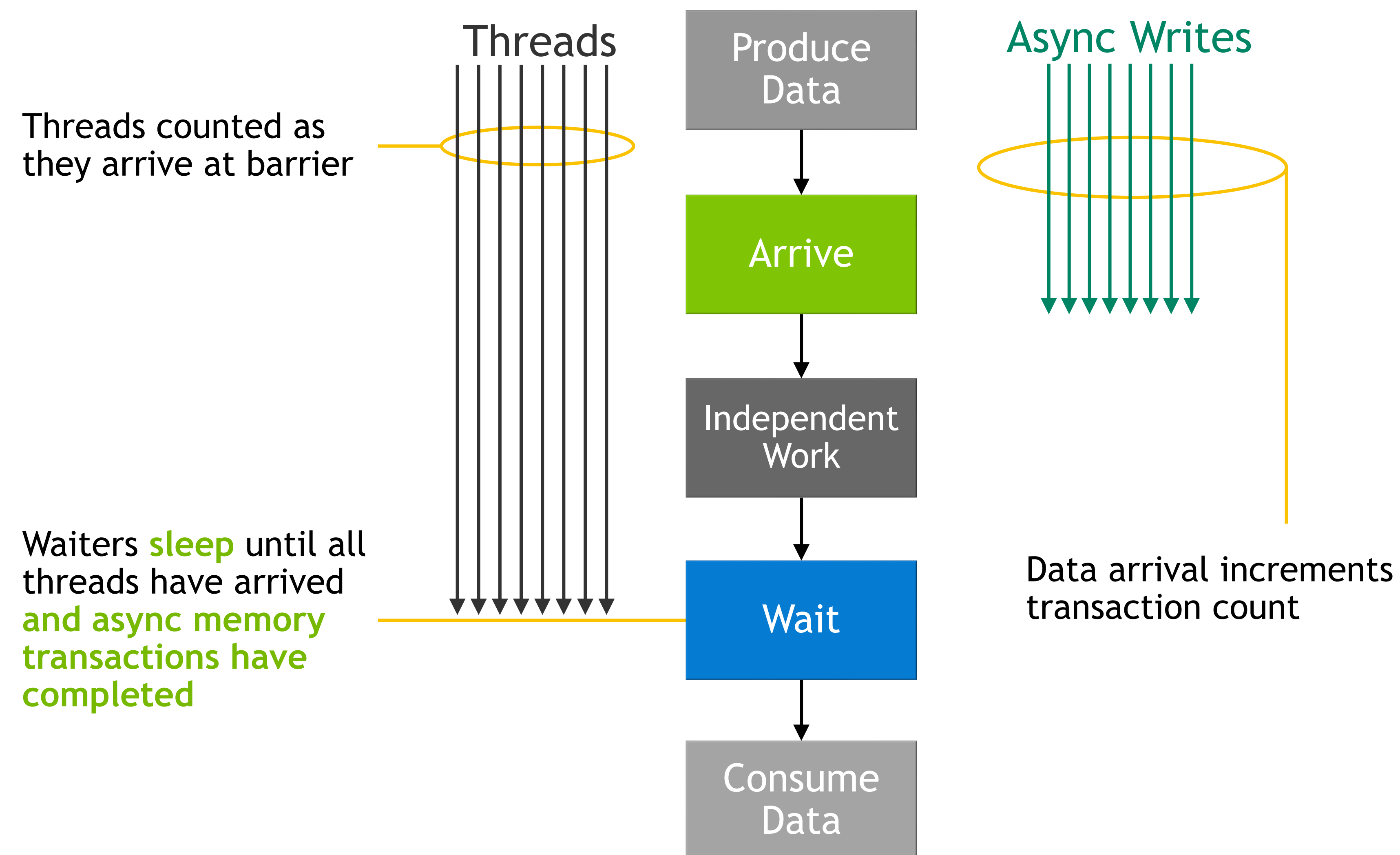
Async Transaction Barrier (New on H100)



ASYNCHRONOUS SIMT PROGRAMMING

Asynchronous Transaction Barrier

Async Transaction Barrier (New on H100)



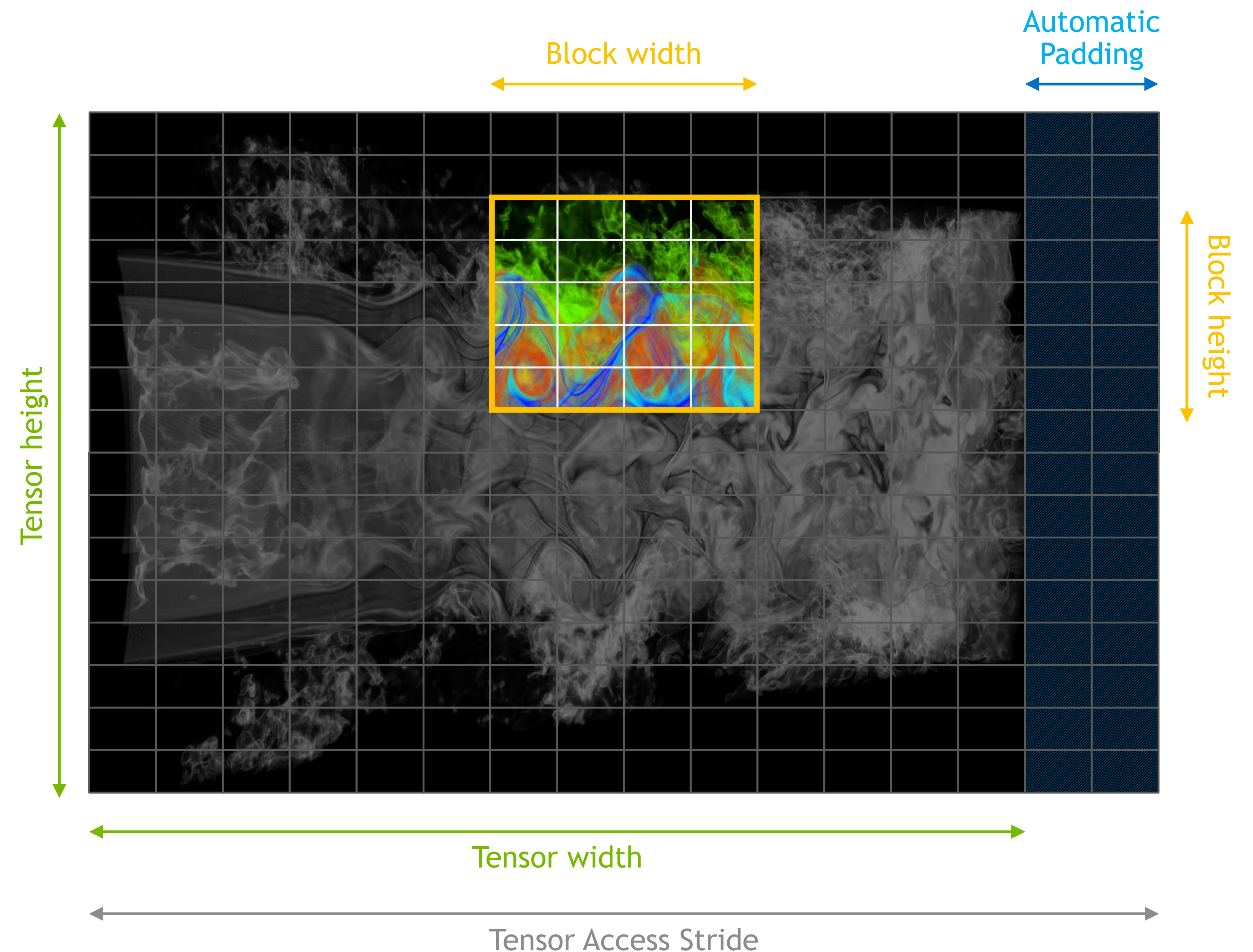
ASYNCHRONOUS SIMT PROGRAMMING

Tensor Memory Accelerator: Efficient Copy of Tensor Memory

Multi-Dimensional Tensor Copying

- Automatic stride & address generation up to tensors of rank 5
- Boundary padding for out-of-bounds accesses
- Fire-and-forget from a single thread – everything handled by TMA
- No iteration or bounds-checking code required
- Saves registers that do loop calculation and boundary checks

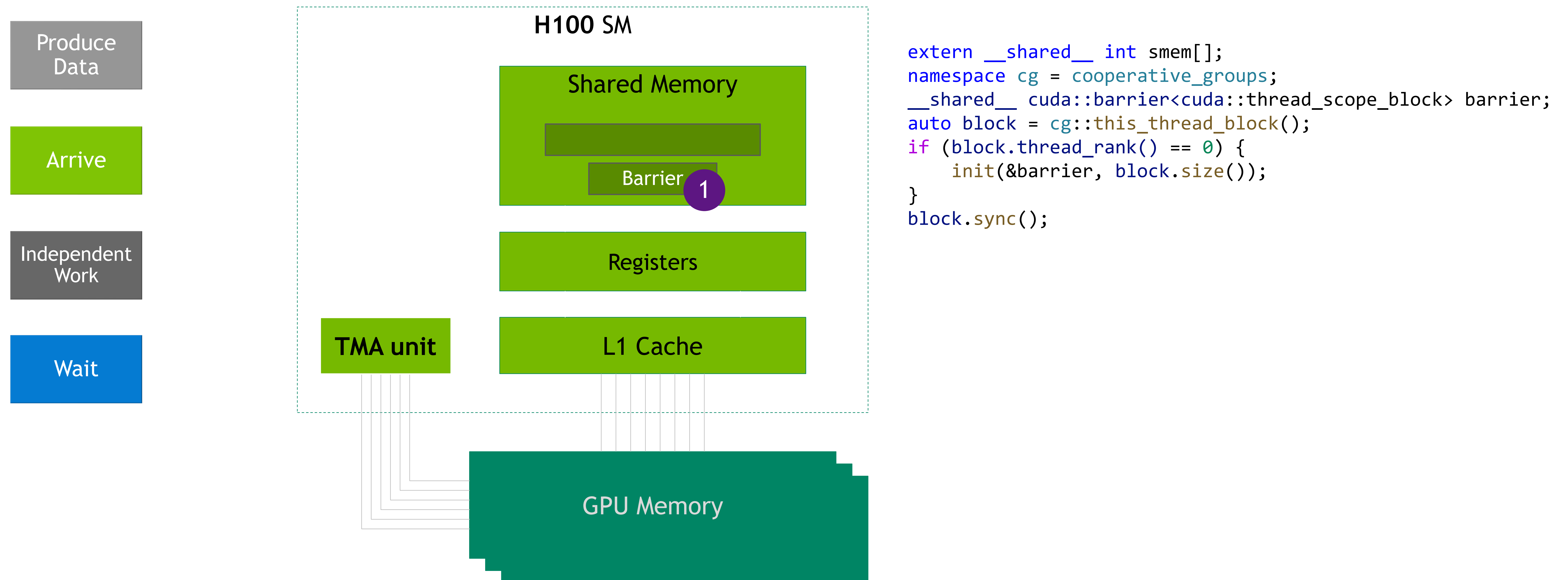
The TMA can copy sub-regions of a multi-dimensional tensor



ASYNCHRONOUS SIMT PROGRAMMING

`cuda::memcpy_async` Global <-> Shared

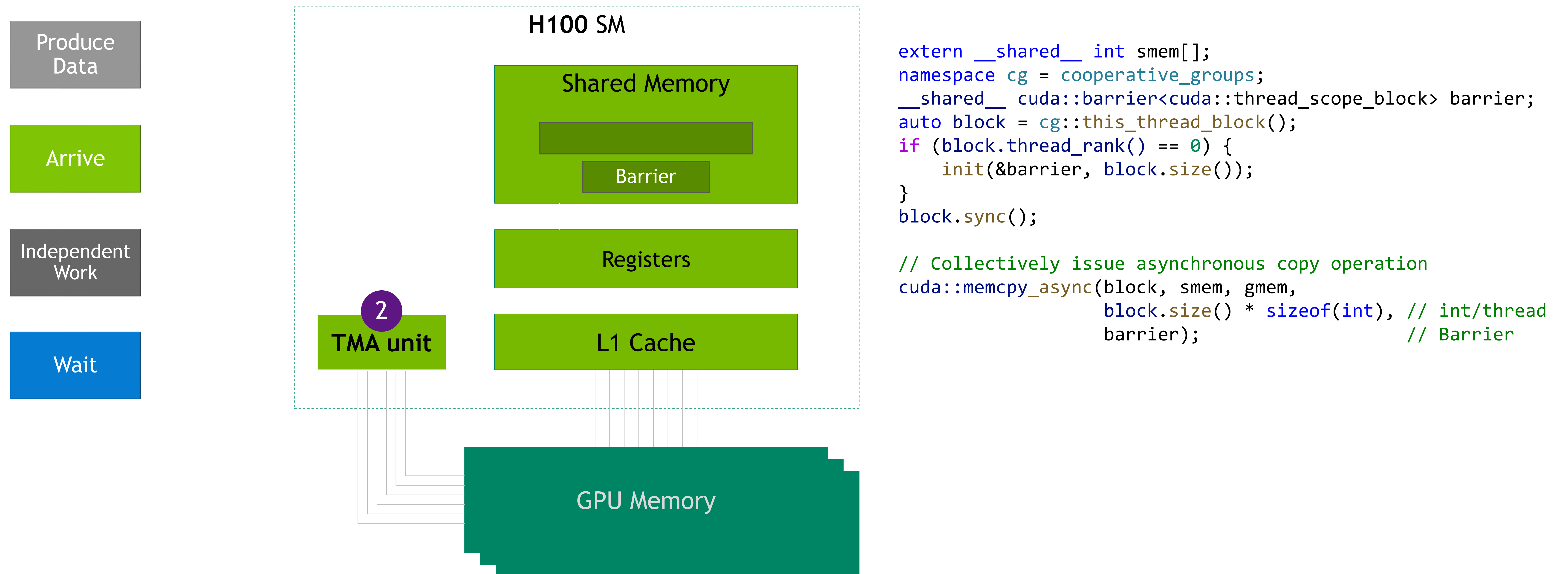
Fully asynchronous with threads and uses **Asynchronous Transaction Barrier** to signal the completion.



ASYNCHRONOUS SIMT PROGRAMMING

`cuda::memcpy_async` Global <-> Shared

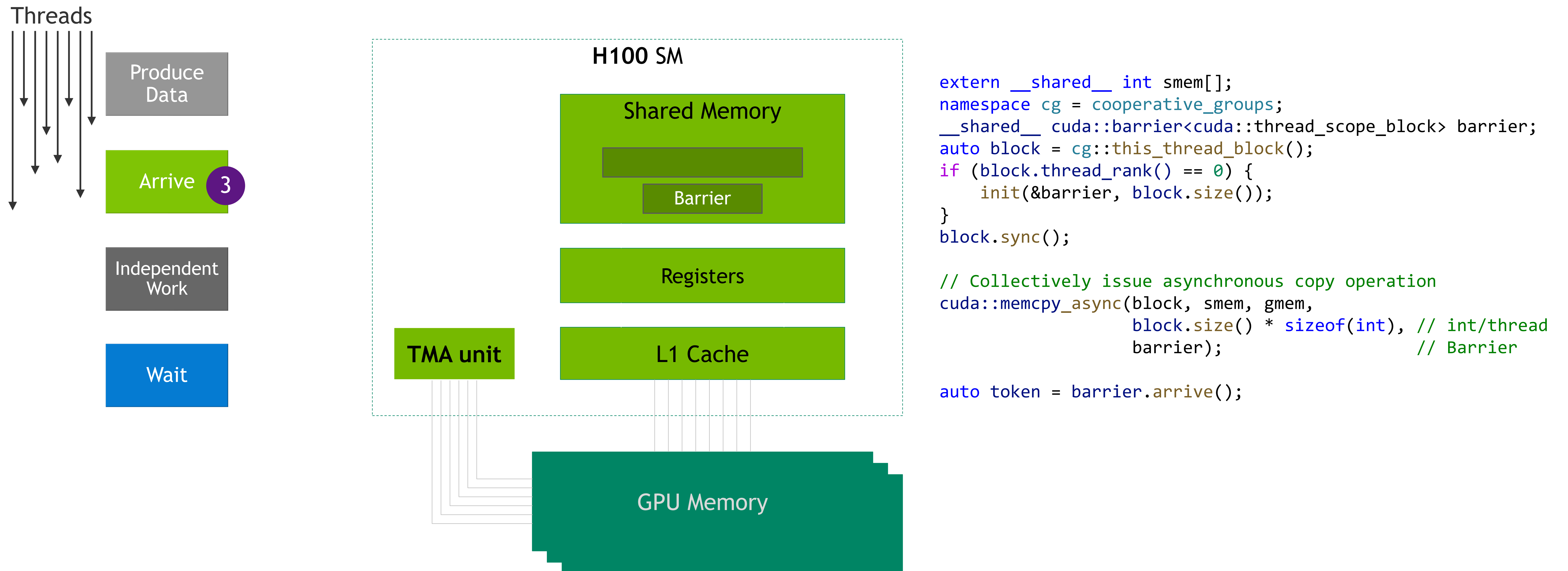
Fully asynchronous with threads and uses **Asynchronous Transaction Barrier** to signal the completion.



ASYNCHRONOUS SIMT PROGRAMMING

`cuda::memcpy_async` Global <-> Shared

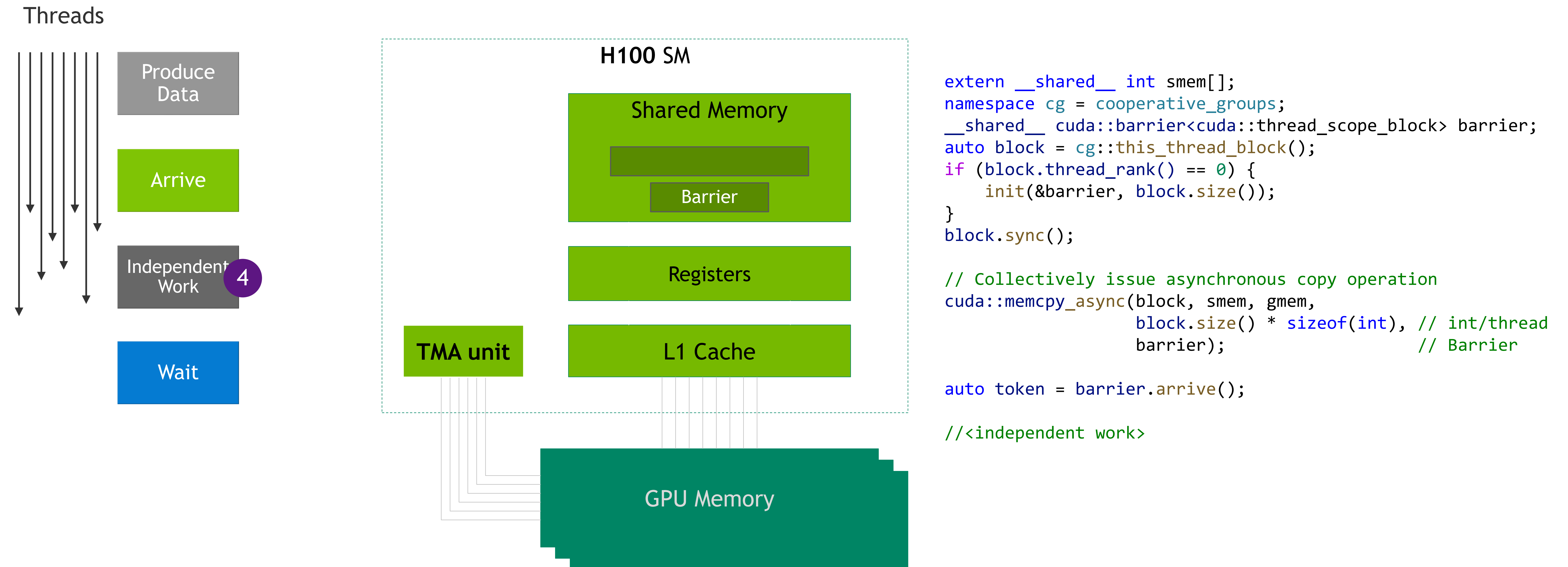
Fully asynchronous with threads and uses **Asynchronous Transaction Barrier** to signal the completion.



ASYNCHRONOUS SIMT PROGRAMMING

`cuda::memcpy_async` Global <-> Shared

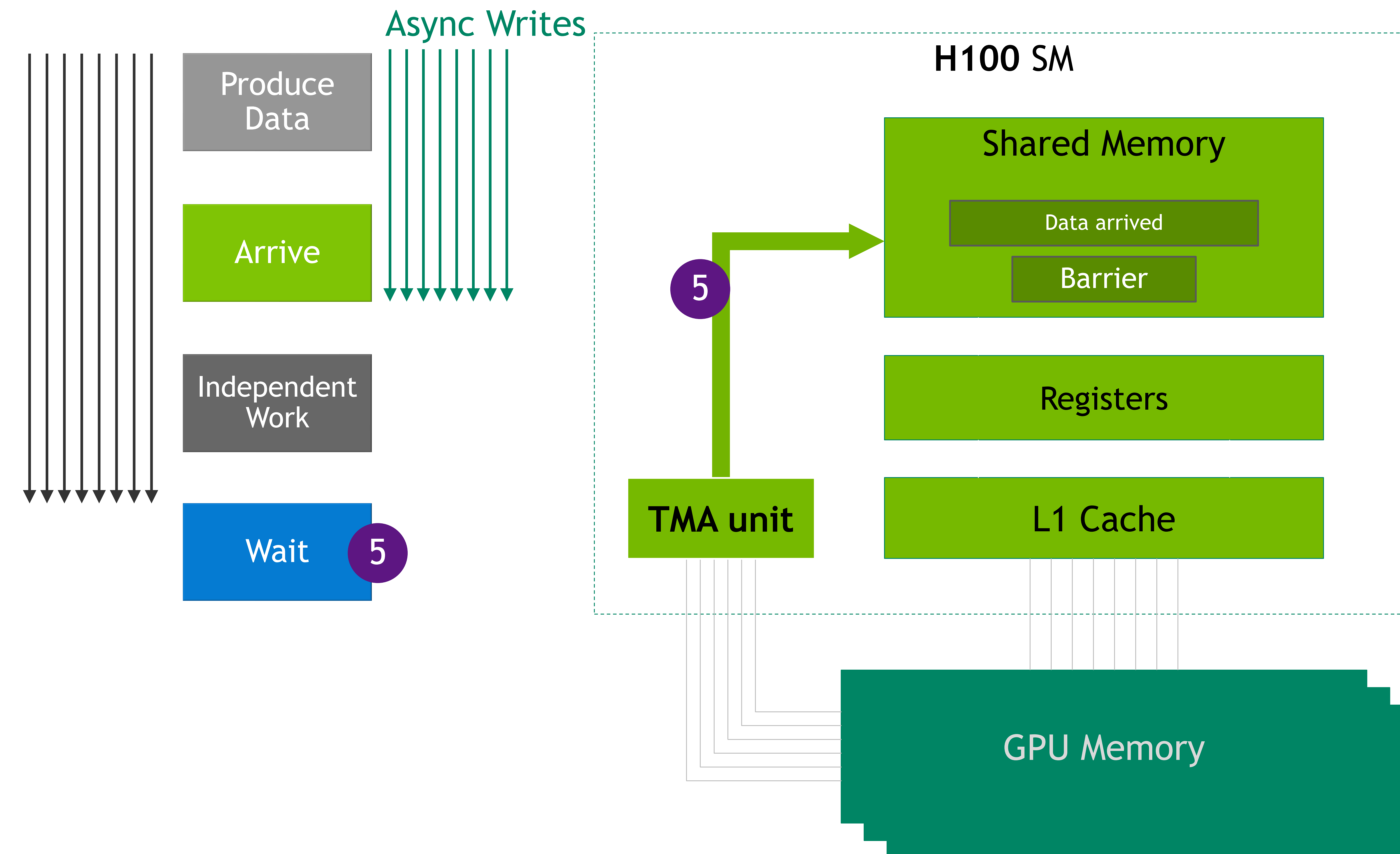
Fully asynchronous with threads and uses **Asynchronous Transaction Barrier** to signal the completion.



ASYNCHRONOUS SIMT PROGRAMMING

`cuda::memcpy_async` Global <-> Shared

Fully asynchronous with threads and uses **Asynchronous Transaction Barrier** to signal the completion.



```
extern __shared__ int smem[];
namespace cg = cooperative_groups;
__shared__ cuda::barrier<cuda::thread_scope_block> barrier;
auto block = cg::this_thread_block();
if (block.thread_rank() == 0) {
    init(&barrier, block.size());
}
block.sync();

// Collectively issue asynchronous copy operation
cuda::memcpy_async(block, smem, gmem,
                   block.size() * sizeof(int), // int/thread
                   barrier);

auto token = barrier.arrive();

//<independent work>

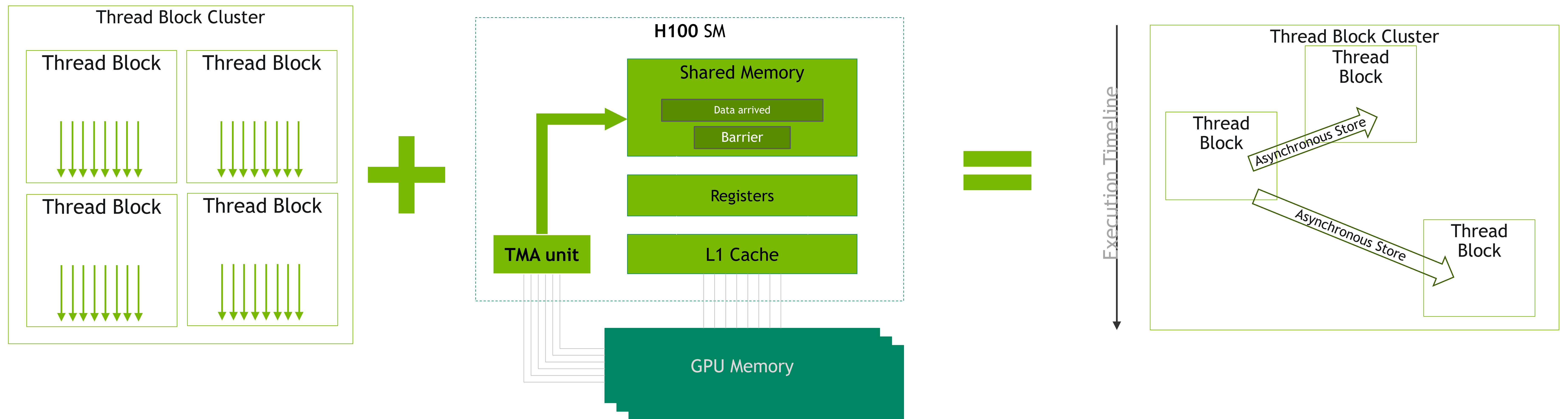
barrier.wait(std::move(token));
```


The background of the slide is a black field filled with numerous thin, curved, and slightly blurred lines in shades of green and yellow. These lines create a sense of motion and depth, resembling light trails or data paths. On the far left, there is a solid vertical green bar.

Asynchronous SIMT Programming in Clusters

Asynchronous SIMT Programming

Producer -> Consumer Pattern in Cluster to achieve over 5x performance gains



ASYNCHRONOUS SIMT PROGRAMMING

Initialize the barrier with producer and consumer thread counts

Thread Block 0 - Producer (2 threads)

Shared Memory

Thread 0, Producer



Thread 1, Producer



Thread Block 1 - Consumer (2 threads)

Barrier (4)

Shared Memory

Thread 0,
Consumer



Arrive

Thread 1,
Consumer



Arrive

ASYNCHRONOUS SIMT PROGRAMMING

Consumers arrive and wait for data from producers

Thread Block 0 - Producer (2 threads)

Shared Memory

Thread 0, Producer



Thread 1, Producer



Thread Block 1 - Consumer (2 threads)

Barrier

Shared Memory

Thread 0,
Consumer



Thread 1,
Consumer



Arrive

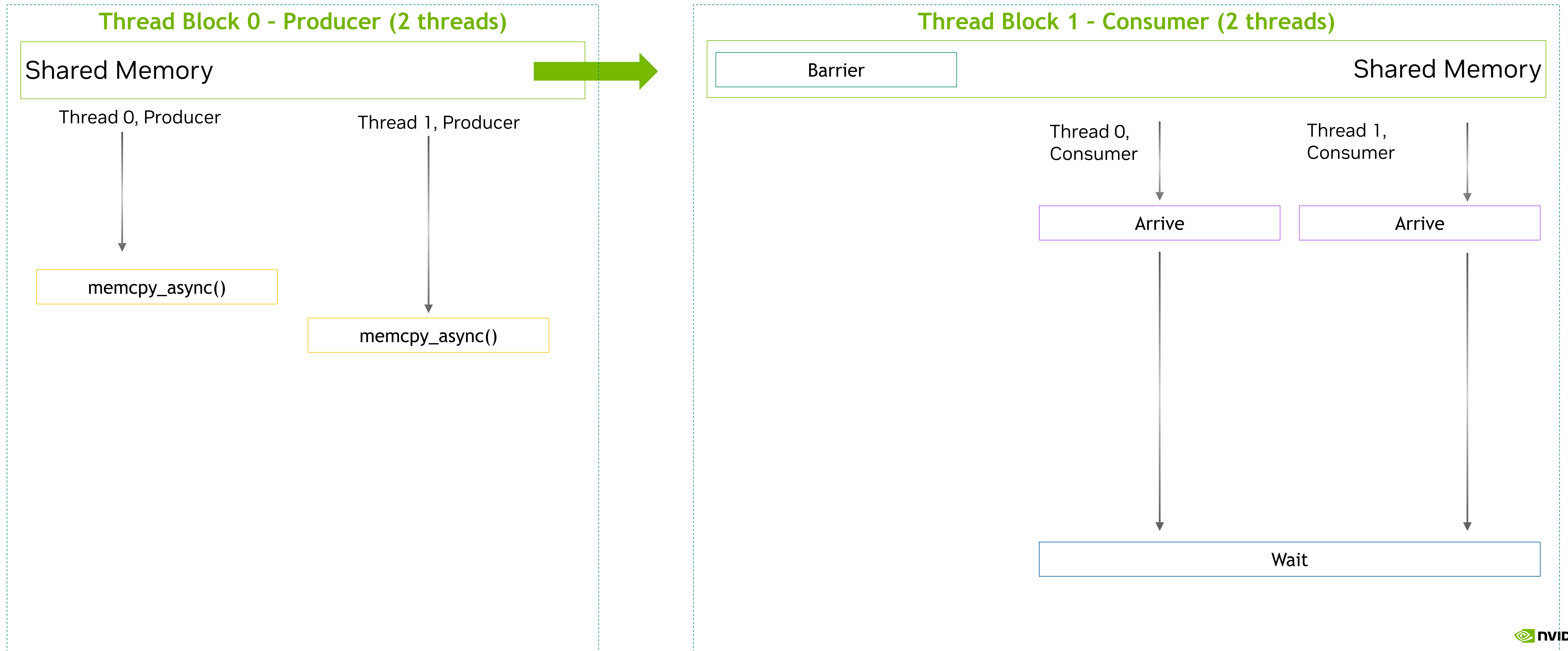
Arrive



Wait

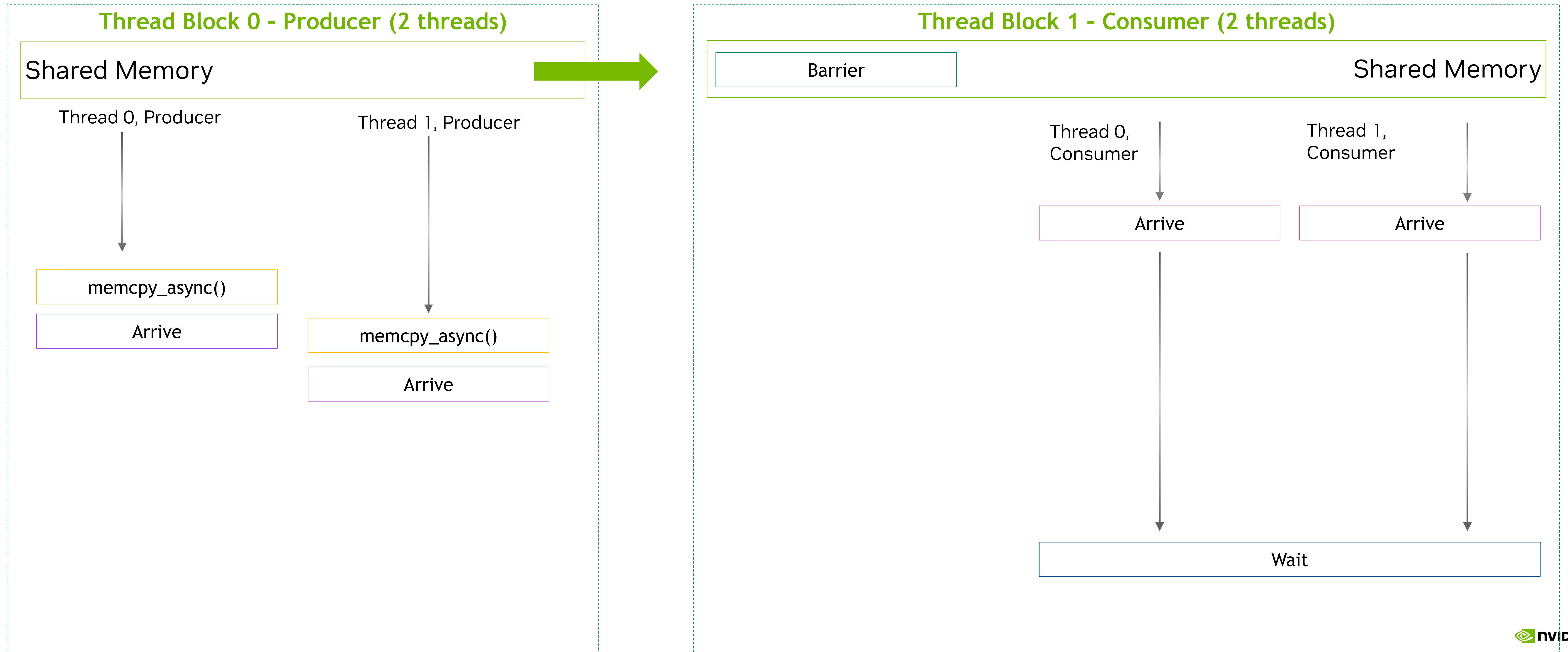
ASYNCHRONOUS SIMT PROGRAMMING

Producers issue's memcpy_async to consumer's shared memory using consumer barrier



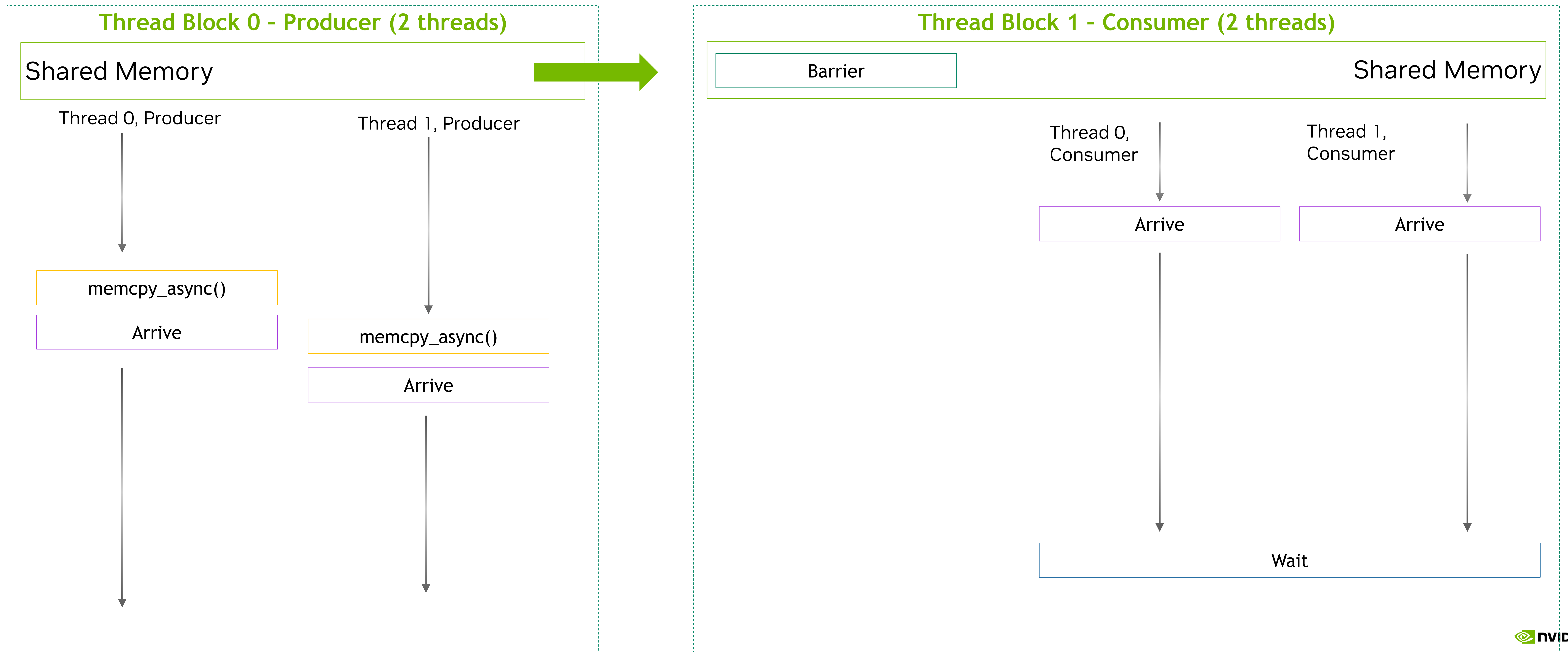
ASYNCHRONOUS SIMT PROGRAMMING

Producers perform barrier arrival, on barrier in Consumer Thread Block



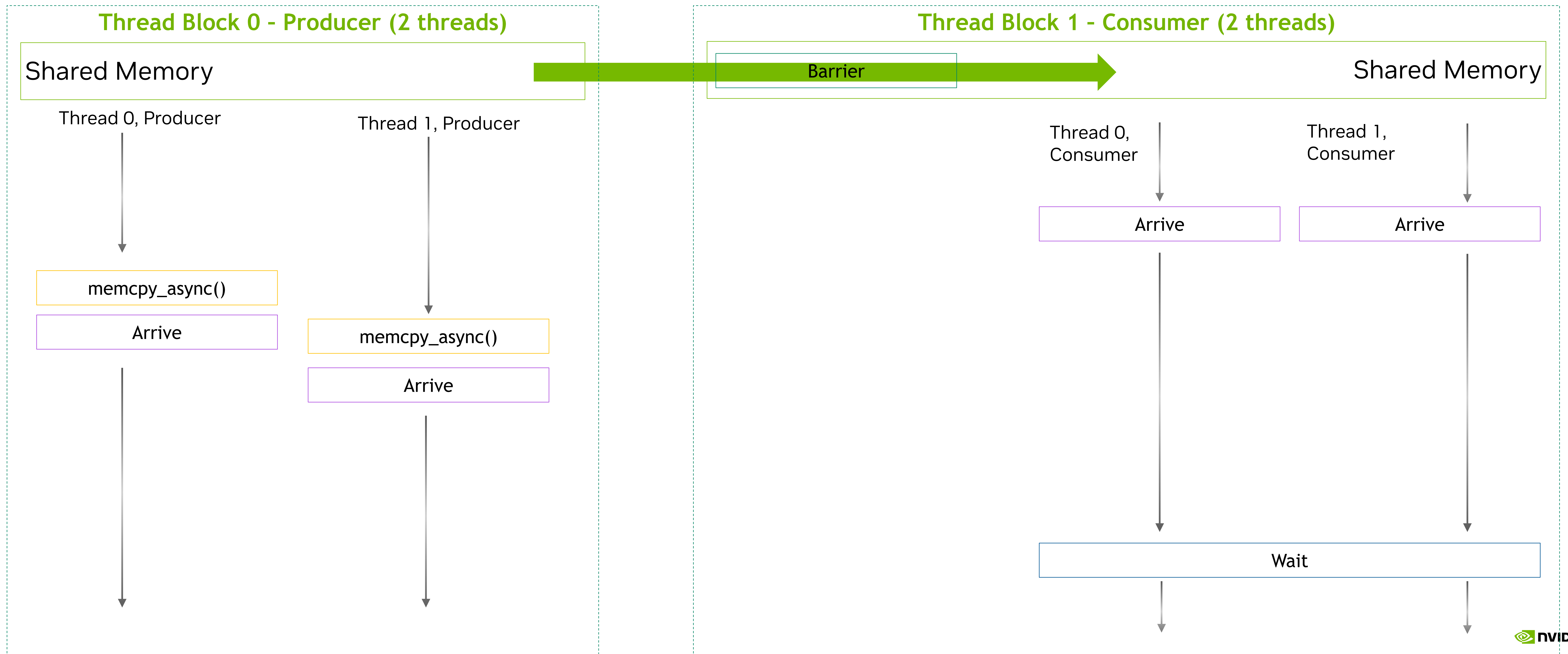
ASYNCHRONOUS SIMT PROGRAMMING

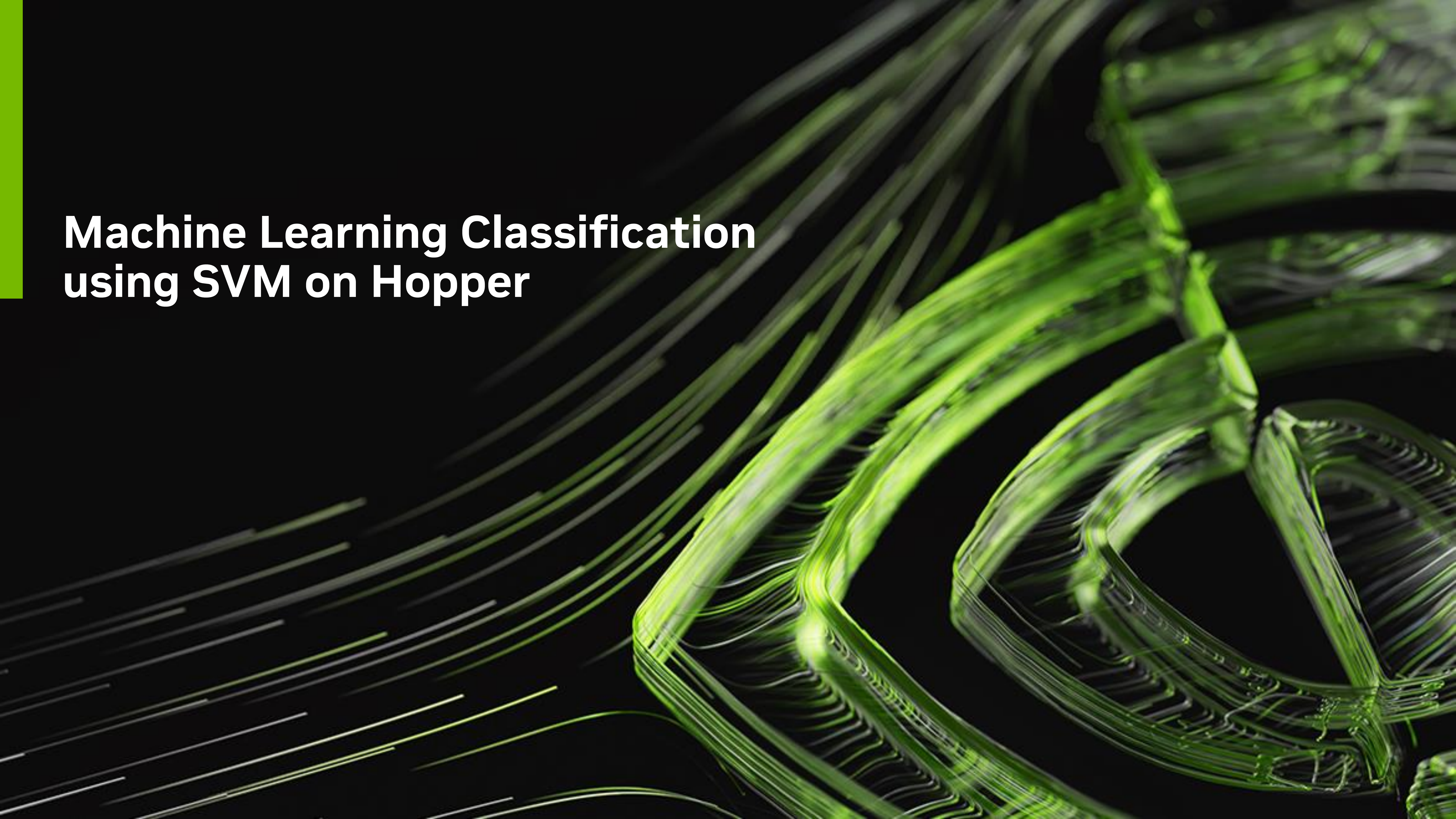
Consumers are blocked until all data has arrived as well as the producer threads



ASYNCHRONOUS SIMT PROGRAMMING

All threads have arrived as well as data. Consumers are unblocked.

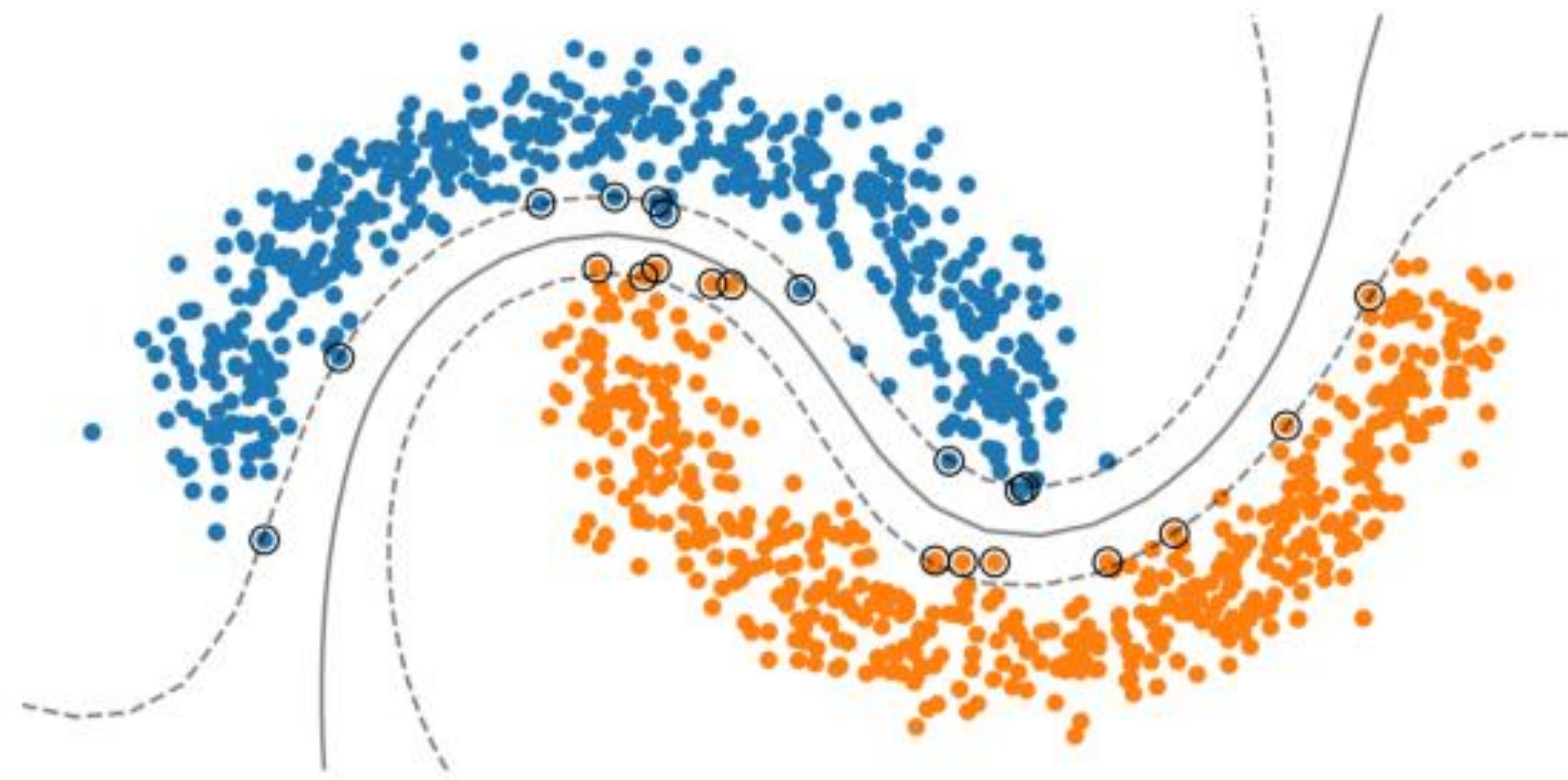
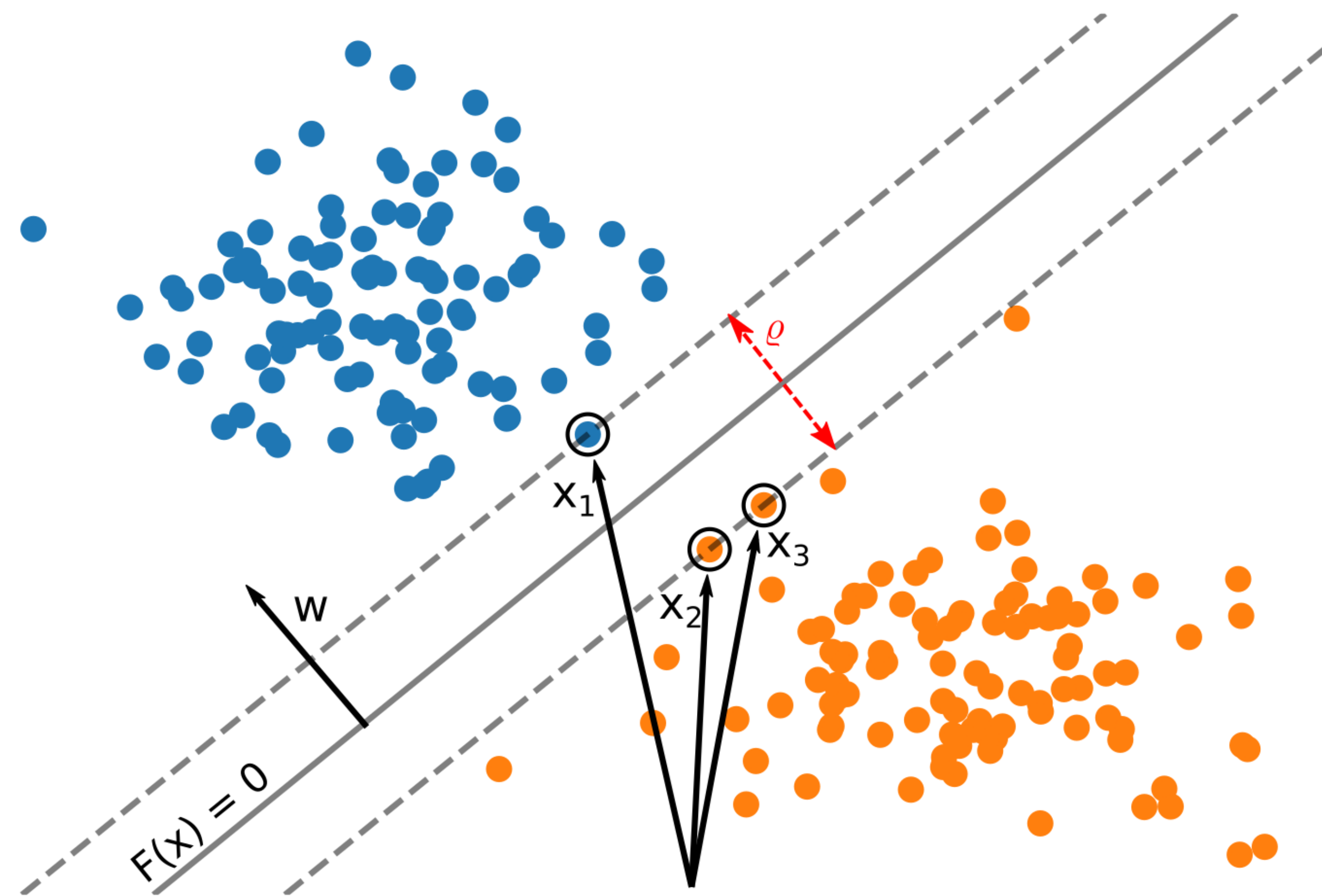




Machine Learning Classification using SVM on Hopper

Support Vector Machine (SVM)

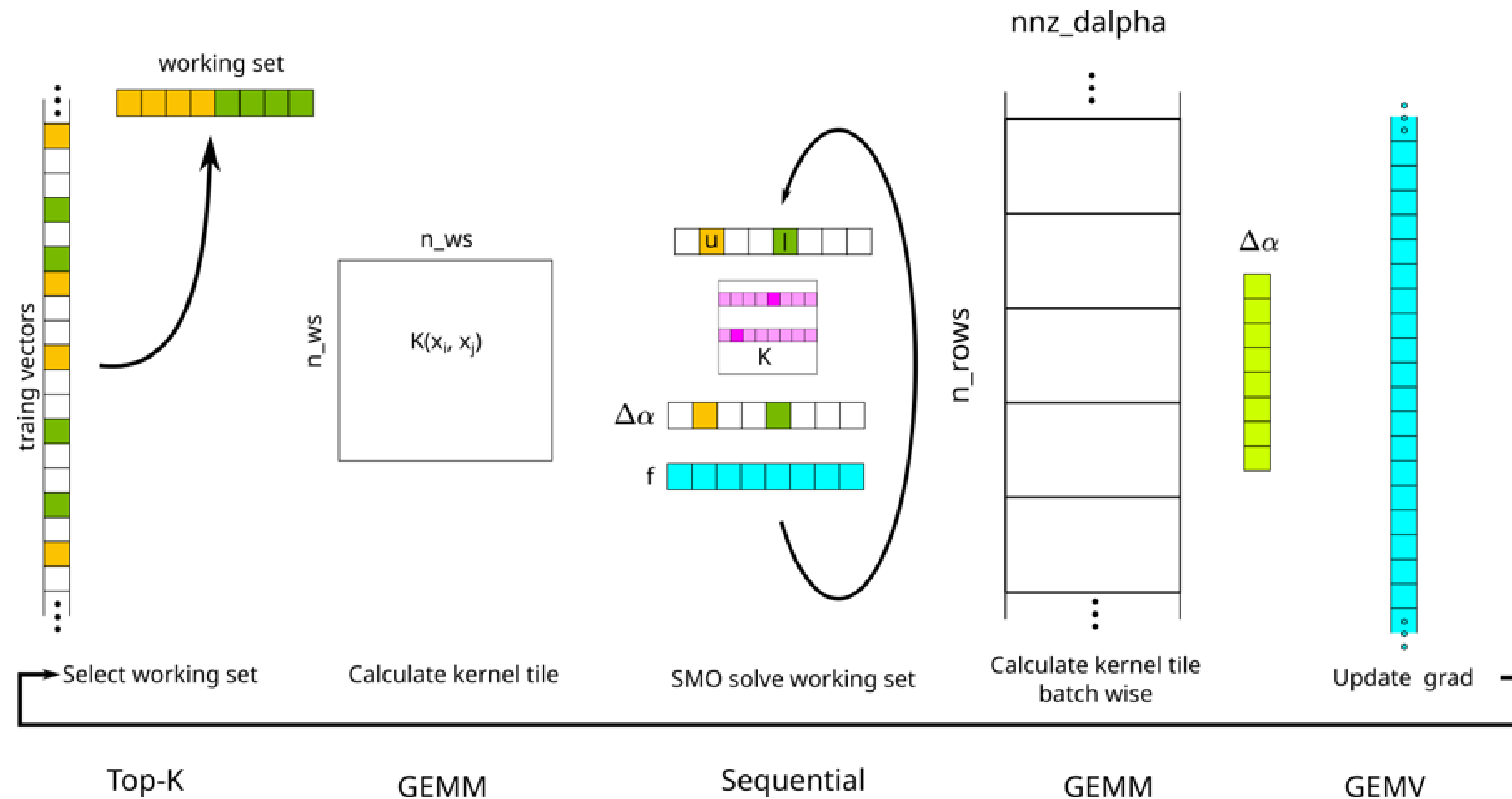
Large Margin Classifier



$$F(\vec{x}) = \sum_i \alpha_i y_i K(\vec{x}_i, \vec{x}) + b.$$

Support Vector Machine (SVM)

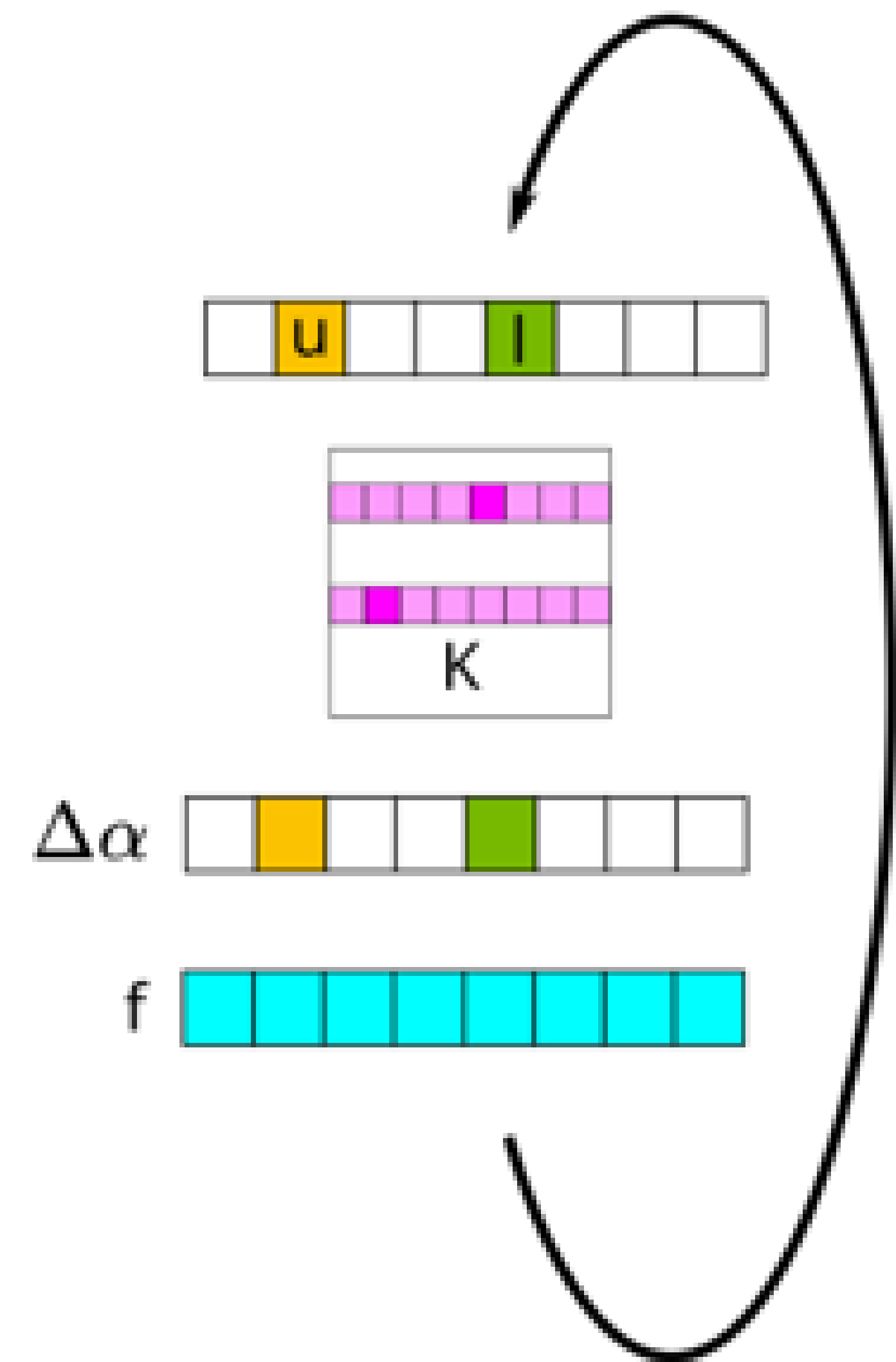
Algorithm



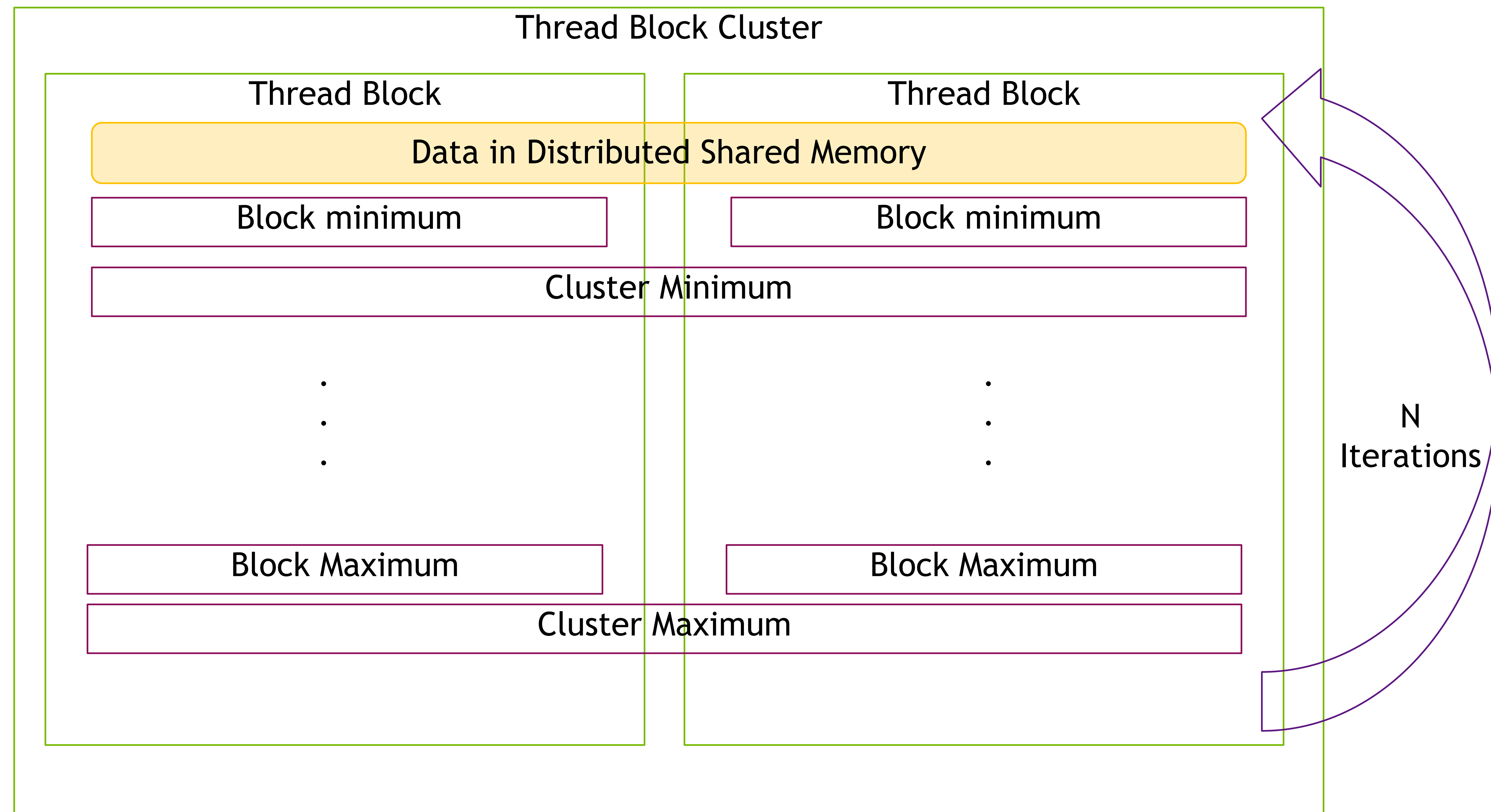
- CuML's SVM in [RAPIDS](#): Optimal hierarchical Decomposition
- Sequential inner step, single thread block: SmoBlockSolve

Support Vector Machine (SVM)

SMO Block Solve - Sequential part of the algorithm

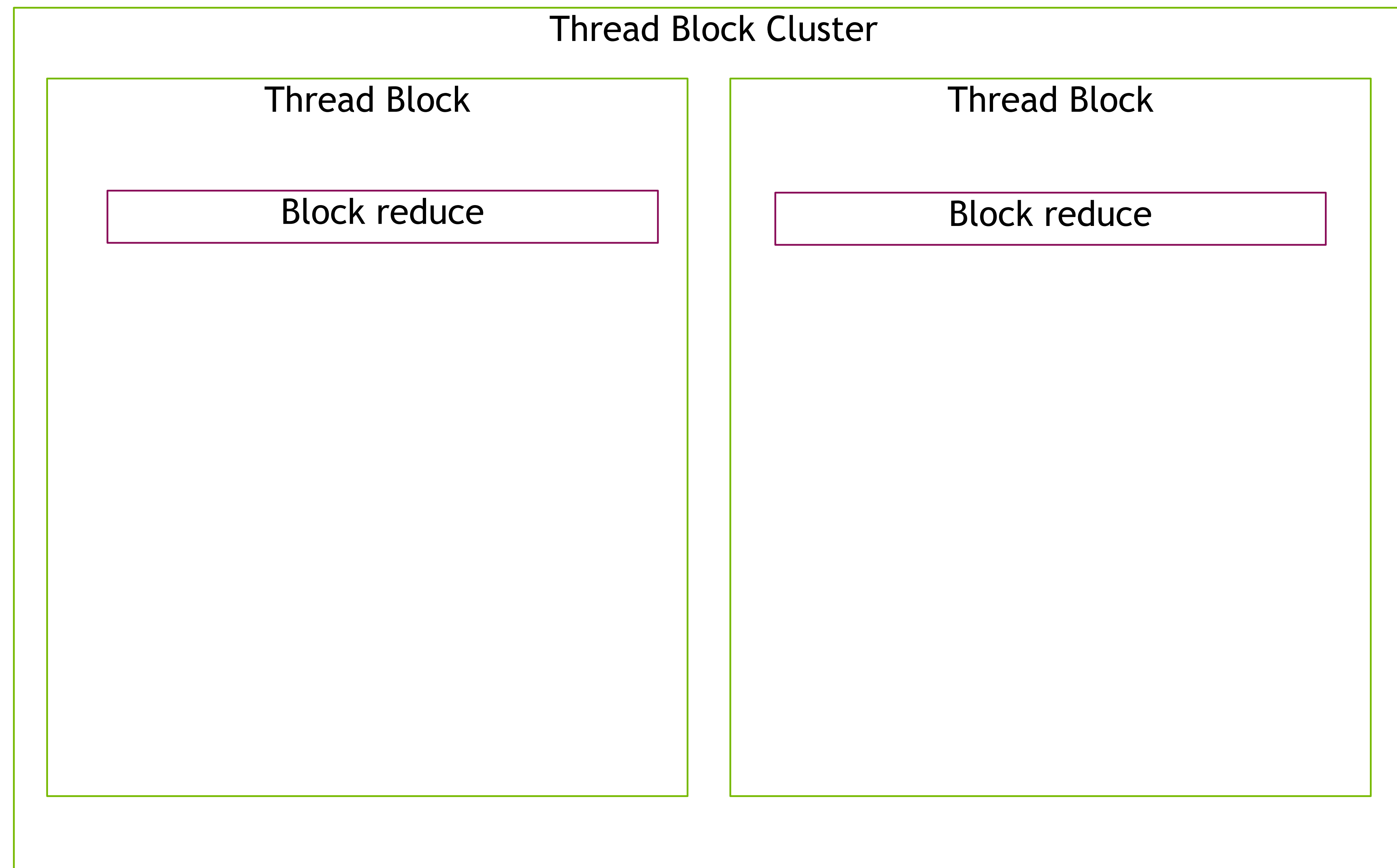


SMO solve working set



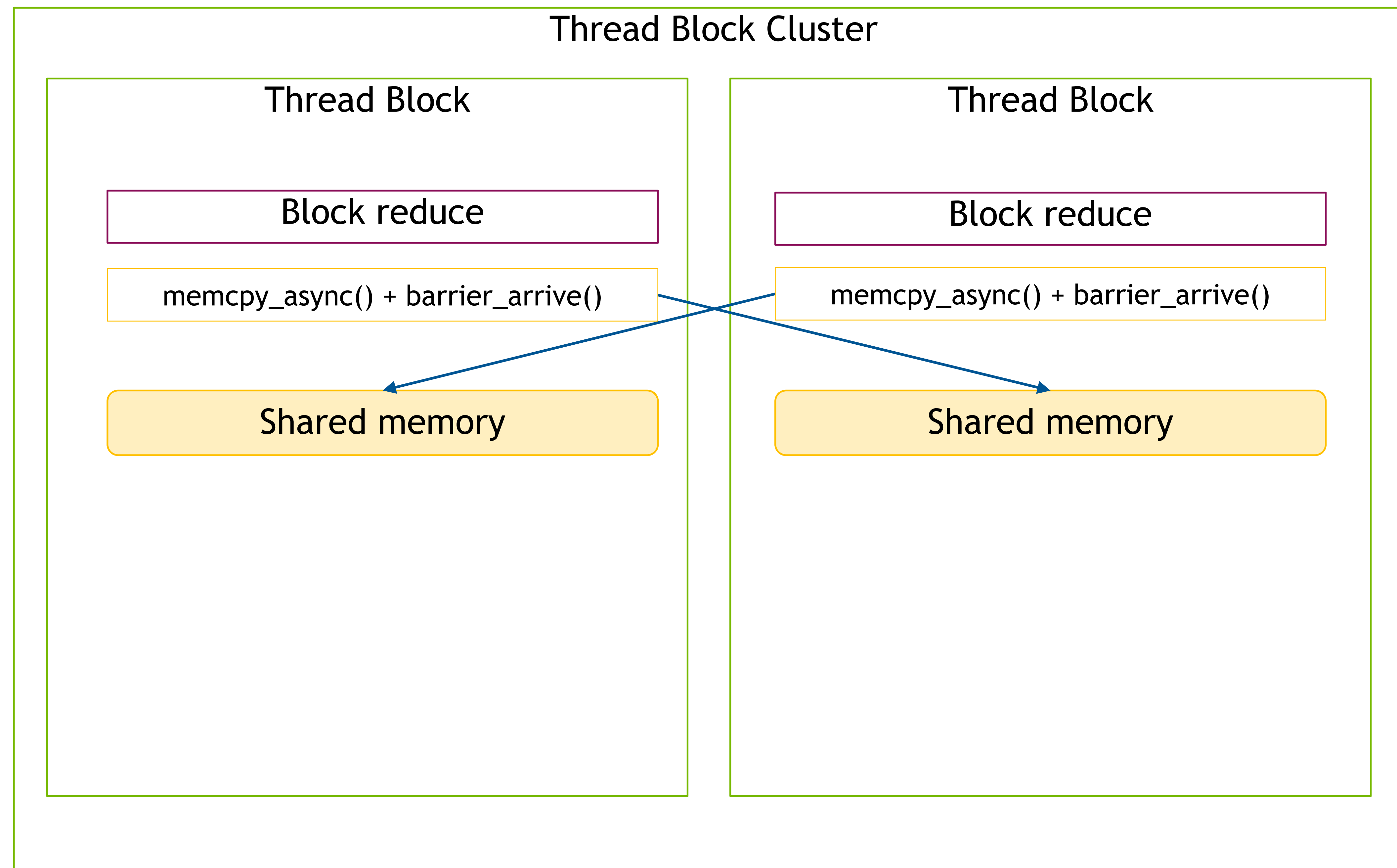
Support Vector Machine (SVM)

One sided communication with Barriers



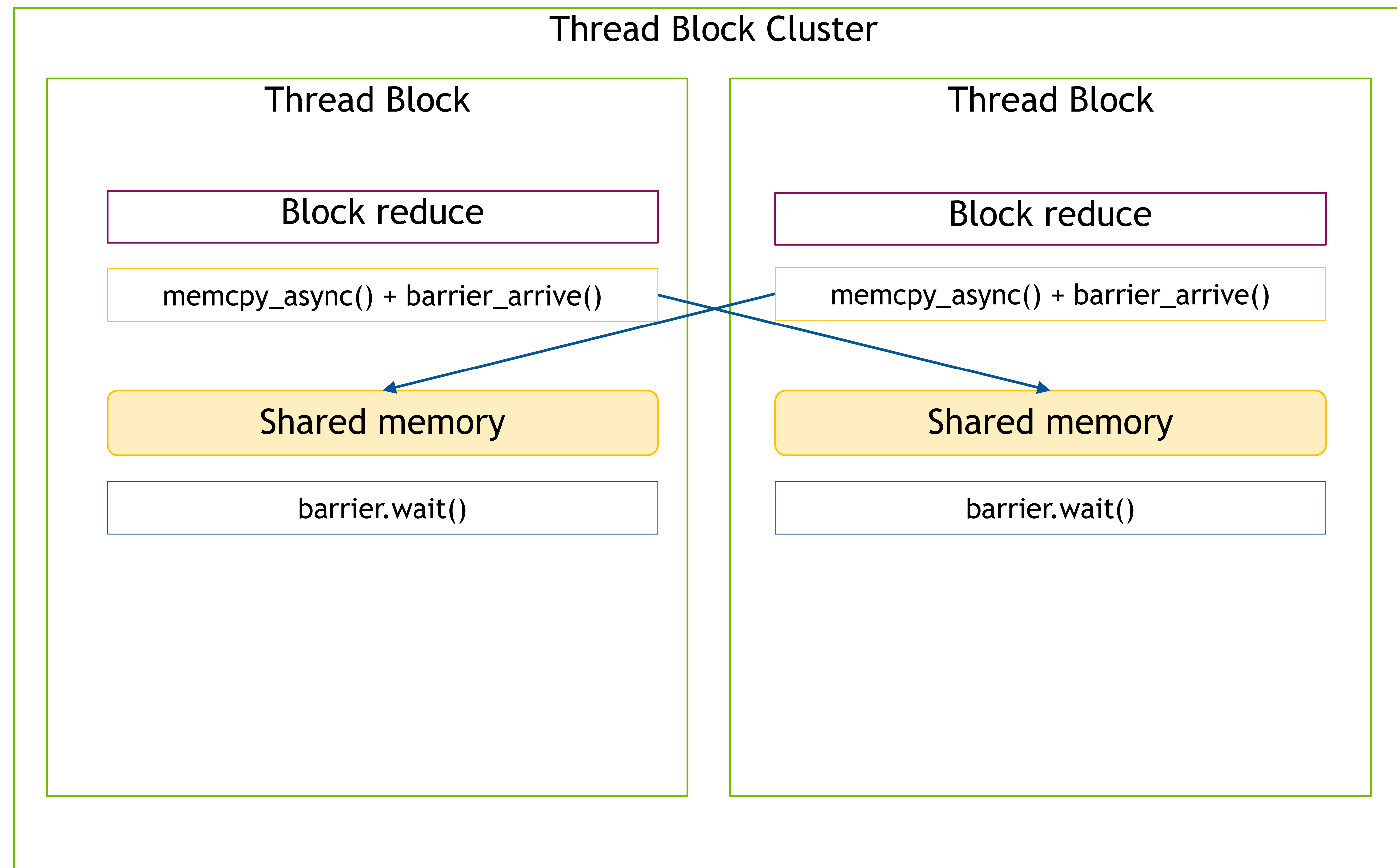
Support Vector Machine (SVM)

One sided communication with Barriers



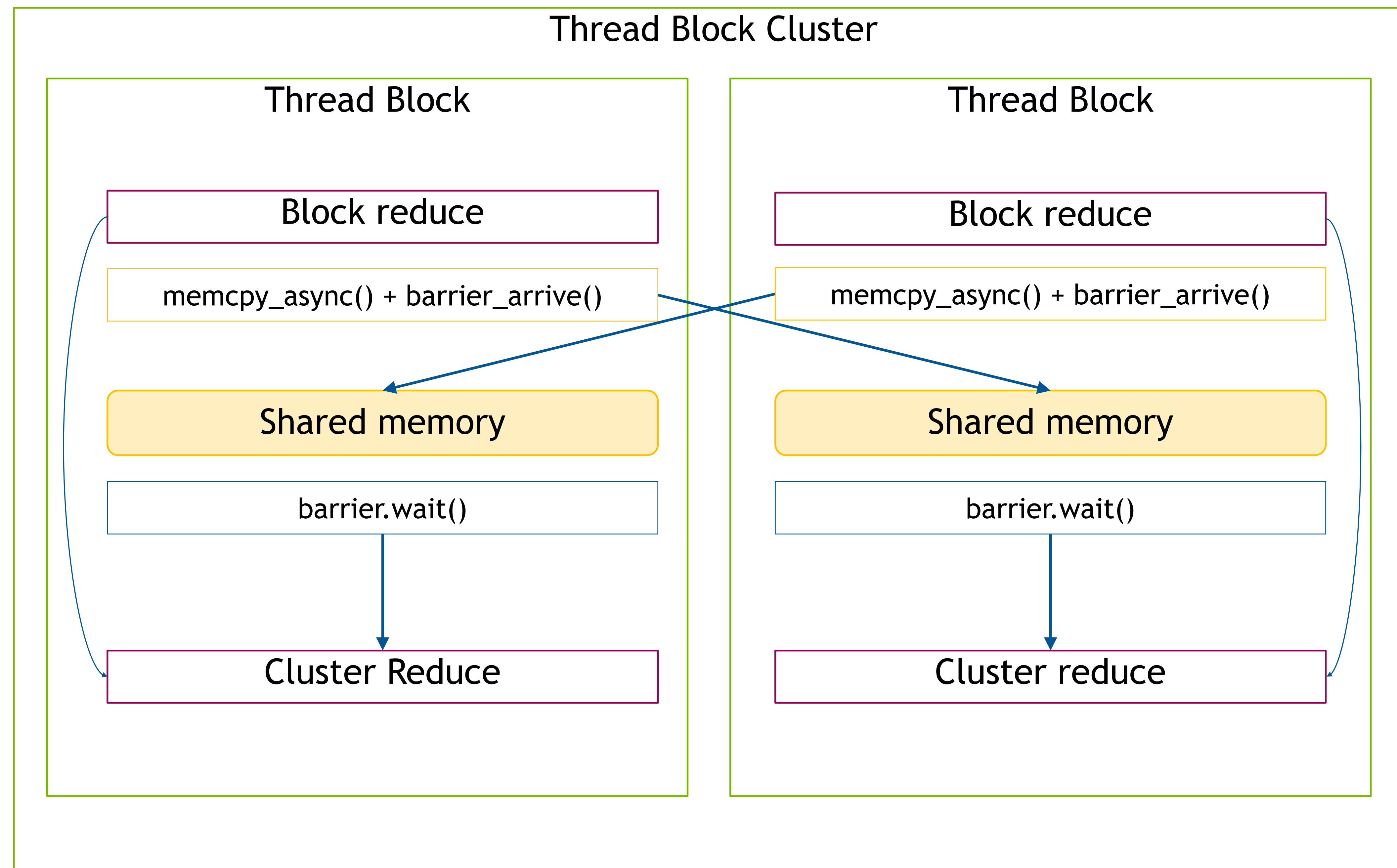
Support Vector Machine (SVM)

One sided communication with Barriers



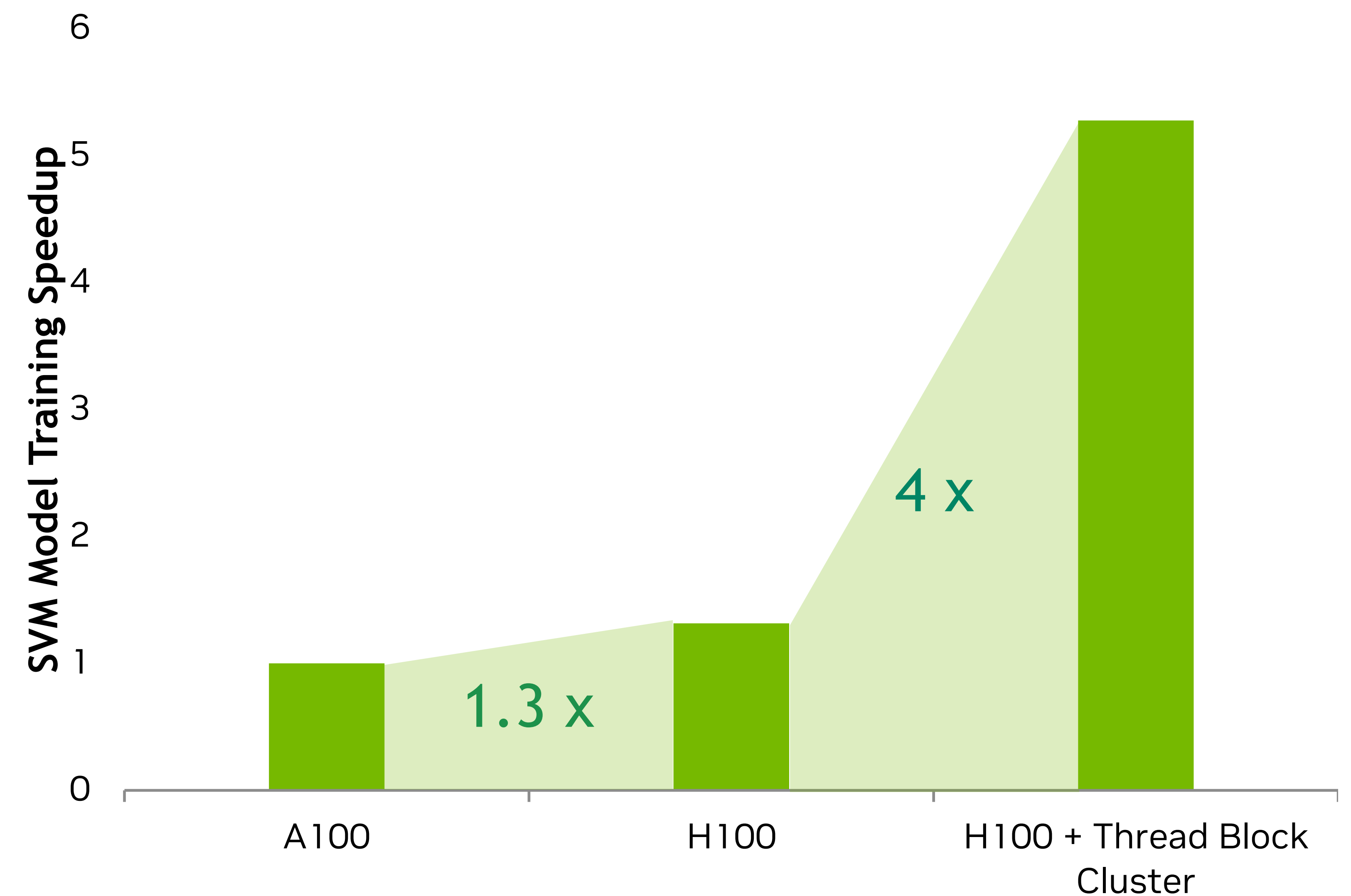
Support Vector Machine (SVM)

One sided communication with Barriers



Support Vector Machine (SVM)

5.3x Performance A100 vs H100

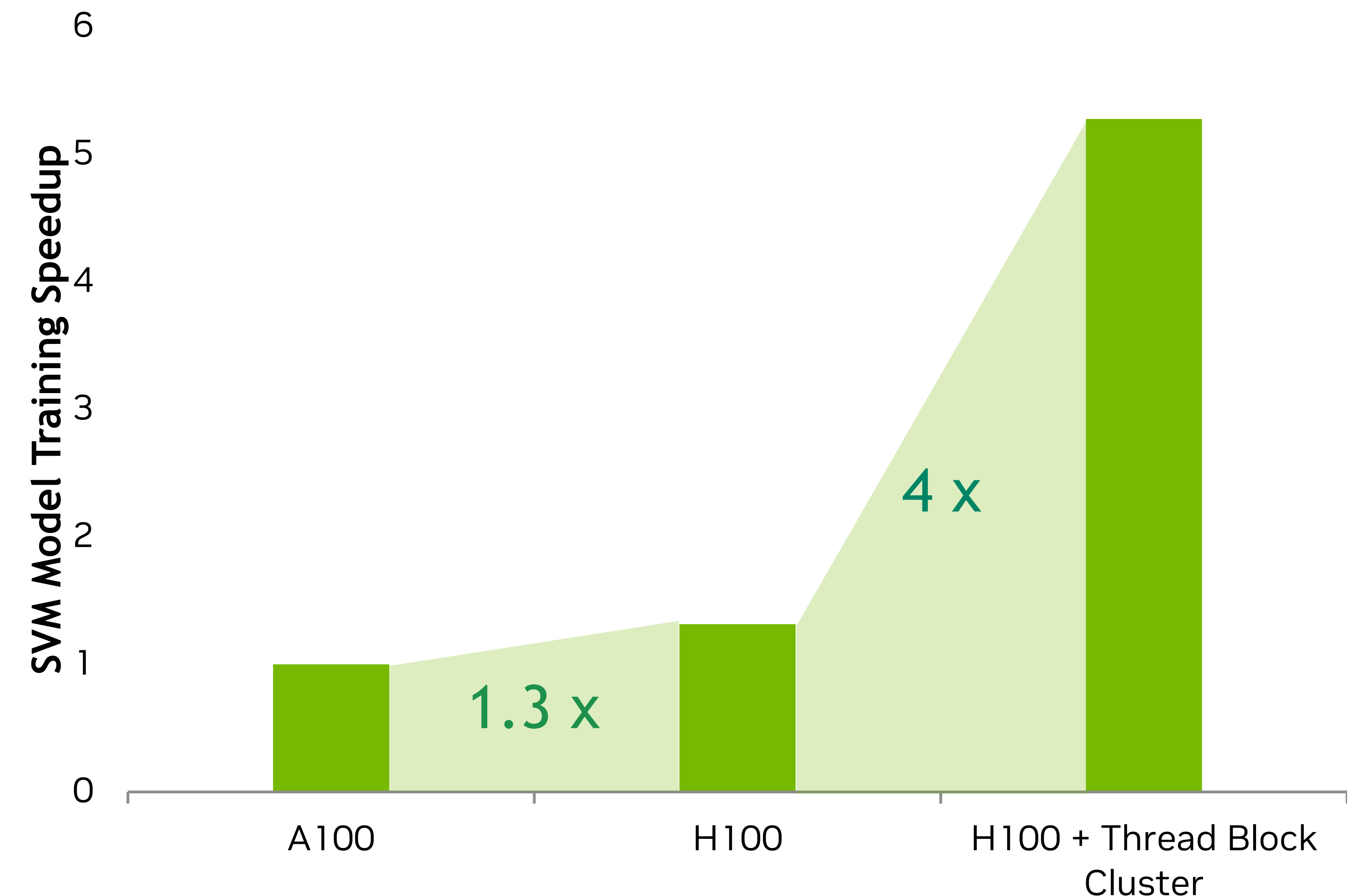


A9A dataset (preprocessed version of the UCI Adult data set)
39k samples, 123 features RBF kernel, C=500, gamma=1/123

Support Vector Machine (SVM)

5.3x Performance A100 vs H100

- Using thread block cluster, the working set size of the inner solver can be increased, which in turn requires fewer outer solver iterations.
- Thread Block Clusters improve numerical efficiency of the algorithm
- Performance gain of 5.3x between A100 and H100
 - Fewer kernel function evaluations & outer solver iterations
 - Low latency communication between thread blocks

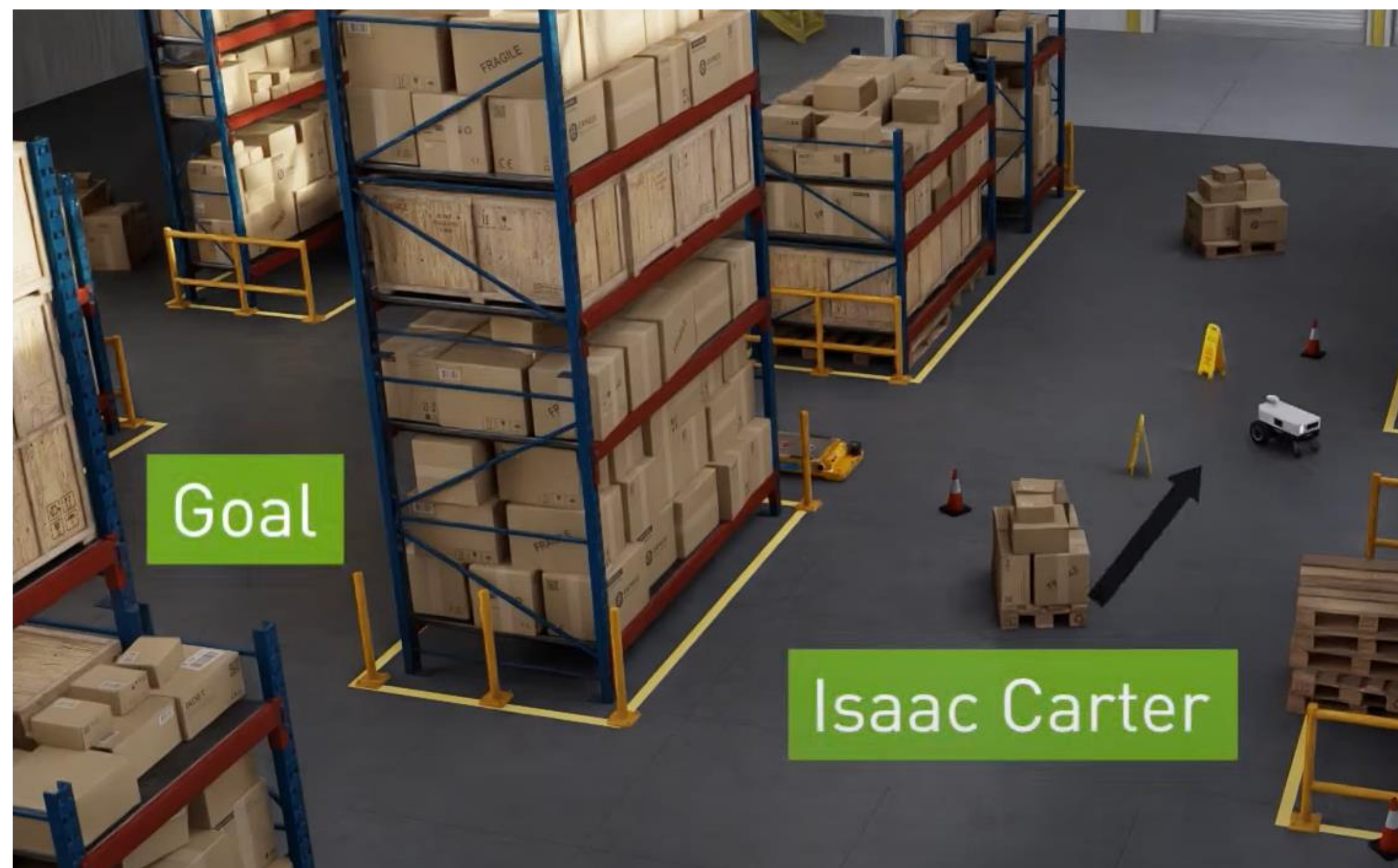


A9A dataset (preprocessed version of the [UCI Adult data set](#))
39k samples, 123 features RBF kernel, C=500, gamma=1/123

The background features a complex pattern of bright green lines and shapes on a black field. On the left, numerous thin, parallel green lines radiate outwards. On the right, there are larger, more intricate structures resembling stylized leaves or overlapping rectangular frames, also composed of green lines.

Dynamic Programming in Hopper

Dynamic Programming Applications



Robotic Path Planning
Issac Sim + nvblox



CLARA Genomics

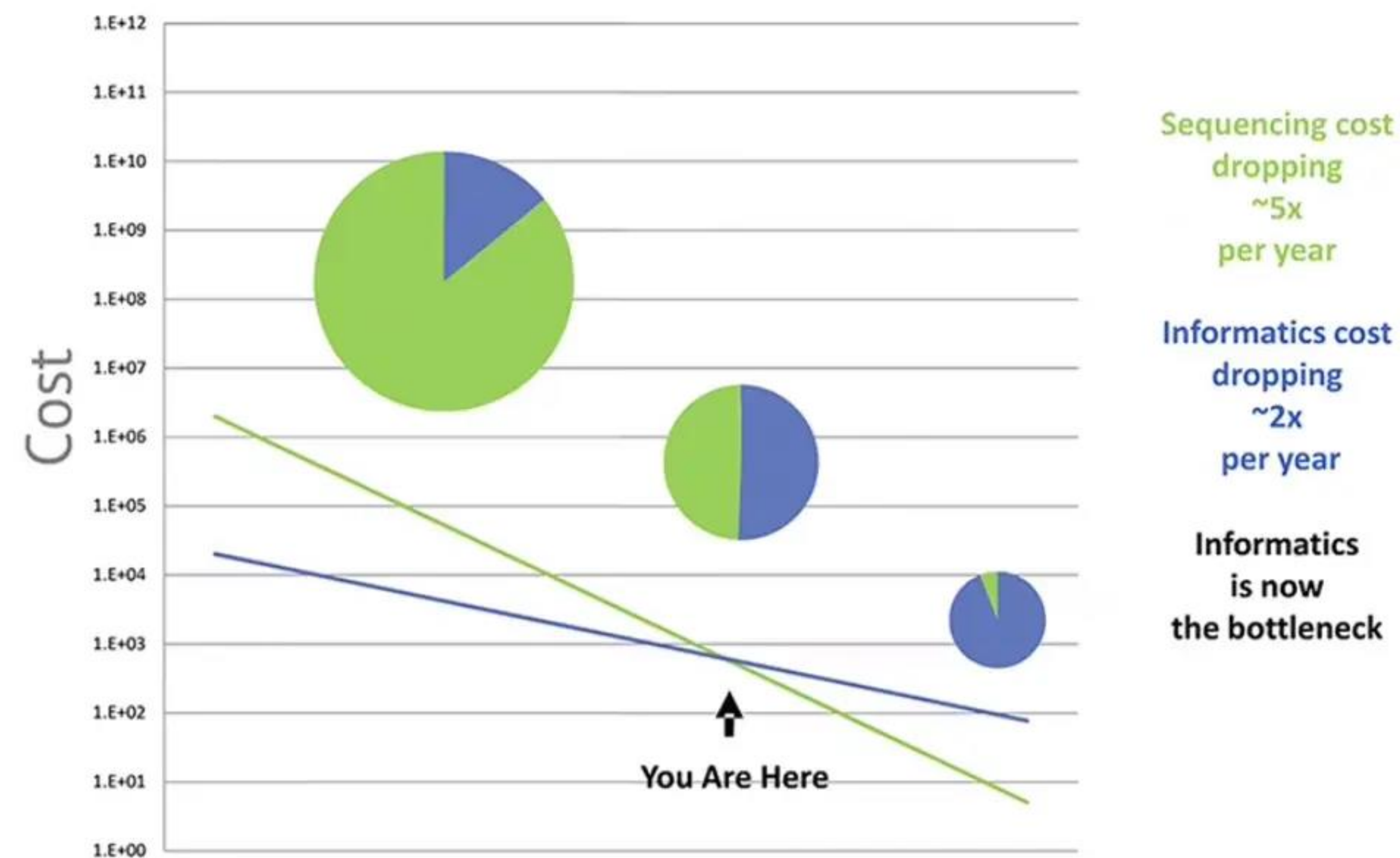


Large scale Fleet Routing
cuOPT

Dynamic Programming

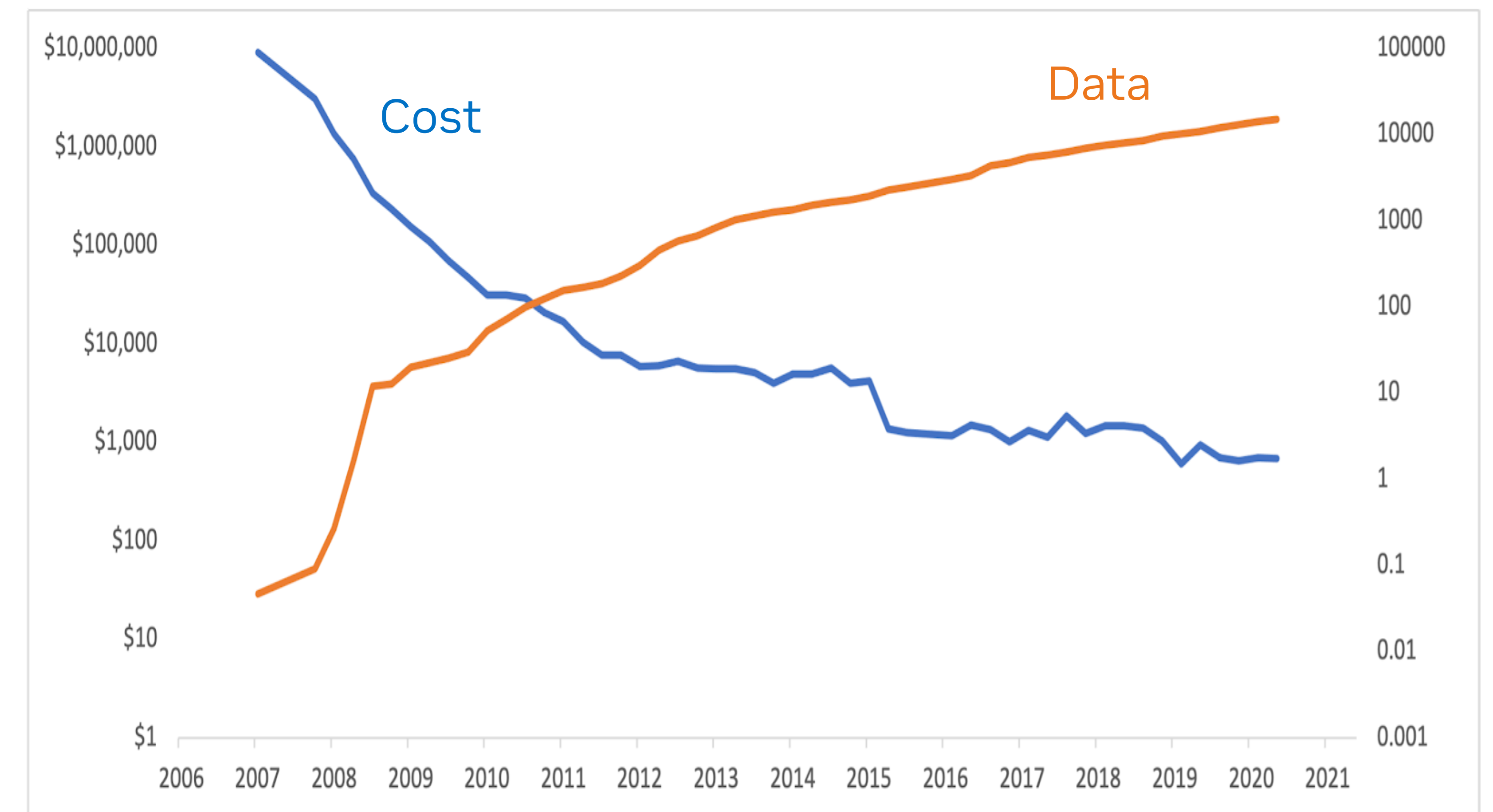
Why accelerating Sequence alignment matters?

DNA Sequencing Economics



Cost of **sequencing** dropping twice as fast as **informatics**

Source: <https://www.businessinsider.com.au/super-cheap-genome-sequencing>



1000 Genomes
Project launches
(2008)

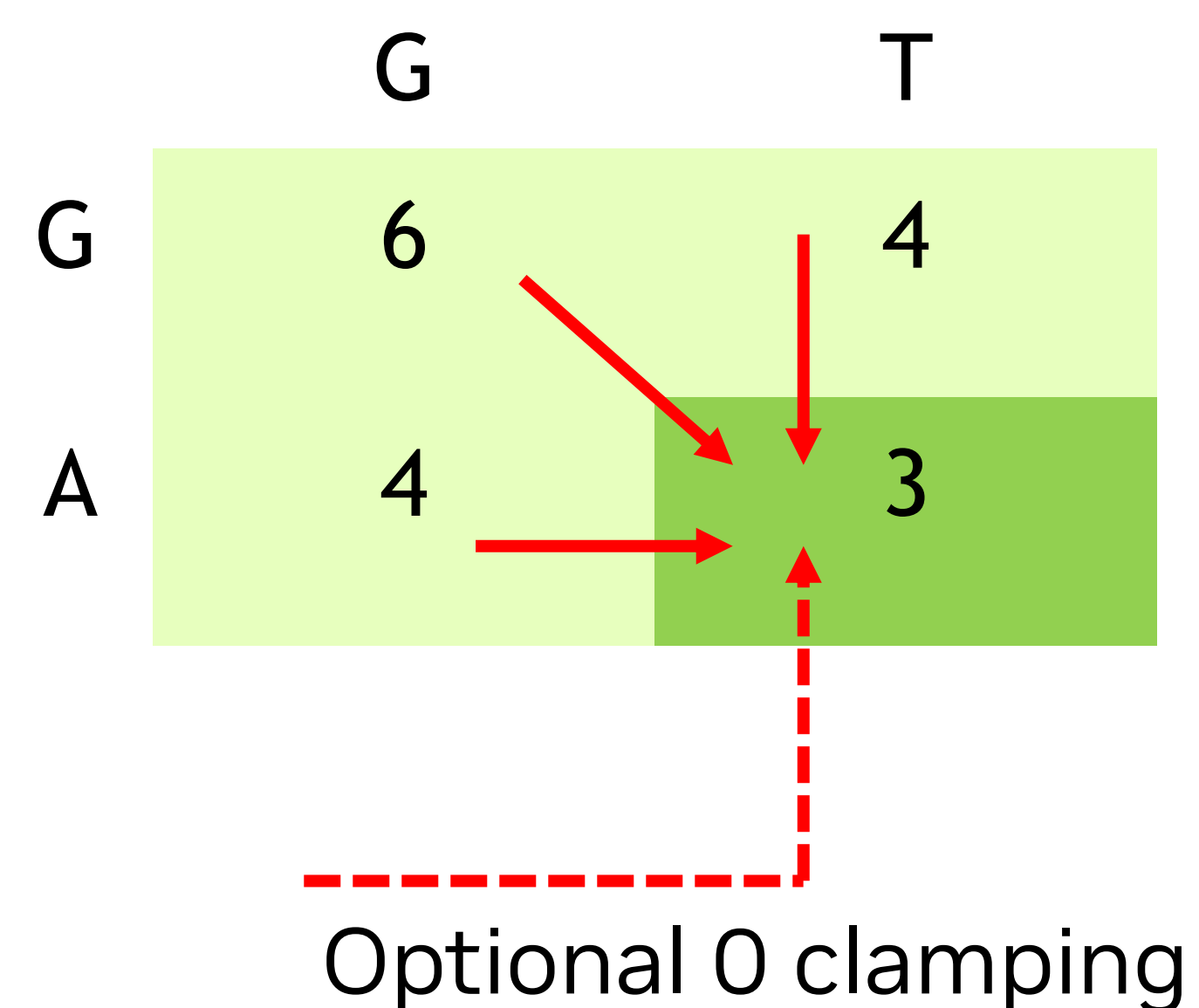
100,000 Genomes
Project launches
(2013)

1,000,000 Genomes
Project launches
(2018)

Dynamic Programming

Sequence Alignment Smith Waterman Algorithm

new value = max (diagonal cell - 3 // mis-match cost
 diagonal cell + 3, // match reward
 vertical cell - 2, // insertion cost
 horizontal cell - 2, // deletion cost
 0) // clamp to a min of zero



Inputs	Output

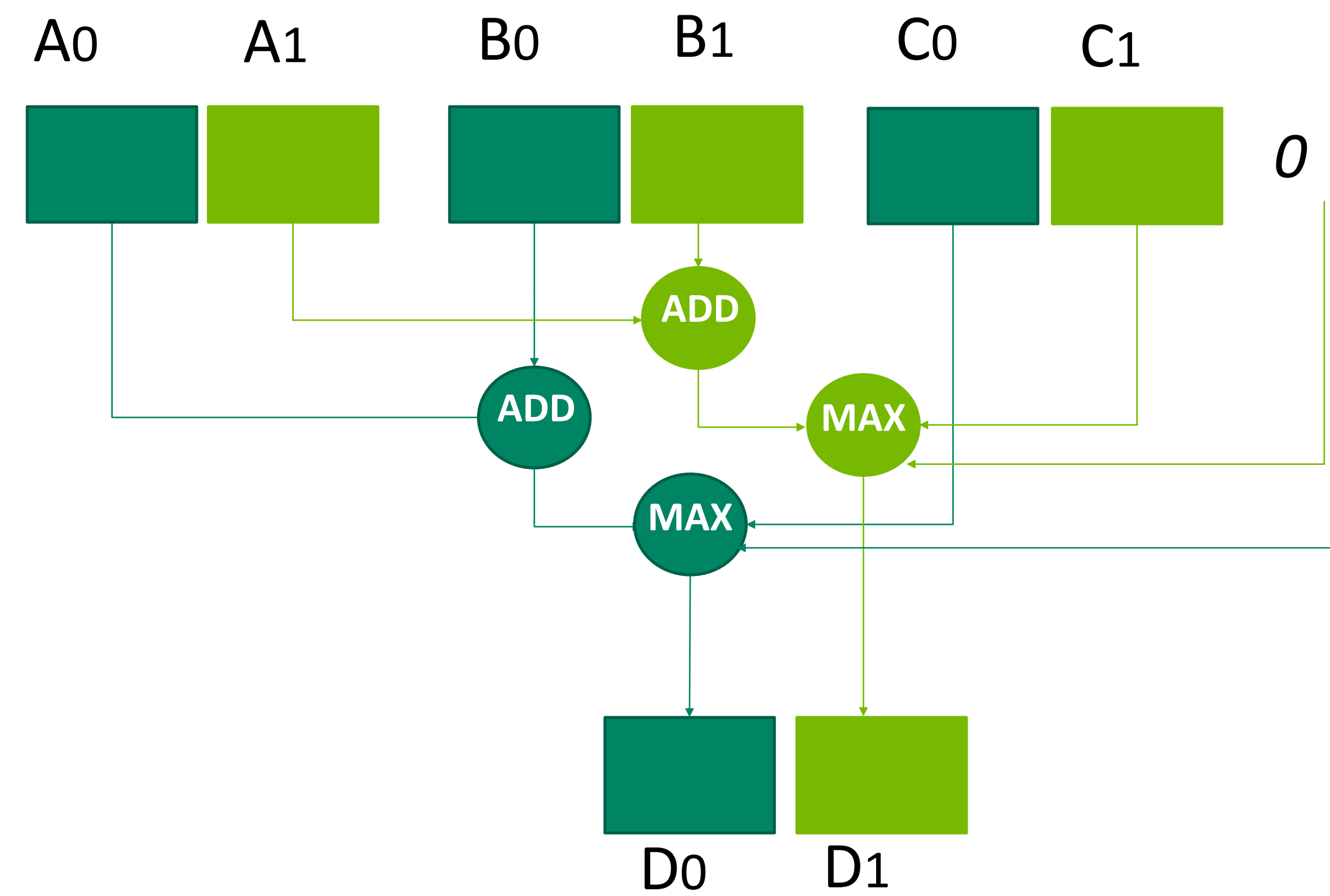
Simplified calculations.

	T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0
G	0	0	3	1				
G	0	0	3					
T	0	3						
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

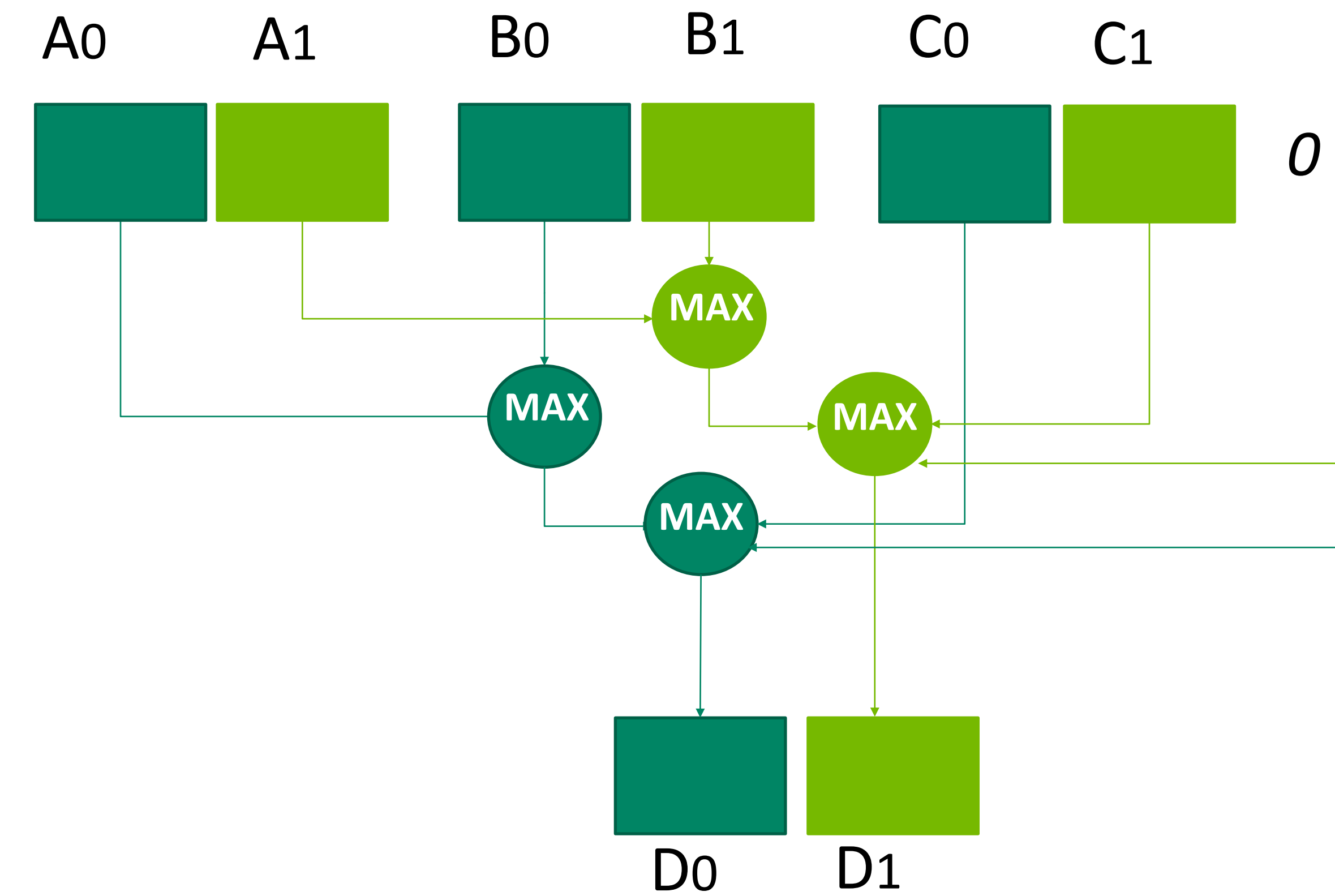
Dynamic Programming

CUDA Math functions for dynamic Programming

$D = \text{MAX}(A+B, C, 0)$ INT32/16 datatype. Optional 0 clamping

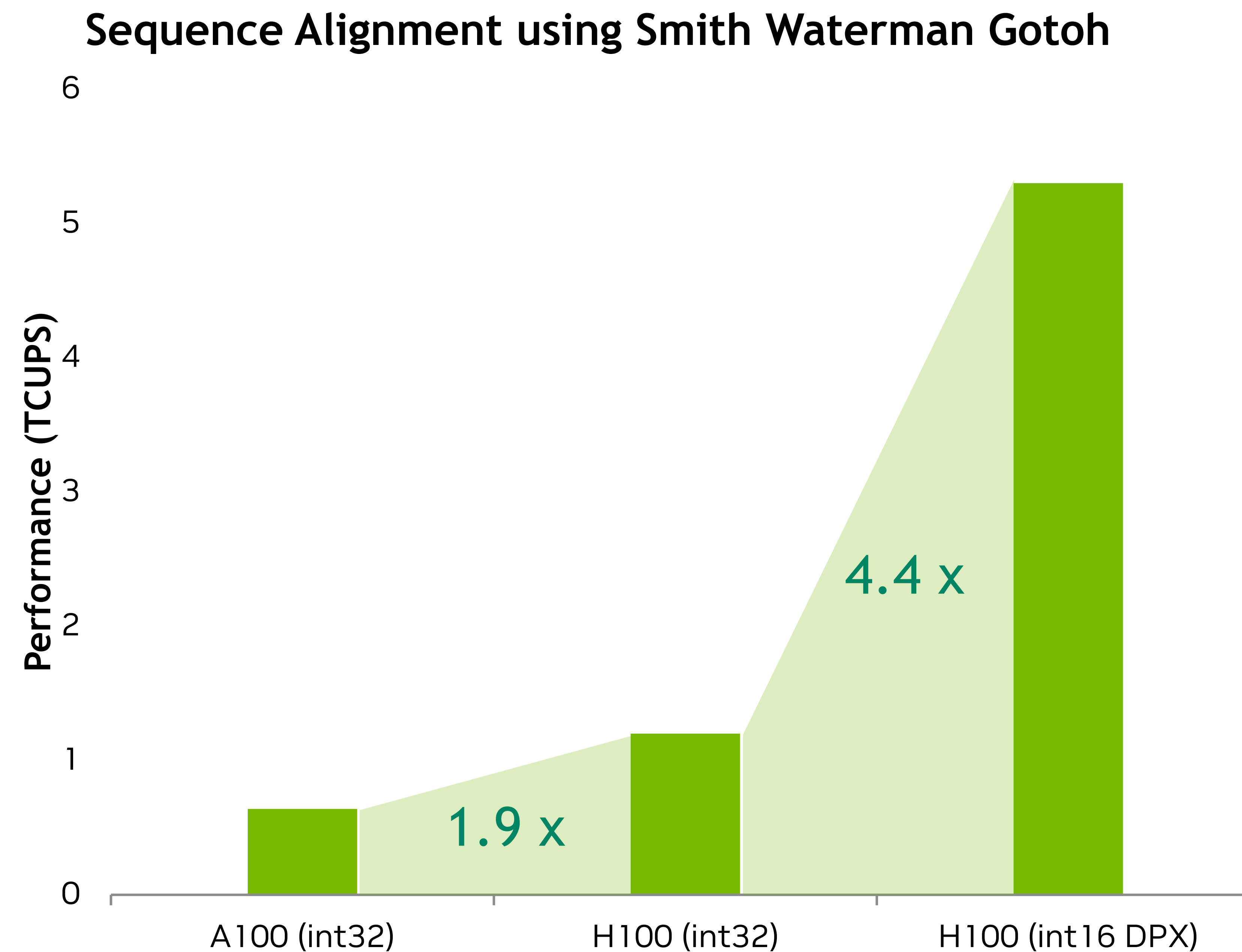


$D = \text{MAX}(A, B, C, 0)$ INT32/16 datatype. Optional 0 clamping



Dynamic Programming

Accelerating Genomics in H100 by over 8x compared to A100



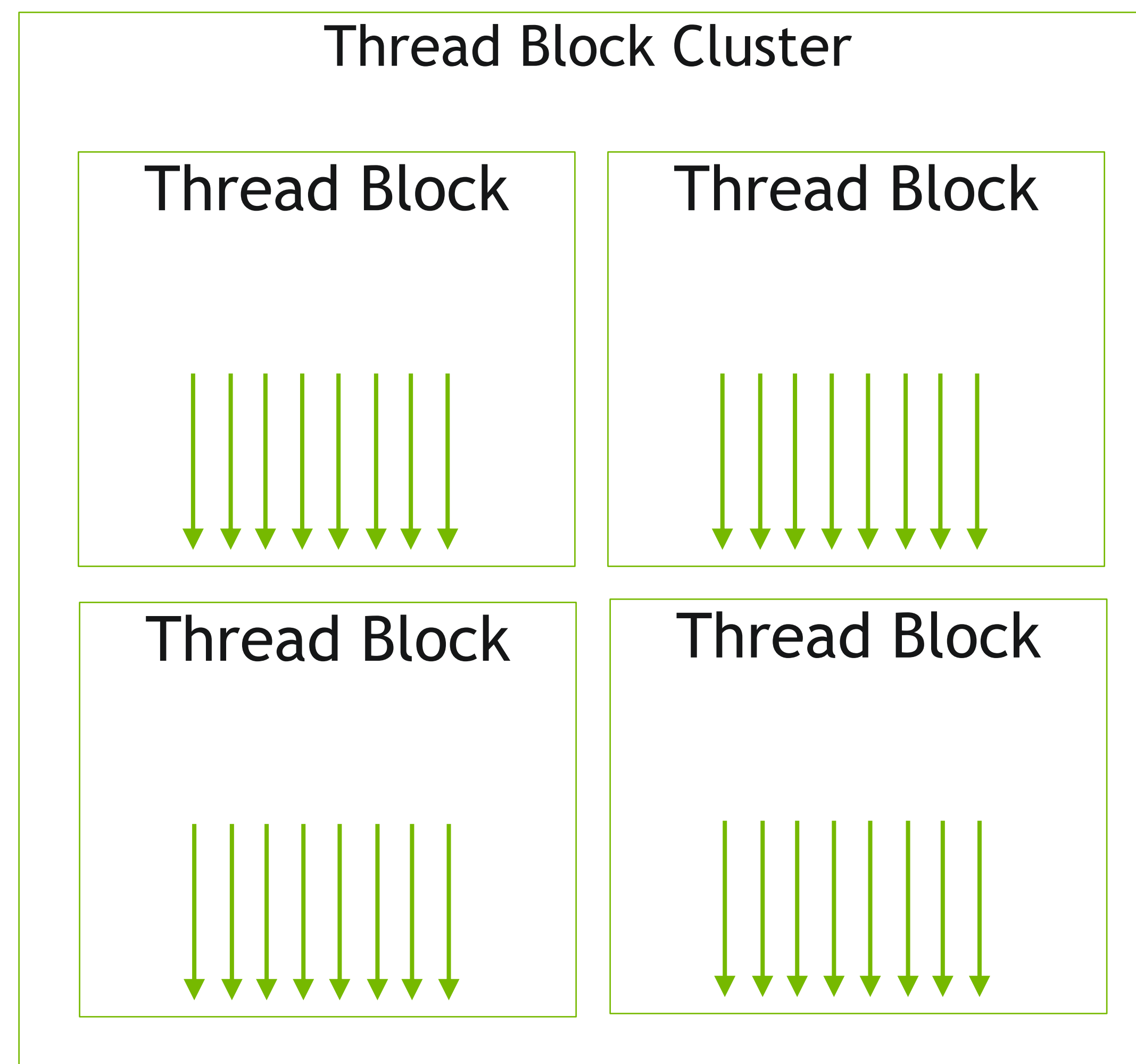
Affine gap alignment (score calculation)

Weights used from BWA.

Data: [HG002 \(NA24385\)](#) pair-end protocol using Illumina Sequencers.

CUDA Programming Model for Hopper

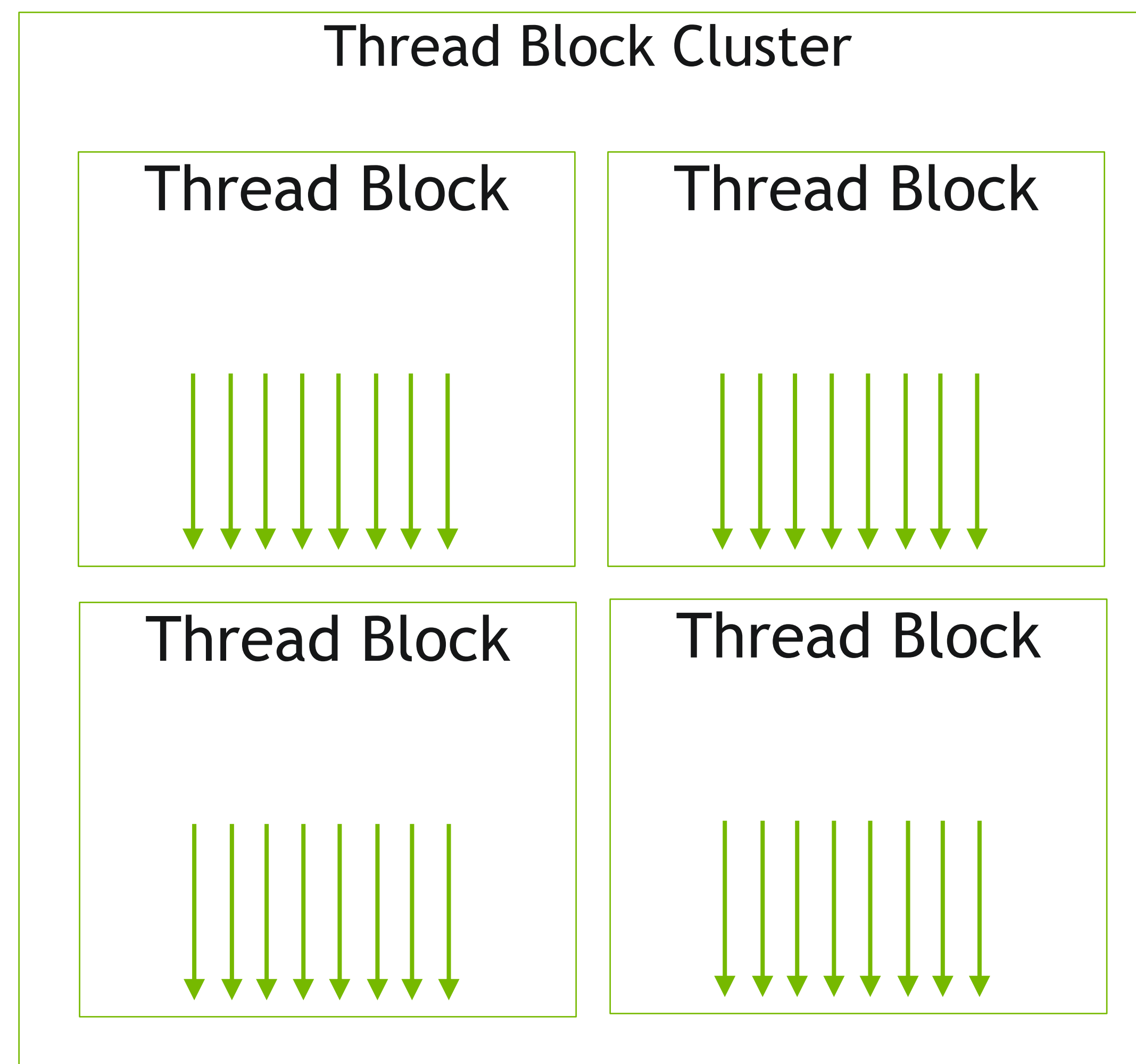
How to write more scalable
algorithms ?



Enables More Localized
Algorithms

CUDA Programming Model for Hopper

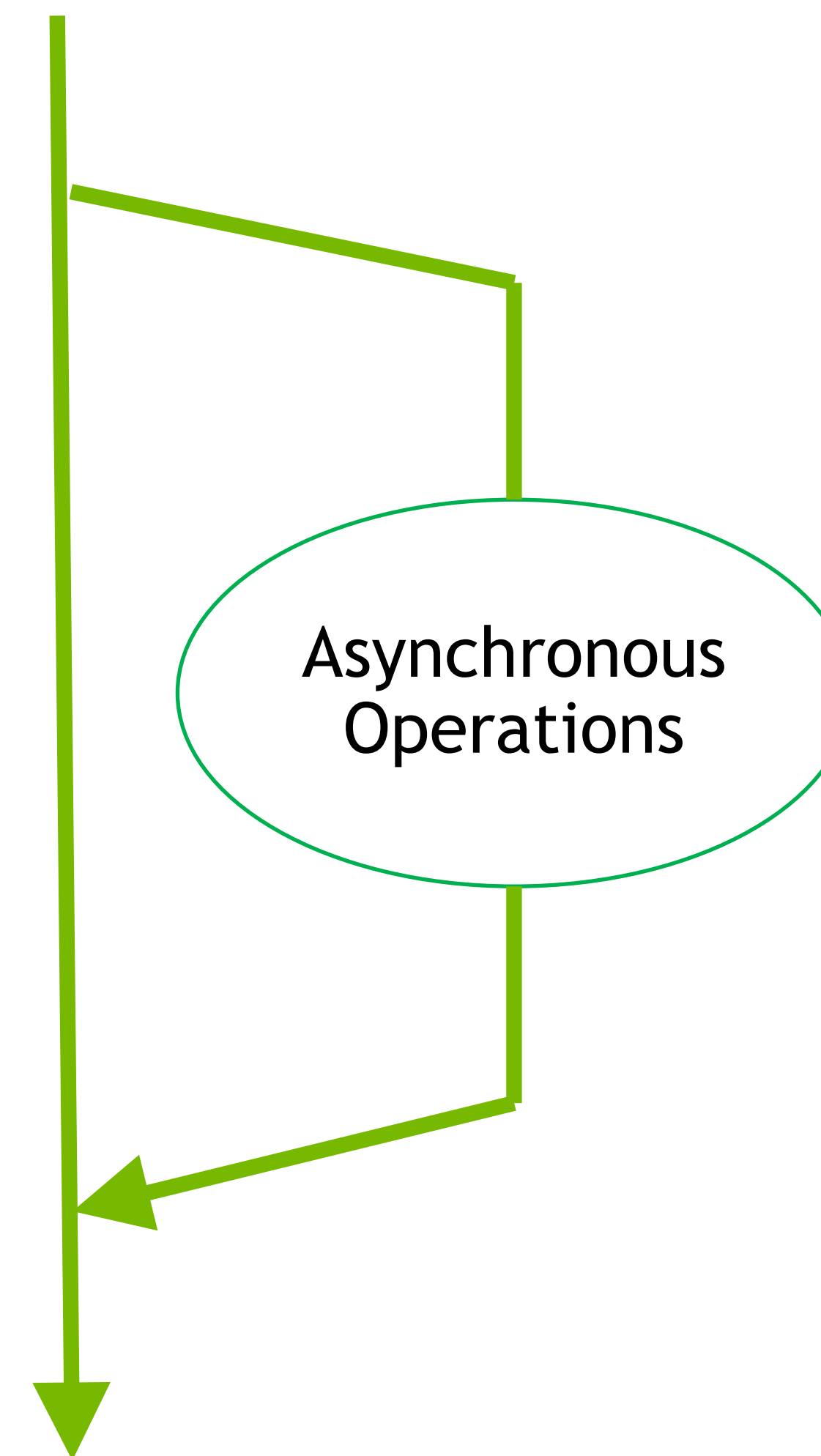
How to write more scalable algorithms ?



Enables More Localized Algorithms

How to keep all GPU units busy ?

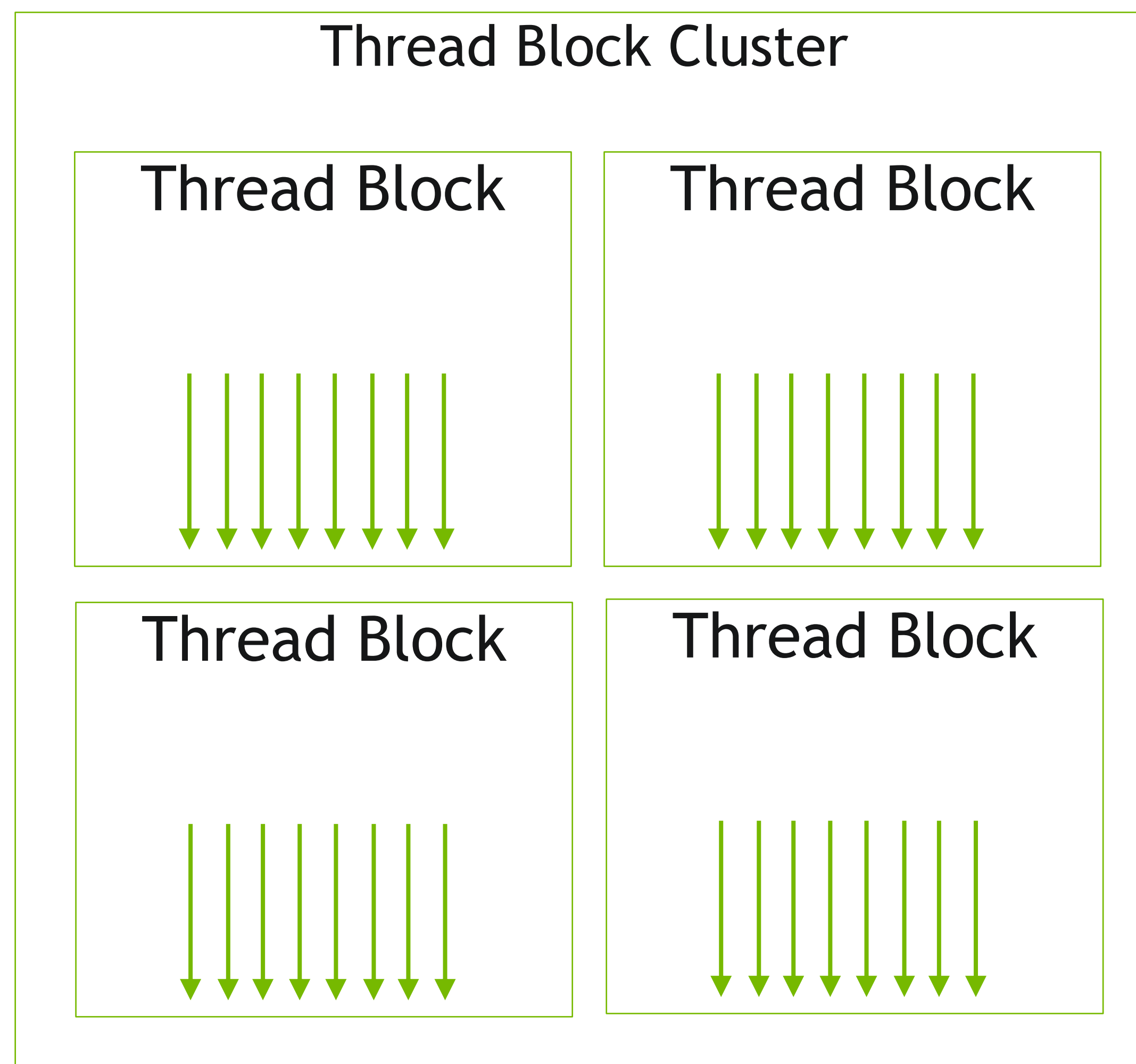
CUDA thread



Overlapping Compute and Data Operations

CUDA Programming Model for Hopper

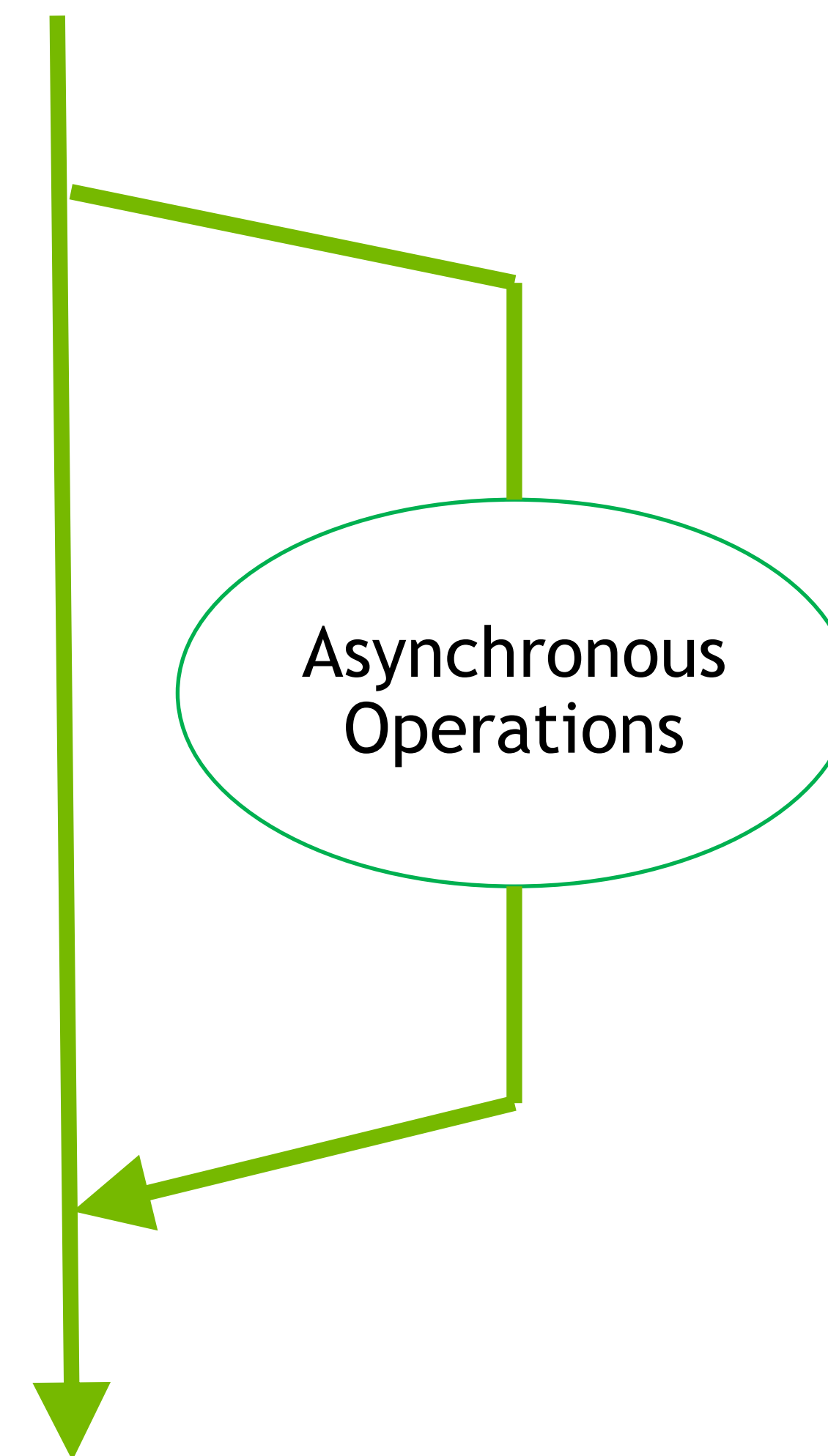
How to write more scalable algorithms ?



Enables More Localized Algorithms

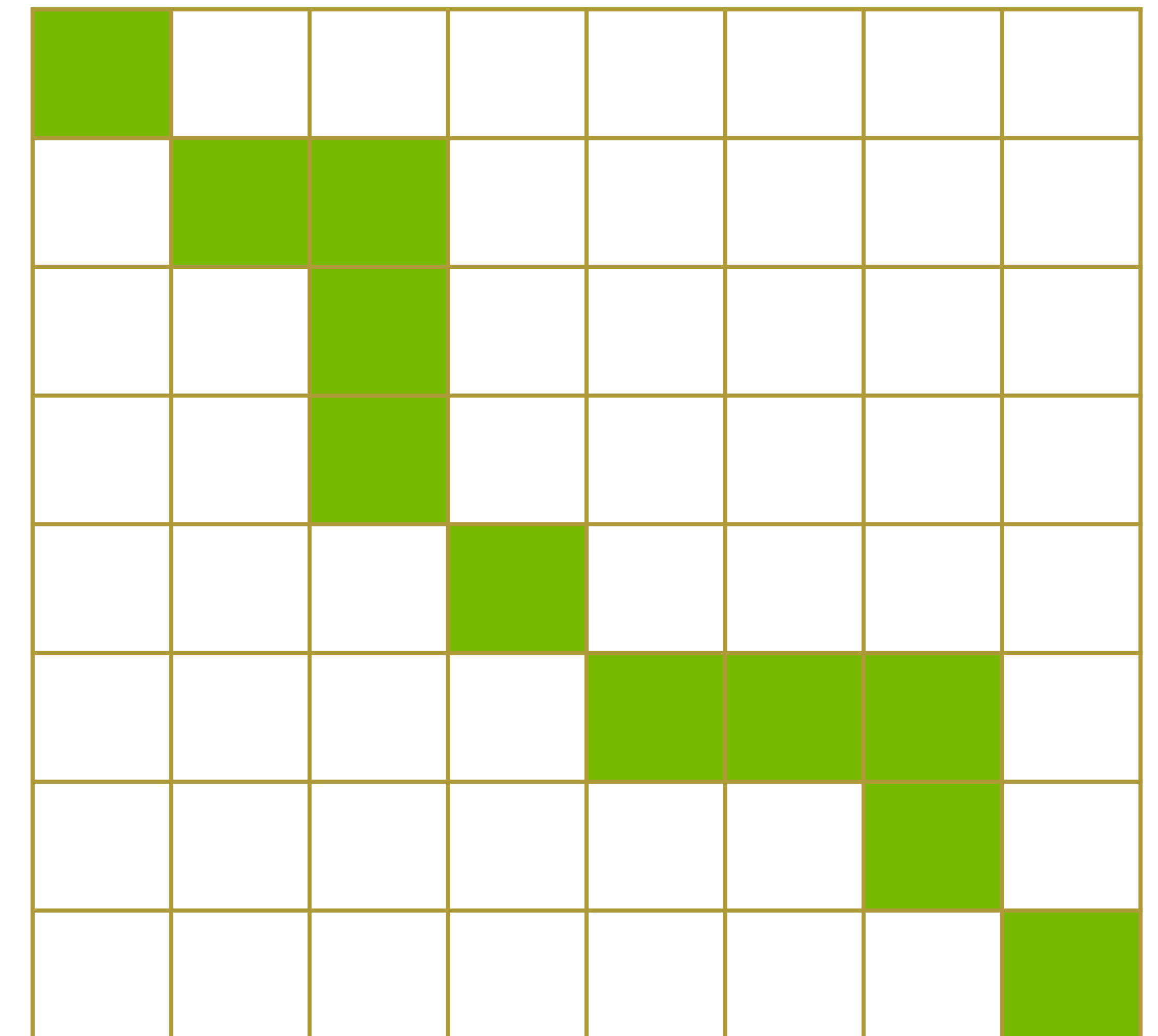
How to keep all GPU units busy ?

CUDA thread



Overlapping Compute and Data Operations

How to accelerate computations in dynamic programming ?



Enabling recursive Dynamic Programming Algorithms



Thank You